NETWORK SYNCHRONIZATION • FROM LOCKSTEP TO REPLICATION

网络同步全景
从 HOK 帧同步到 Unreal 状态同步

以《HOK 帧同步机制详解》为主线，把帧同步（Lockstep / 定频广播 / 回滚）与状态同步（快照插值 / 客户端预测 / 延迟补偿 / Unreal 复制系统）串成一条完整的知识脉络；每个关键结论都配一个 C++ 实测，每章收尾给面试追问。

15 章 + 4 附录 | HOK 帧同步全链路 | Unreal 复制 / RPC / CMC / GAS 预测

12 组网络实测 + 7 组确定性实测（3 种编译选项） | 秋招 · 游戏客户端

PROLOGUE

导读：一张图看懂网络同步

先搞清楚这本书在讲什么、按什么顺序读、每个结论的可信度怎么标。

CHAPTER 00

导读：知识地图、阅读路线与事实等级

网络同步只回答三个问题：谁说了算、网上传什么、什么时候往前走。帧同步和状态同步是这三个问题的两组不同答案，其余的技术都是在补各自的短板。

本章目标

- 记住网络同步的“三问”，用它给任何一种方案定位。
- 看懂全书知识地图：HOK 帧同步一条线、Unreal 状态同步一条线、两条线在“预测 + 回滚”处汇合。
- 知道书里每个结论的来源：HOK 文档、UE 源码 / 官方文档、公开资料、本书实测、推断。

30 秒说明：这本书怎么用

你给的《HOK 帧同步机制详解》已经把王者荣耀的帧同步讲得很完整：**服务器定帧汇总 + 三层冗余抗丢包 + 900 帧滑动窗口定序 + 本地确定性模拟 + 逻辑与表现彻底解耦。**这本书做三件事：

1. **补原理**：为什么这么设计？不这么设计会怎样？每个设计点都用一个可运行的 C++ 模型量化（例如“一包带 3 帧冗余能挡住多少丢包”“严格 Lockstep 下一个人断流全场卡多久”）。
2. **补另一半**：状态同步（尤其是 Unreal）——快照插值、客户端预测与和解、延迟补偿、Actor 复制管线、属性复制与 RPC、CharacterMovementComponent、GAS 预测、Iris / Mover / 网络物理。
3. **补面试**：每章都有 30 秒答案、面试注意、坑位清单和自测；第 15 章汇总 50 道追问。

0.1 网络同步的三问

问题	帧同步（Lockstep 家族）	状态同步（权威服务器家族）
谁说了算（权威在哪）	没有“权威状态”：每个客户端都跑完整逻辑，大家算得一样就是真相；服务器只定序	服务器的状态就是真相；客户端只是“播放器”，最多做预测
传什么	输入（第 N 帧谁按了什么键）	状态（位置、血量、技能状态）+ 事件（RPC）
何时推进	拿到第 N 帧的全部输入才推进（或先猜着推进，错了回滚）	服务器按自己的 tick 推进；客户端按收到的快照插值，按本地输入预测
最难的事	确定性、抗丢包、断线追帧	带宽、手感（延迟）、命中判定公平

0.2 知识地图

全书分四部分。Part II 以 HOK 为主线讲帧同步，Part III 以 Unreal 为主线讲状态同步；第 9 章的“回滚”和第 10、13 章的“预测 + 和解”其实是同一个思想在两种架构里的投影——这是两条线的汇合点。



图 0-1 全书知识地图。★ 三章是两条线的汇合点：帧同步里的“回滚”和状态同步里的“预测 + 和解”本质相同——先按本地猜测推进，权威结果到达后回到分歧点重算。

0.3 建议阅读路线

路线 A • 面试速通（半天）

Ch1 → Ch3（三条路线 + 实测）→ Ch6 / Ch7 前两节 → Ch10 → Ch11 / Ch12 的 30 秒答案 → Ch14 对照表 → Ch15 追问。

路线 B • 吃透 HOK（1~2 天）

Part II 顺序读，配合原文档；每章末尾的“对照原文档”看补充了什么。附录 A 是协议和参数速查，附录 D 是勘误。

路线 C • 吃透 UE（1~2 天）

Ch2 → Ch10 → Ch11 → Ch12 → Ch13 → Ch14 第 5 节（在 UE 里做帧同步）。附录 B 是 API / 宏 / CVar 速查。

0.4 事实等级标签

网络同步是谣言重灾区（“帧同步不能防作弊”“状态同步天然防作弊”“回滚只影响自己”……）。为了让你答题时心里有底，书里的关键结论都带来源标签：

标签	含义	可信度
HOK 文档	来自你提供的《HOK 帧同步机制详解》（协议号、参数、类名、流程）	项目事实；对外面试时只讲机制，不讲内部细节
UE 源码 / 文档	Unreal Engine 源码与官方文档中的类、函数、宏、CVar 与默认值	引擎事实；随版本变化的地方会标出版本
公开资料	GDC 演讲、官方技术博客、开源库（GGPO、Valve 开发者文档、Riot 技术博客、Gaffer On Games 等）	业界事实；只引用能查证的结论
实测 Ex	本书 `demos/` 下 C++ 模型的输出（数字由构建脚本从 JSON 读入，书和 md 同源）	模型结论；参数是假设的，趋势与量级可信，绝对值不代表线上
推断	由以上事实推出、但没有直接证据的结论	需要你在项目里验证

0.5 实测程序一览

编号	实验	章节
E1	带宽模型：只传输入 vs 全量快照 vs 增量快照 vs 增量 + 兴趣范围（AOI）	Ch1
E2	15 Hz 逻辑、60 fps 渲染：直接画逻辑状态 / 插值 / 外推	Ch2
E3	10 人对局、一人断流 2 秒：严格 Lockstep / 定频广播不等人 / 回滚，逐帧时间线	Ch3
D1–D7	同一份源码用 `/fp:precise`、`/fp:fast /arch:AVX2`、`/fp:strict /Od` 编译：求和顺序、群体模拟逐帧 hash、Q16.16 定点、1 ulp 蝴蝶效应、unordered_map 顺序、浮点累加时间	Ch4
E5	下行一包带 k 帧冗余：随机丢包 vs 突发丢包下仍需补发的比例（附闭式解）	Ch6
E7	上行技能命令：单发 / 短冗余 / 全量冗余 + 序号	Ch7
E8	ping：主线程取包时刻 vs 网络线程到达时间戳	Ch8
E9	格斗游戏 2 人 60 fps：回滚深度 / 次数 与 输入延迟 d 的权衡，对照纯延迟型	Ch9
E10	快照插值：插值延迟 × 网络抖动 → 外推比例与误差	Ch10
E11	客户端预测：不预测 / 预测 + 直接覆盖 / 预测 + 和解（重放未确认移动）	Ch10 · Ch13
E12	延迟补偿：不补偿 / 服务器回溯 / 回溯设上限	Ch10
E13	带宽饱和时的复制调度：UE 式“优先级 × 等待时间” / 静态优先级 / 轮询	Ch11
E14	属性复制（状态，最终一致）vs 可靠 / 不可靠 RPC（事件）	Ch12

关于 HOK 内容的使用边界

Part II 的 HOK 细节（协议号、类名、参数）来自内部文档，适合用来**理解**和在内部讨论；对外面试时建议只讲机制（“定频广播不等人” “冗余 + 补发 + 追帧三层兜底” “逻辑表现分离”），不要背协议号和内部类名。

本章速记

- 三问：**谁说了算 / 传什么 / 何时推进**。帧同步 = 无权威状态 + 传输入 + 拿到输入才推进；状态同步 = 服务器权威 + 传状态 + 服务器自己推进。
- 两条线的汇合点：**预测 + 回滚重算**（GGPO 的回滚 = UE CMC 的“纠正后重放 SavedMoves”）。
- 每个结论看标签：`实测` 给趋势和量级，`UE` / `公开资料` 给事实。

PART I

原理篇：同步到底在同步什么

先把问题定义清楚：一组互不信任、网络各异、没有全局时钟的设备，如何对一段持续演进的战斗达成共识。

CHAPTER 01

同步问题与两大流派

状态同步“服务器算、客户端演”，帧同步“大家都算、服务器定序”。两者没有高下，只是把难题放在了不同的地方。

本章目标

- 能用“三问”说清状态同步和帧同步的本质区别，而不是背“带宽低 / 防作弊”这种结论。
- 用实测数据理解“帧同步省带宽”成立的前提：单位多、输入少。
- 画出同步方案的谱系图，知道 Lockstep 家族、状态同步家族和混合方案各有哪些代表。
- 拿到一个游戏需求，能按决策树选型并说出理由。

30 秒面试版答案

多人游戏要让所有人看到同一场战斗。**状态同步**由服务器跑权威逻辑、把状态（快照 / 属性）发给客户端，客户端插值播放、本地预测减少延迟感，优点是天然权威、断线重连只要拉一份最新状态，代价是带宽随单位数增长、手感要靠预测和延迟补偿弥补。**帧同步**让每个客户端跑同一份确定性逻辑，网络上只传每帧的输入，服务器（或 P2P）只负责定序和转发，优点是带宽与单位数无关、录像观战几乎免费，代价是必须保证确定性、必须自己处理丢包和断线追帧，并且客户端掌握全量信息，反透视更难。MOBA / RTS 多用帧同步（王者荣耀），FPS / MMO 多用状态同步（Unreal 的默认模式），但英雄联盟、DOTA2 这样的 MOBA 也是状态同步——选型看约束，不看品类标签。

1.1 同一场战斗，同一个真相

5v5 对局里，十台手机必须对“某英雄此刻站在哪、还剩多少血、那个技能到底中没中”给出完全一致的答案。难点有三个：

- **不互信**：任何一台设备都可能被篡改，不能让某个客户端说了算。

- **不直连、网络各异**：设备之间隔着运营商、Wi-Fi、基站，延迟从十几毫秒到几百毫秒不等，还会丢包、乱序、抖动。
- **没有全局时钟**：每台设备的“现在”都不一样，连“同时”都需要定义。

所有同步方案都是在回答零章那三个问题：**谁说了算、网上传什么、什么时候推进。**

1.2 两种经典解法

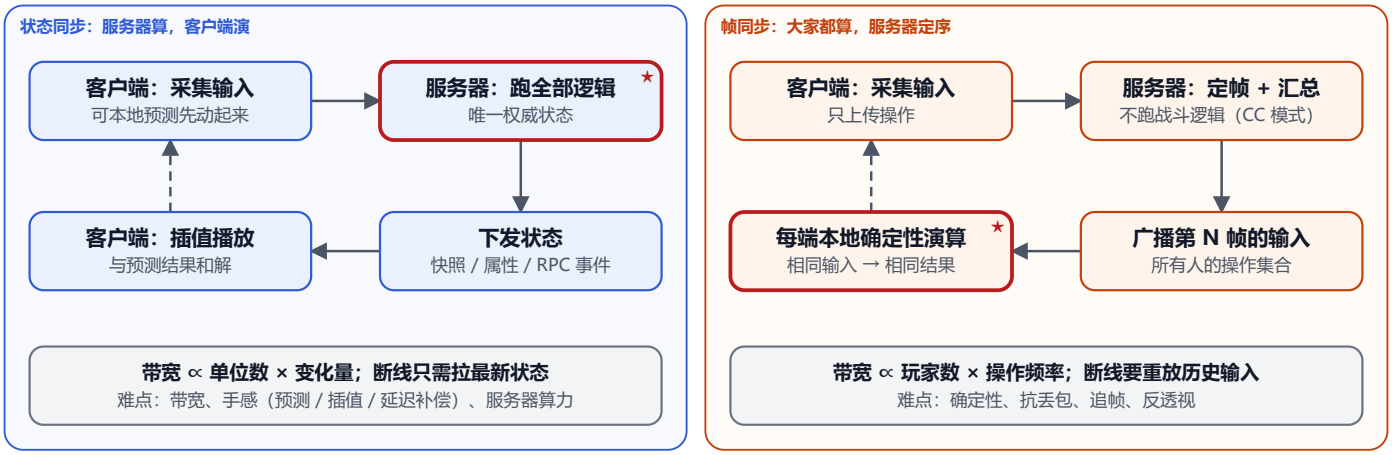


图 1-1 两种经典解法。红框★是各自的“真相来源”：状态同步的真相是服务器内存里的状态，帧同步的真相是“输入序列 + 确定性逻辑”。

注意帧同步里的服务器并非可有可无：HOK 的 Relay 服务器负责分配帧号、汇总去重、定频广播、缓存历史帧，只是**不算战斗结果** HOK 文档 。早期 RTS 的严格 Lockstep 甚至可以纯 P2P，没有服务器。

1.3 一张对照表（加注版）

维度	状态同步	帧同步	容易说错的地方
谁算逻辑	服务器（客户端可预测一部分）	每个客户端各算一份完整逻辑	帧同步服务器也可以跑一份逻辑做校验 / 托管（HOK 的 CS 模式、LoL 的确定性服务器就是反例 / 变体）
传什么	状态快照、属性增量、RPC 事件	每帧的输入集合	帧同步也会传少量“服务器命令”（出生、结算、掉线托管）
带宽	随单位数 × 变化频率增长	随玩家数 × 操作频率增长	单位少时，增量快照 + 兴趣范围并不比帧同步贵（见 1.4 实测）
手感	需要预测、插值、延迟补偿	本地能立即演算；定频广播模式下操作要等一个来回才生效	“帧同步手感直接”只对回滚 / 本地预表现成立，HOK 的操作仍要经服务器回来才进入逻辑
反作弊	客户端拿不到不该知道的数据（可做兴趣裁剪）	客户端有全量数据，透视 / 全图难防；改数值会被 hash 校验发现	状态同步也防不住自瞄、加速预测，只是“改结果”变难
断线重连	拉一份最新状态即可	必须重放（追）历史输入	帧同步可以定期存快照缩短追帧，但快照本身要确定性序列化
录像 / 观战	录下行数据流（UE 的 DemoNetDriver），体积较大	存输入序列即可，极小；观战 = 接收同一份输入	帧同步录像依赖“同版本同逻辑”，跨版本回放要保留旧逻辑
核心难点	带宽、手感、服务器算力	**确定性**、**抗丢包**、追帧	—
代表	Unreal 默认模式、FPS、MMO、英雄联盟、DOTA2	王者荣耀、早期 RTS、格斗游戏（回滚）	品类不是决定因素

1.4 实测：帧同步到底省多少带宽 实测 E1

模型：15 Hz、10 名玩家，每人每秒约 5 条命令（摇杆变化 + 技能）。帧同步每包带最近 3 帧（冗余），每条命令 6 字节；状态同步每个单位全量 32 字节，增量编码只发变了的字段（id 2 B + 掩码 1 B

+ 位置 4 B + 血量 2 B + 其他 2 B，每 tick 约 60% 单位在动、10% 掉血），“增量 + AOI”再只发视野内约 35% 的单位。包头按 UDP 28 B + 应用层 8 B 计。

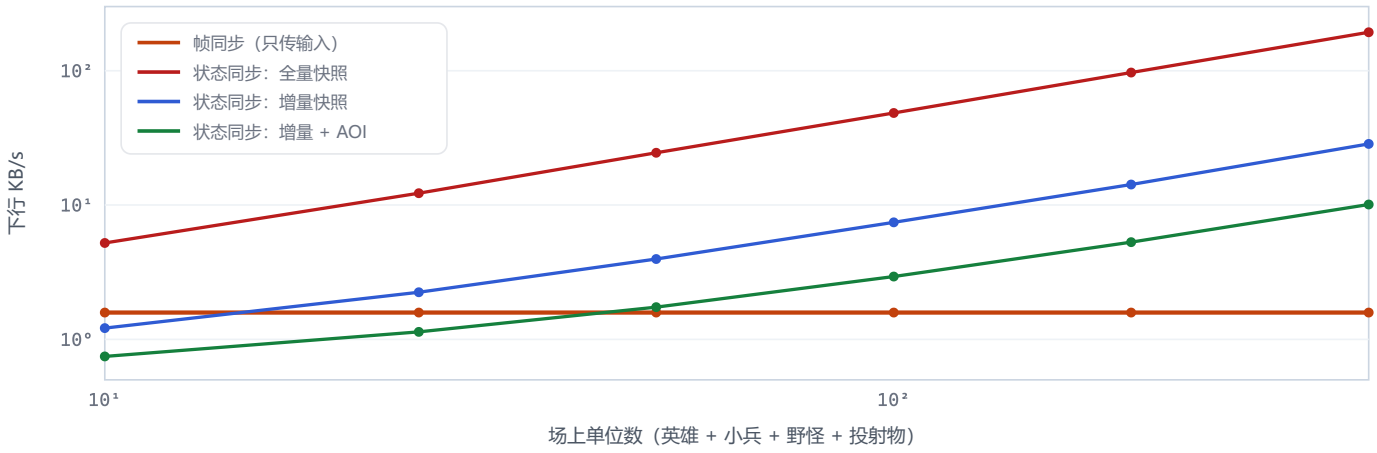


图 1-2 每个客户端的下行带宽（对数坐标）。帧同步是一条水平线（约 1.58 KB/s），与单位数无关；全量快照随单位数线性增长；增量 + AOI 在单位少时甚至比帧同步更省。

单位数	帧同步 KB/s	全量快照 KB/s	增量快照 KB/s	增量 + AOI KB/s	全量 / 帧同步
10	1.58	5.21	1.21	0.75	3.3
25	1.58	12.25	2.25	1.14	7.7
50	1.58	24.49	3.96	1.74	15.5
100	1.58	48.46	7.42	2.94	30.6
200	1.58	96.91	14.19	5.30	61.3
400	1.58	193.30	28.46	10.08	122.2

表 1-1 E1 实测数据（demos/netsync_demo.cpp）。100 个单位时全量快照是帧同步的 30 倍，但增量 + AOI 只有约 2 倍。

读这张图要得出两个结论：

1. “帧同步省带宽”在**单位多**时成立：MOBA 后期几十个小兵、上百个 Buff / 弹道，RTS 上千个单位，状态同步要么砸带宽，要么砸复杂度（增量、AOI、优先级）。

2. 单位少时差距并不大：一个 10 人、只有英雄的竞技游戏，增量快照 + AOI 完全扛得住——这就是为什么 FPS 普遍用状态同步。带宽只是选型的一个维度，下一节的确定性、反作弊、手感往往更决定性。

1.5 同步方案谱系

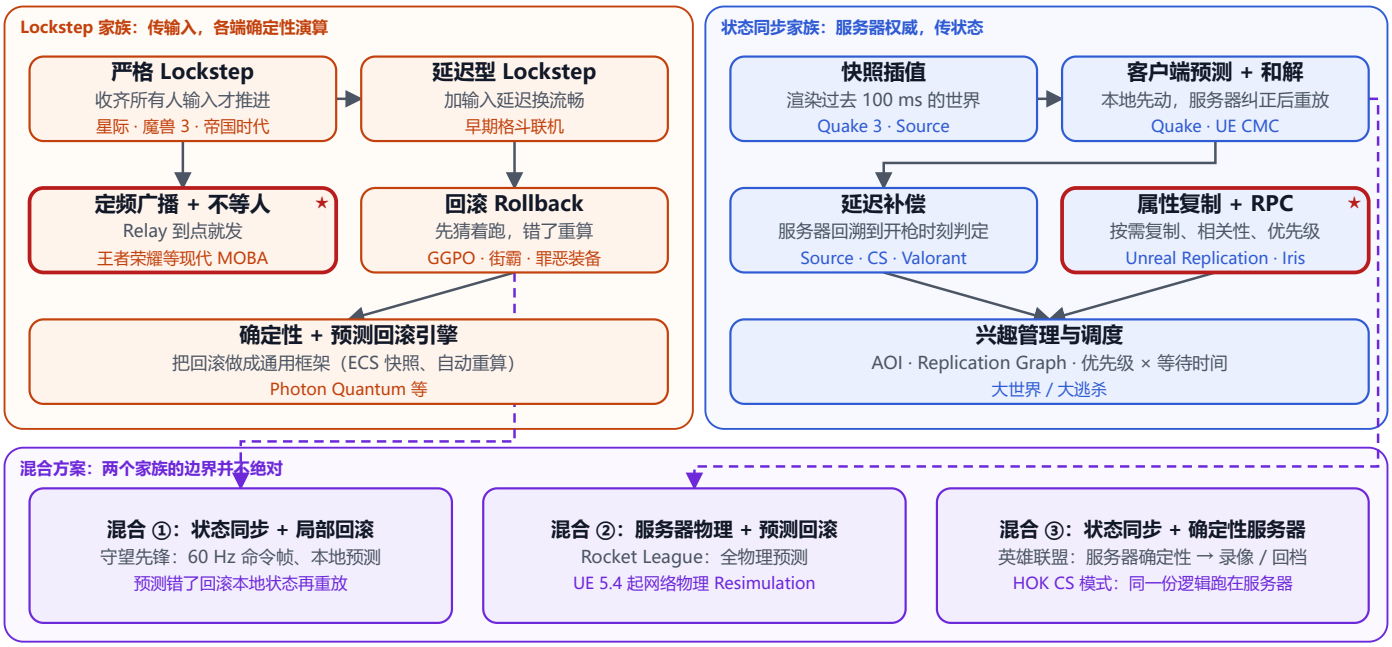


图 1-3 同步方案谱系。左边从“等所有人”演化到“不等人”和“先猜后改”，右边从“播放快照”演化到“预测 + 补偿 + 调度”；下方三种混合方案说明边界并不绝对。

谱系背后的规律

两个家族都在同一对矛盾里找平衡：**一致性（大家看到的一样）vs 响应性（我按了马上有反应）**。等待换一致性（严格 Lockstep、快照插值），猜测换响应性（回滚、客户端预测），而“猜错了怎么办”的答案几乎总是同一个：**保存历史 → 回到分歧点 → 用正确数据重算**。

1.6 选型决策树

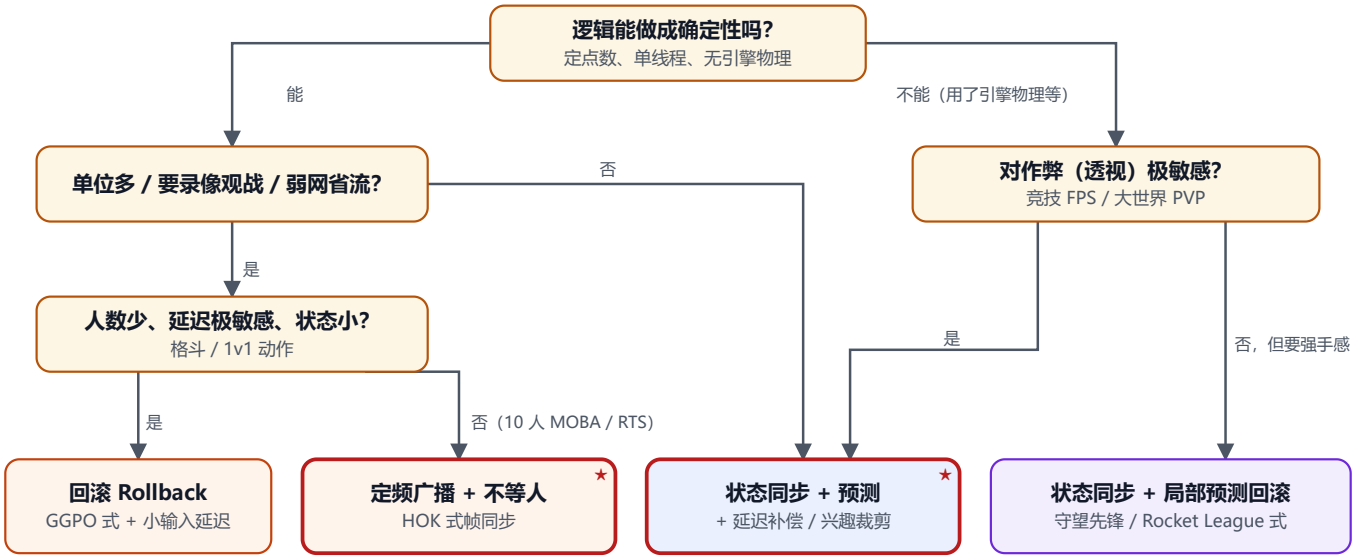


图 1-4 选型决策树（简化版，第 14 章有完整对照）。第一问永远是“能不能确定性”：用了引擎物理、多线程逻辑、浮点的项目，帧同步的代价会高到不现实。

面试注意

被问“帧同步和状态同步的区别”时，**不要只背优缺点表。**先用一句话给本质（谁算逻辑、传什么），再挑两个维度展开（比如带宽和反作弊），最后加一句“选型看约束”并举反例（LoL / DOTA2 是状态同步的 MOBA，守望先锋在状态同步里用了回滚）。能说出反例，面试官就知道你不是背的。

1.7 坑位与易错说法

易错说法	问题在哪	更准确的说法
“帧同步服务器什么都不干”	忽略了定帧、汇总去重、冗余广播、缓存、补发、追帧、校验	帧同步服务器不算战斗结果，但负责**定序与可靠投递**
“状态同步天然防作弊”	只防得住“改结果”，防不住自瞄、宏、利用预测的加速	状态同步让权威在服务器，**改结果难**；输入层作弊仍需反作弊系统
“帧同步一定省带宽”	单位少时增量快照 + AOI 不贵（表 1-1）	带宽优势随单位数放大；10 人纯英雄对战差距不大
“帧同步手感一定好”	HOK 式定频广播下操作要经服务器回来才进逻辑（实测约 112 ms）	手感好来自**本地预表现**（按键即出指示器）或回滚
“MOBA 用帧同步，FPS 用状态同步”	LoL、DOTA2 都是状态同步的 MOBA	品类只是经验，约束（确定性、单位数、反作弊、网络）才决定选型

本章速记

- 状态同步：**服务器算、传状态、客户端插值 + 预测**；帧同步：**各端算、传输入、拿到输入才推进。**
- 帧同步带宽 ≈ 常数（本模型约 1.58 KB/s），全量快照 ∝ 单位数；增量 + AOI 让状态同步在小规模下同样便宜。
- 谱系：严格 → 延迟型 → 定频不等人 / 回滚；快照插值 → 预测和解 → 延迟补偿 → 兴趣管理；混合方案无处不在。
- 选型第一问：**能否确定性**；第二问：单位规模与录像需求；第三问：反作弊与手感。

▶ **自测 1：一个 3v3 的竞技射击游戏，每局 5 分钟，需要强反作弊，用什么同步?**

▶ **自测 2：为什么说帧同步的“真相”是输入序列?**

▶ **自测 3：表 1-1 中 10 个单位时增量 + AOI 比帧同步还省，那 MOBA 为什么还用帧同步?**

CHAPTER 02

时间、网络与固定步长

同步的敌人不是“慢”，而是“不确定”：抖动比平均延迟更致命，渲染帧率不能影响逻辑，各端的“现在”要对齐到同一条时间轴上。

本章目标

- 说清延迟、抖动、丢包、乱序各自造成什么现象，为什么游戏用 UDP + 自定义可靠层。
- 写出固定步长循环（accumulator），理解“插值多一帧延迟、外推会出错”的取舍，并看实测数据。
- 会推 NTP 式对时公式，知道 UE `GetServerWorldTimeSeconds` 的精度局限。

30 秒面试版答案

网络有四个“坏”：延迟、抖动、丢包、乱序。TCP 可靠有序但会队头阻塞，一个包丢了后面全部等重传，所以实时游戏用 UDP，自己在上面做“按需可靠”：序号 + ACK 位图、只对关键数据重传、状态类数据直接发最新值。逻辑必须用**固定步长**推进（帧同步靠它保证确定性，状态同步靠它让服务器 / 客户端步调一致），渲染帧率任意；渲染时用两次逻辑状态**插值**保证平滑，代价是多一个逻辑帧的延迟。各端要对齐时间轴：用 NTP 式往返测量估计时钟偏移，取 RTT 最小的样本、做平滑。

2.1 网络的四个“坏”

现象	定义	玩家感受	常见对策
延迟（Latency / RTT）	单程或往返时间；物理距离 + 排队 + 处理	按键到反馈慢、被“隔墙杀”	预测、延迟补偿、就近部署
抖动（Jitter）	延迟的波动	一卡一卡、忽快忽慢	抖动缓冲（插值延迟、输入缓冲）
丢包（Loss）	包没到；移动网络常见**突发**丢包（连丢）	操作没生效、画面停住	冗余、选择性重传、补发
乱序（Reordering）	后发先至	状态回跳、技能顺序颠倒	序号 + 接收窗口重排

经验量级（仅供建立直觉）：同城有线 RTT 几十毫秒以内；4G / 5G 通常几十到一百多毫秒，抖动明显；Wi-Fi 信号弱时会出现几百毫秒的突发停顿；跨国 150~300 ms。**平均值能用缓冲吸收，抖动和突发丢包才是体验杀手**——HOK 文档里所有的冗余、补发、卡顿统计都在对付后两者 HOK 文档 。

2.2 为什么游戏用 UDP



图 2-1 TCP 的队头阻塞 vs 带冗余的 UDP。TCP 的“可靠有序”对文件传输是优点，对“只关心最新状态”的游戏是负担；游戏要的是**按需可靠**。

实时游戏在 UDP 上自建可靠层，常见做法：

- **序号 + ACK 位图**：每个包带自己的序号、对方最新序号和一个 32 位位图（表示之前 32 个包哪些收到了），不用单独的 ACK 包 Gaffer On Games 。
- **分通道**：不可靠（移动、状态）/ 可靠有序（聊天、结算）/ 可靠无序。Unreal 的 RPC 分 Reliable / Unreliable，属性复制丢了不重发旧包、而是下次发**最新值** UE 。
- **冗余代替重传**：状态和输入很小，与其等一个 RTT 重传，不如每个包把最近几份一起带上（HOK 下行一包多帧、上行三条历史，第 6、7 章）。
- **以带宽换延迟**：KCP 这类协议用快速重传、不做 TCP 式拥塞退让，换来更低的重传延迟。

HOK 用腾讯的 Apollo 中间件同时维持 UDP 和 TCP：帧命令、ping、补帧请求走 UDP；大厅协议、开局结算等必须可靠的走 TCP；取包时**先读 UDP、再读 TCP**，实时性优先 HOK 文档 。

2.3 固定逻辑步长，渲染与之解耦

逻辑如果跟着渲染帧率走（`pos += v * deltaTime`），不同机器、不同帧率算出来的结果就不同——帧同步直接 desync，状态同步的预测也对不上服务器。所以逻辑一律用固定步长，渲染帧率随便：

固定步长 + 渲染插值（Gaffer “Fix Your Timestep” 的标准写法）

```
const double kStep = 1.0 / 15;          // 逻辑步长：HOK 约 66 ms 一帧
double accumulator = 0;
State prev = state, cur = state;

void OnRenderFrame(double frameDt) {    // 渲染帧率任意（30 / 60 / 120 fps，还会抖）
    accumulator += std::min(frameDt, 0.25);   // 防“死亡螺旋”：一帧最多补 0.25 s
    while (accumulator ≥ kStep) {             // 落后几步就补几步，逻辑永远是整步
        prev = cur;
        cur  = Simulate(cur, kStep);          // 确定性：只依赖 (状态, 输入, 固定步长)
        accumulator -= kStep;
    }
    double alpha = accumulator / kStep;       // 0~1：当前时刻在两个逻辑帧之间的位置
    Render(Lerp(prev, cur, alpha));           // 插值：平滑，但比最新逻辑晚约 1 帧
}
```

渲染时有三种选择：直接画最新逻辑状态、在上一帧和当前帧之间插值、按速度往前外推。实测如下 **实测 E2** ：

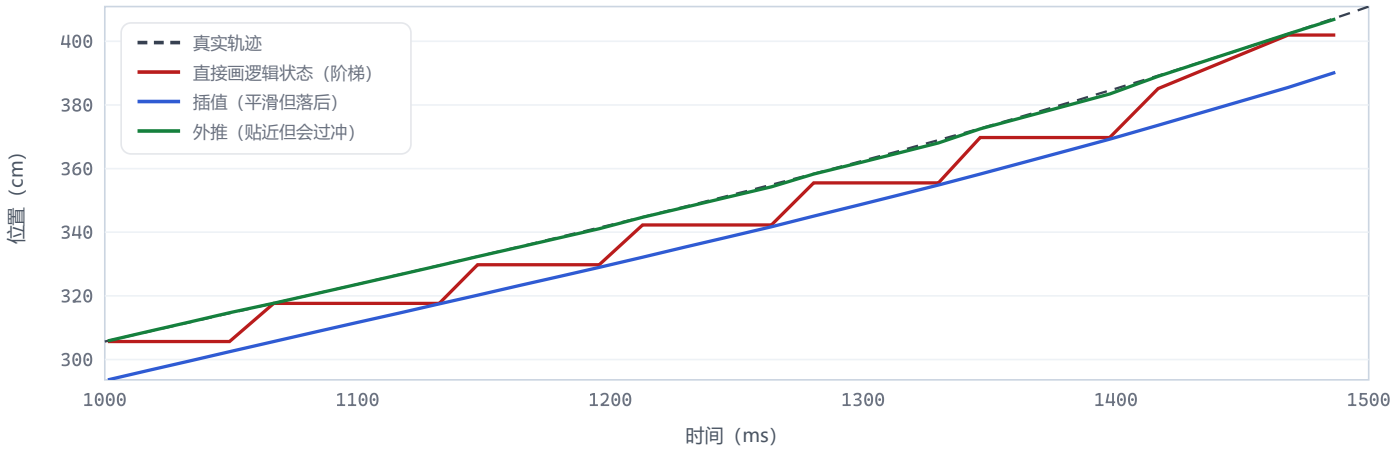


图 2-2 15 Hz 逻辑 + 约 60 fps 渲染（4% 的帧多卡 34 ms）。直接画逻辑状态是阶梯（每 66 ms 跳一次），插值是平滑曲线但整体右移约一个逻辑帧，外推贴近真实但在速度变化时会过冲。

渲染方式	平均滞后（相对真实）	每帧速度抖动（标准差）	适用
直接画逻辑状态	33.6 ms	**506 cm/s**	只适合逻辑帧率 ≥ 渲染帧率
插值（prev → cur）	66.6 ms	**20 cm/s**	默认选择：平滑、永不出错
外推（cur + v·α）	0.0 ms	29 cm/s	延迟敏感、运动可预测时

结论：插值把阶梯感降了一个数量级以上，代价是多一个逻辑帧（66 ms）的显示延迟。HOK 的做法是逻辑 15 Hz 由服务器驱动、客户端逻辑帧率可以更高，**“表现插值就发生在多出来的帧里”**，帧号按 `客户端帧号 = 服务器帧号 × 服务器帧间隔 ÷ 客户端帧间隔` 换算 **HOK 文档**。状态同步里同样的思想放大到网络层面就是第 10 章的“快照插值”。

2.4 对齐时间轴：NTP 式对时

客户端在本地时刻 t_0 发出请求，服务器在其时钟的 t_1 收到、t_2 回复，客户端在本地 t_3 收到。假设上下行延迟对称：

推导 • 时钟偏移与往返时间

① $\mathrm{RTT} = (t_3 - t_0) - (t_2 - t_1)$ 总耗时减去服务器处理时间

② $t_1 = t_0 + \theta + \delta_{\uparrow}, \quad t_3 = t_2 - \theta + \delta_{\downarrow}$ θ：服务器钟 − 客户端钟；δ：单程延迟

③ $\theta = \dfrac{(t_1 - t_0) + (t_2 - t_3)}{2} + \dfrac{\delta_{\downarrow} - \delta_{\uparrow}}{2}$ 两式相减；对称时第二项为 0

误差来自上下行不对称：最坏可达 RTT/2，所以实践中只用 RTT 最小的那几次样本（排队最少、最接近对称）

工程要点：连续采样多次，**取 RTT 最小的样本**估计偏移，再做指数平滑；运行中定期校正，校正时“拉伸”时间而不是跳变（否则插值时间轴会倒退）。

Unreal 里的服务器时间

`AGameStateBase::GetServerWorldTimeSeconds()` 的实现是服务器周期性复制自己的世界时间，客户端在 OnRep 里用“收到的值 − 本地时间”算偏移——**没有扣除单程延迟**，所以客户端算出的服务器时间会落后约半个 RTT，且受复制间隔影响 UE 源码 。做延迟补偿、技能倒计时、回放对齐等需要精确时间的功能时，通常自己用一对 RPC 做 NTP 式对时，或者像 CMC 那样用客户端时间戳 + 服务器校验。

2.5 统一“帧”的概念

	HOK（帧同步）	Unreal（状态同步）
逻辑节拍	服务器约 15 帧/秒（66 ms），帧号由服务器分配并广播 HOK 文档	服务器按 `NetServerMaxTickRate`（引擎默认配置 30）tick；每个 Actor 再按 `NetUpdateFrequency` 决定多久复制一次 UE
客户端节拍	收到第 N 帧才推进第 N 帧；C# 的帧号由 C++ 回报	按本地帧率 tick；移动用客户端时间戳（CMC 的 TimeStamp）与服务器对账
渲染	与逻辑解耦，插值	SimulatedProxy 插值平滑，AutonomousProxy 本地预测

面试注意

“为什么要固定步长”要答到**确定性**和**与服务器步调一致**两层；“插值有什么代价”要答出**多一帧延迟**；“怎么对时”要能写出偏移公式并说明**取最小 RTT 样本**的原因（不对称误差）。能顺带说出 UE 默认服务器时间不做延迟补偿，是加分项。

2.6 坑位与易错说法

坑	后果	正确做法
逻辑里用 `deltaTime` / 墙钟	帧同步 desync；状态同步预测与服务器对不上	逻辑只认帧号 × 固定步长
accumulator 不设上限	卡一下后疯狂补帧 → 更卡 → 死亡螺旋	单帧最多补若干步，超出则放慢或丢弃
用 float 累加游戏时间	20 分钟累计误差约 -116 ms（实测 D6，第 4 章）	用整数帧号推导时间
对时直接跳变	插值时间轴倒退，画面回跳	平滑拉伸（略快 / 略慢地走）
只看平均 ping	抖动导致卡顿却看不出原因	统计分布（p95 / p99）和卡顿事件，HOK 用 10 档分桶（第 8 章）

本章速记

- 四个坏：延迟（预测 / 补偿）、抖动（缓冲）、丢包（冗余 / 重传）、乱序（序号 + 窗口）。突发丢包与抖动最致命。
- TCP 队头阻塞 → 游戏用 UDP + 按需可靠（序号、ACK 位图、冗余、状态发最新值）。
- 固定步长 + 渲染插值：平滑但晚一帧（实测 67 ms，速度抖动 506 → 20 cm/s）。
- 对时：$\theta = \frac{(t_1 - t_0) + (t_2 - t_3)}{2}$，取最小 RTT 样本 + 平滑；UE 默认服务器时间不扣单程延迟。

▶ **自测 1：渲染 120 fps、逻辑 15 Hz，不插值会怎样?**

▶ **自测 2：为什么 NTP 式对时要取 RTT 最小的样本?**

▶ **自测 3：Unreal 的属性复制丢包后会重发原来那个包吗?**

PART II

帧同步篇：以王者荣耀为主线

从三块基石和三条路线出发，依次拆解确定性、HOK 的三层架构、下行与上行的抗丢包体系、质量保障，最后看回滚网络如何把 Lockstep 推到极限。

CHAPTER 03

Lockstep 的三块基石与三条路线

“各端跑相同逻辑、靠相同输入达成一致”这句话没有规定什么时候允许推进下一帧——这正是帧同步内部最大的分野，也决定了一个人网卡时会连累谁。

本章目标

- 说出帧同步的三块基石，以及“输入序列 = 整场战斗”带来的四个红利。
- 区分严格 Lockstep、定频广播 + 不等人、回滚三条路线的推进条件和故障影响域。
- 用 10 人对局实测看清：一个人断流 2 秒，三条路线分别让谁卡、卡多久。
- 修正一个常见误解：P2P 回滚并不能让“卡顿只影响自己”，超过预测窗口后全员都要等。

30 秒面试版答案

帧同步有三块基石：**定帧**（把时间切成固定逻辑帧，收到第 N 帧才推进第 N 帧）、**确定性**（相同输入得到相同输出）、**广播输入而非结果**（整场战斗 = 一串输入，录像、重连、观战、校验共用一套机制）。按“何时推进”分三条路线：严格 Lockstep 要收齐所有人的输入，一人卡全员卡；定频广播不等人（王者荣耀）由服务器到点就广播，谁没上报这一帧就没有谁，把全局故障降级为局部故障；回滚先预测远端输入立即推进，错了回到分歧帧重算，延迟最低，但需要每帧快照、重算成本高，而且预测窗口用完后仍然要全员等待。

3.1 三块基石

基石	含义	被破坏的后果	HOK 的做法 HOK 文档
定帧	时间离散成等长逻辑帧；推进依据是“拿到第 N 帧”，不是本地时钟	各端在不同时刻算不同帧，时间无法统一	Relay 约 15 帧/秒定频广播；本地帧号由 C++ 回报，C# 不自己累加
确定性	相同初始状态 + 相同输入序列 → 逐位相同的结果	静默 desync，并逐帧放大	独立 C++ GameCore：定点数、帧随机源、单线程、倍速用有理数
广播输入	网络上只传输入，不传结果	带宽暴涨，或者退化成状态同步	`SCID_FRAPBOOTINFO(1035)` 只装一帧内所有人的命令

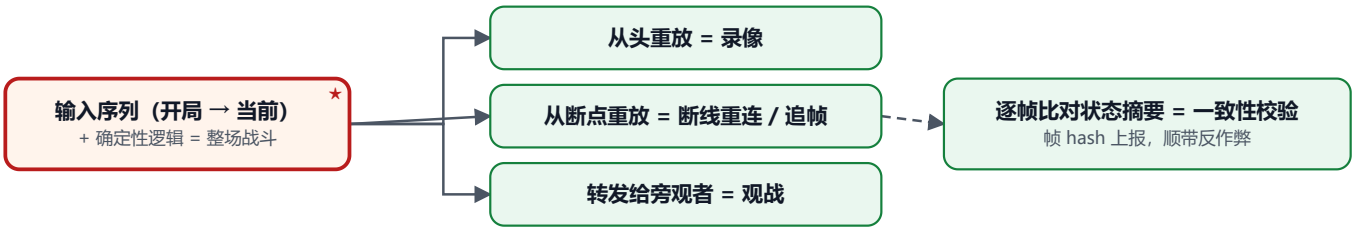


图 3-1 帧同步最大的工程红利：录像、重连、观战、校验共用“输入序列”这一份数据，HOK 甚至让回放与实战走完全相同的代码路径（第 8 章）。

3.2 核心问题：什么时候允许推进下一帧



图 3-2 三条路线的推进条件与故障影响域。② 的关键创新不是“有服务器”，而是**服务器不等人**：把一个可能全局化的故障主动降级成局部故障。

3.3 实测：一个人断流 2 秒，谁在卡? 实测 E3

模型（`demos/netsync_demo.cpp` 的 `run_p2p` / `run_relay`）：10 名玩家，15 帧/秒（66 ms）；玩家 0~8 网络良好（单程 18~42 ms、指数抖动均值 6 ms、丢包 1%），玩家 9 网络较差（单程 45 ms、抖动 25 ms、丢包 4%），并在 **20~22 s 完全断流**。三条路线：

- **严格 Lockstep**：P2P，输入延迟 2 帧，每个包带自己最近 40 帧输入（冗余），有新输入立刻发、卡住时每 66 ms 重发。
- **定频广播 + 不等人**：服务器每 66 ms 广播一帧，每包带最近 3 帧；上行短冗余（同一命令 0 / 33 / 66 ms 各发一次）；客户端入窗即消费，发现空洞或 90 ms 没推进就请求补发（紧急 120 ms 一次，卡顿超 1 s 后降到 1000 ms 一次）。
- **回滚**：P2P，输入延迟 1 帧，最多预测 8 帧，预测策略是“重复对方上一次的输入”。

“卡顿”定义为相邻两次逻辑推进的间隔超过 2 帧（132 ms）时多出来的时间。下图是 19~23.5 s 的逐帧时间线：

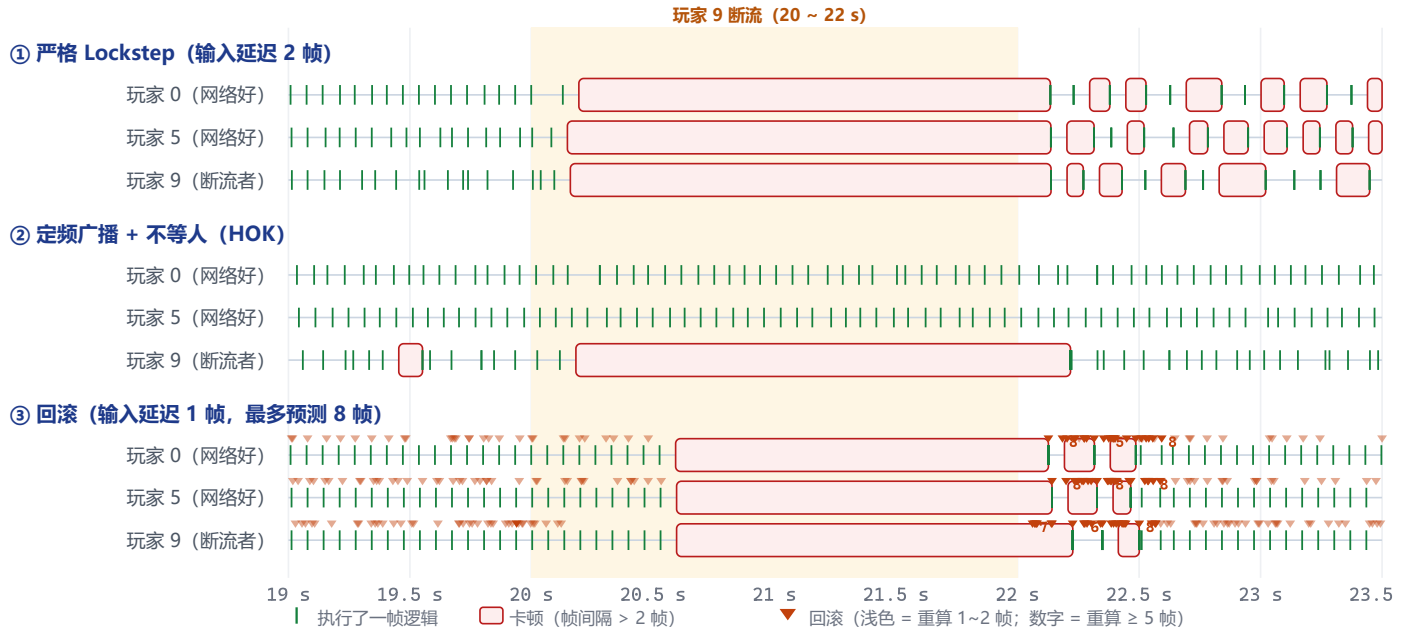


图 3-3 三条路线的逐帧时间线（C++ 实测）。① 所有人一起停 2 秒以上；② 只有断流者停，好玩家的绿色竖线连续不断；③ 前 8 帧（约 0.5 s）大家靠预测继续走，之后**所有人**都停下等断流者，恢复瞬间出现深度为 8 的回滚。本模型的逻辑时间锚定墙钟，① 恢复后要追回落后的帧，而追赶速度受断流者的上行延迟限制，所以还会断续卡一阵。

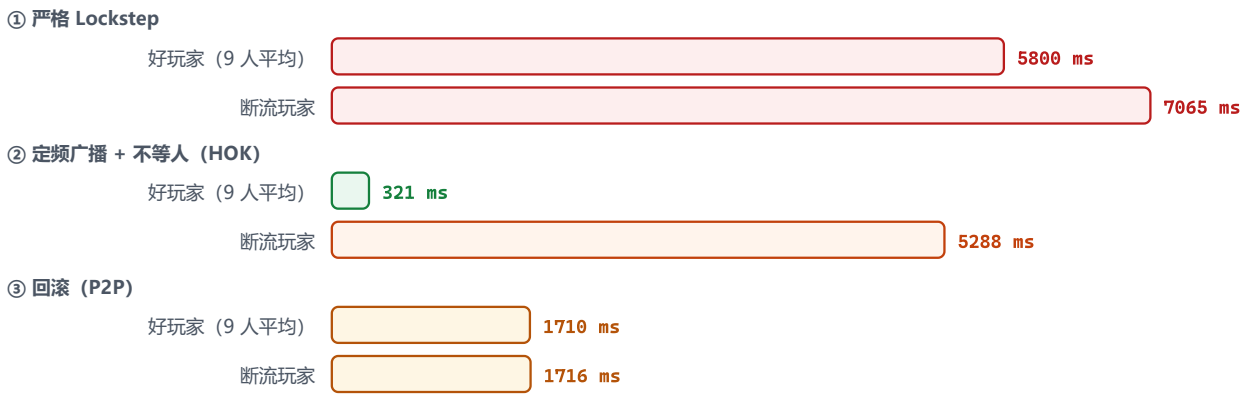


图 3-4 60 秒对局里的总卡顿时间。只有 ② 把“断流者的问题”关在了断流者自己身上。

路线	好玩家卡顿 ms	断流者卡顿 ms（窗口内 / 外）	好玩家操作延迟 均值 / p95 ms	回滚 次/秒
严格 Lockstep（P2P，输入延迟 2 帧）	5800	7065（3482 / 3583）	147 / 186	0.0 · 0.0 · (
定频广播 + 不等人（HOK）	321	5288（2035 / 3253）	112 / 141	0.0 · 0.0 · (
回滚（P2P，输入延迟 1 帧，最多预测 8 帧）	1710	1716（1716 / 0）	68 / 66	24.0 · 1.1 ·

表 3-1 E3 汇总。“窗口外”是断流者在 20~25 s 之外、由自身差网络（抖动 25 ms、丢包 4%）造成的卡顿。

从数据里读出的五个结论

1. **严格 Lockstep 的木桶效应是真实的**：好玩家平均卡了 5800 ms——不仅是那 2 秒断流，断流者平时的抖动和丢包也在一直拖累所有人。

2. **定频广播把故障关进了笼子**：好玩家只卡了 321 ms（几乎全是自己的网络抖动），断流者自己卡了 5288 ms。代价是断流期间他的 13 条命令**永久丢失**（短冗余只覆盖 66 ms），这就是 HOK 文档说的“上行丢包损失的是我的操作”。
3. **断流结束后恢复也要时间**：断流者在断流窗口内卡了 2035 ms，比 2000 ms 多出来的部分来自补发请求降频（卡顿超过 1 s 后改为 1000 ms 一次）——这是 HOK 有意为之的“防请求风暴”取舍。
4. **回滚没有消灭木桶效应，只是推迟了它**：预测窗口 8 帧（约 0.5 s）内大家照常推进；一旦断流超过窗口，P2P 的每个人都缺这位玩家的真实输入，**全员**卡了约 1710 ms。
5. **操作延迟**：回滚最低（68 ms ≈ 1 帧输入延迟），定频广播 112 ms（上行 + 等服务器 tick + 下行），严格 Lockstep 147 ms（2 帧输入延迟 + 等最慢的人）。

对照原文档的一处补充（勘误）

原文 2.4.5 节的总对照表写“回滚：一人卡顿的影响 = 仅自己画面跳”。这只在**卡顿短于预测窗口**时成立。实测表明，P2P 回滚在断流超过预测窗口后同样会让所有人停下（GGPO 在这种情况下会阻塞推进，并通过“时间同步”让领先的一方放慢）。更准确的说法是：**短抖动 → 别人轻微跳变；长断流 → 仍是全局等待。**原文 2.5.4 节“回滚并没有消灭木桶效应，只是推迟到更糟的网络条件下才暴露”的说法是对的，与这里一致。

3.4 为什么 HOK 选了路线二

MOBA 的约束	对严格 Lockstep	对回滚	对定频广播不等人
10 个玩家，移动网络丢包、抖动常态化	**任何一人波动 → 九人陪卡（E3 ①）**	预测 9 路输入，抖动频繁 → 持续画面跳变	**故障局部化（E3 ②）**
数十单位 + 上百 Buff / 弹道	无影响	**每帧快照 + 多帧重算成本高**	**不需要快照**
单局 15~20 分钟，强依赖录像观战	输入序列即录像	预测与重算让录像语义变复杂	**输入序列即录像**
已经有 Relay 服务器	不需要	不需要	**让它承担定频广播，顺理成章**
对操作延迟敏感，但不到格斗的帧级	延迟取决于最慢者	延迟最低	延迟取决于自己的链路，可接受

HOK 文档的总结很到位：**严格 Lockstep 太脆弱，回滚太昂贵，定频广播不等人是 MOBA 约束下刚刚好的解** HOK 文档 。补一句工程视角：路线二把“我网卡”的代价全部留给丢包者本人，于是 HOK 需要一整套让丢包者**自救**的机制——下行冗余、补发、追帧（第 6 章），上行冗余与技能保序（第 7 章），卡顿归因与双链路（第 8 章）。

3.5 “不等人”带来的两种损失



图 3-5 定频广播不等人模型下，丢包的两种损失都只落在丢包者身上，但对他本人都很严重：上行丢包 = “手残”，下行丢包 = “卡死”。

面试注意

“王者荣耀一个人卡了，其他人会卡吗？”是高频题。标准答案：**不会**，因为服务器定频广播不等人，卡的人只是自己的操作进不去、自己的画面停住；对比早期 RTS 的严格 Lockstep（一人卡全员卡）。追问“那回滚呢”，要说出预测窗口的限制。千万别说“帧同步一人卡全员卡”——那是严格 Lockstep，不是现代 MOBA。

3.6 坑位与易错说法

易错说法	更准确的说法
“帧同步一个人卡所有人都卡”	只有严格 Lockstep 如此；定频广播不等人模型下只影响自己（E3）
“回滚只影响自己”	短于预测窗口时对别人只是轻微跳变；超过窗口仍全员等待（E3 ③）
“定频广播模型里服务器会等慢的客户端”	不会；客户端用期待帧号做 ACK，服务器据此补发，但广播节奏不变
“帧同步不需要服务器”	严格 Lockstep 可以 P2P；现代 MOBA 都需要 Relay 做定序、冗余、缓存、补发

本章速记

- 三基石：**定帧 / 确定性 / 广播输入**；输入序列 = 录像 = 重连 = 观战 = 校验。
- 三路线看“何时推进”：收齐才走（严格）/ 服务器到点就发（不等人）/ 先猜后改（回滚）。
- E3：好玩家卡顿 严格 5800 ms、不等人 321 ms、回滚 1710 ms；断流者在不等人模型下丢了 13 条命令。
- 回滚延迟最低（68 ms），但预测窗口之外仍是木桶。

▶ **自测 1：严格 Lockstep 为什么常配合“输入延迟”？延迟设多大合适?**

▶ **自测 2：定频广播模型下，丢包者的命令为什么会“永久丢失”而不是“迟到”？**

▶ **自测 3：为什么 HOK 不用回滚？说出两个最关键的理由。**

CHAPTER 04

确定性工程：让十台设备算出同一个比特

帧同步最危险的故障不是卡，而是“静默分叉”：某一帧某个浮点数末位不同，之后每一帧都在放大，最后变成“我明明打中了却没伤害”。

本章目标

- 说清浮点“不确定”的真正来源：不是 IEEE 754 本身，而是编译器、指令集、数学库和运算顺序。
- 亲眼看到：同一份源码、同一台机器，只换编译选项，600 帧群体模拟的 hash 就不同；定点版三份完全一致。
- 会写 Q16.16 定点数的乘除、开方、正弦，知道精度和溢出的代价。
- 掌握随机、容器顺序、时间累加等其他确定性陷阱，以及 desync 的检测和定位方法。

30 秒面试版答案

帧同步要求相同输入得到逐位相同的结果。IEEE 754 的加减乘除开方本身是确定的，问题出在**编译器重排和 FMA 收缩（/fp:fast）、不同指令集、各平台数学库的 sin/cos 实现、并行归约顺序**，所以跨平台帧同步一般用**定点数**（如 Q16.16 / Q32.32，整数运算处处一致）。其他陷阱：随机数要用同步种子的确定性生成器，且表现层不能偷用逻辑随机；不能依赖 unordered_map / 指针地址的遍历顺序；逻辑不能读墙钟、不能用浮点累加时间；逻辑层单线程；不用引擎物理。出了 desync 靠**周期性上报帧 hash**发现，靠分层 hash、逐帧日志 diff 和录像重放定位。HOK 把战斗逻辑整体放进独立的 C++ GameCore，物理上隔绝了引擎的浮点、时钟、随机和容器。

4.1 确定性的敌人

敌人	典型例子	对策
浮点差异	`/fp:fast` 重排、FMA、x87 80 位中间结果、sin/cos 库不同、SIMD 归约顺序	定点数；或锁死编译器 + 指令集 + 自研数学函数（只在同平台成立）
随机	各端种子不同；表现层（特效、UI）调用了逻辑随机，导致调用次数不同	可播种的同步随机源，种子随开局下发；逻辑随机与表现随机严格分开
遍历顺序	`unordered_map`、按指针地址排序的 `set`、`std::hash<string>` 跨标准库不同	有序容器或稳定 id 排序；自定义哈希
时间依赖	逻辑里读本地时钟、用 `deltaTime`、浮点累加时间	只用逻辑帧号
多线程	并行更新单位，写入顺序不定	逻辑层单线程串行（或严格的确定性 Job 划分）
未定义行为	未初始化变量、越界读、有符号溢出	静态检查、ASan、确定性回归测试
第三方物理	PhysX / Chaos 的求解器、多线程岛屿、浮点	自研定点碰撞（圆、胶囊、格子），或只用于表现
版本差异	客户端版本 / 配置表不一致	开局校验版本与配置 hash；录像绑定版本

4.2 浮点为什么会“不一样”

先澄清一个常见误解：**IEEE 754 规定加、减、乘、除、开方必须正确舍入**，同样的指令、同样的操作数，在任何符合标准的硬件上结果都一样。不一样来自“指令序列不一样”：

- **结合律**：浮点加法不满足结合律，`(a+b)+c ≠ a+(b+c)`。编译器在 `/fp:fast` / `-ffast-math` 下会重排、向量化求和。
- **FMA 收缩**：`a*b+c` 被融合成一条指令，只舍入一次，结果和分两步不同；是否生成取决于编译选项和目标指令集（AVX2 / ARMv8 都有 FMA）。
- **中间精度**：老的 32 位 x87 用 80 位寄存器保存中间结果，何时溢出到内存（截断成 32 / 64 位）取决于寄存器分配。

- **超越函数**：`sin / cos / exp / pow` 不要求正确舍入，各平台、各版本的数学库实现不同。

实测 D1：同一组数，三种求和顺序 实测 D1

编译选项	正序累加	倒序累加	两两归约	double 参考值
/O2 /fp:precise	62374690816.0	62374449152.0	62692298752.0	62692298792.1
/O2 /fp:fast /arch:AVX2	62692098048.0	62374449152.0	62692298752.0	62692298813.2
/Od /fp:strict	62374690816.0	62374449152.0	62692298752.0	62692298792.1

表 4-1 100 万个 float 求和（determinism_demo.cpp D1）。同一份代码里换个顺序结果就差几十万；`/fp:fast /arch:AVX2` 下编译器把正序累加向量化了，结果又变了。

实测 D2 / D3：600 帧群体模拟的逐帧 hash 实测 D2 · D3

48 个单位在 20 m × 20 m 场地里转向、互相排斥、撞墙反弹（有 `sin / cos / sqrt` 和大量 `a*b+c`），每帧把所有坐标的比特做 FNV-1a 哈希。同一份源码编译三次：

编译选项	浮点版 600 帧 hash	定点版（Q16.16）600 帧 hash
`/O2 /fp:precise`	`6e2253426bfd71bd`	`85013554a55ddb54`
`/Od /fp:strict`（关优化）	`6e2253426bfd71bd`（与 precise 逐帧一致）	`85013554a55ddb54`
`/O2 /fp:fast /arch:AVX2`	`df02559d9a595945` **（第 1 帧起就不同）**	`85013554a55ddb54`

三个观察：① `/fp:precise` 和 `/fp:strict` 即使优化级别不同，结果也逐帧相同——说明“开优化”本身不是问题，**允许改变浮点语义的选项**才是；② `/fp:fast /arch:AVX2` 第一帧就分叉了（三角函数和 FMA 的差异在第一次计算就出现）；③ 定点版三份完全相同。现实中，Android 用 Clang、iOS 用 Apple Clang、PC 用 MSVC，CPU 有 ARM 有 x86，同一份浮点代码要逐位一致几乎不可能——这就是移动端帧同步普遍上定点数的原因。

4.3 定点数：用整数表示小数

Q16.16：用 32 位有符号整数 v 表示实数 $v/2^{16}$，整数部分 16 位（约 ±32768）、小数精度 $1/65536 \approx 1.5 \times 10^{-5}$。加减就是整数加减；乘除要借 64 位中间结果：

$$a \cdot b \;\to\; \frac{a_{\text{raw}} \cdot b_{\text{raw}}}{2^{16}}, \qquad \frac{a}{b} \;\to\; \frac{a_{\text{raw}} \cdot 2^{16}}{b_{\text{raw}}}$$

Q16.16 定点数核心运算（与 demos/determinism_demo.cpp 完全一致，只用整数）

```
typedef int32_t fx;                                  // 实数 = raw / 65536
const fx ONE = 1 << 16;
inline fx fmul(fx a, fx b) { return (fx)(((int64_t)a * b) >> 16); }   // 64 位中间结果防溢出
inline fx fdiv(fx a, fx b) { return (fx)(((int64_t)a << 16) / b); }

uint32_t isqrt64(uint64_t v) {                       // 逐位开方：结果向下取整，处处一致
    uint64_t r = 0, bit = 1ULL << 62;
    while (bit > v) bit >>= 2;
    while (bit) { if (v ≥ r + bit) { v -= r + bit; r = (r >> 1) + bit; } else r >>= 1; bit >>
    return (uint32_t)r;
}
inline fx fsqrt(fx a) { return a ≤ 0 ? 0 : (fx)isqrt64((uint64_t)a << 16); }

const fx PI = 205887, TWO_PI = 411775, HALF_PI = 102944;
fx fsin(fx x) {                                      // 区间规约到 [-π/2, π/2] + 7 次泰勒多项
    x %= TWO_PI;
    if (x > PI) x -= TWO_PI;
    if (x < -PI) x += TWO_PI;
    if (x > HALF_PI) x = PI - x;
    if (x < -HALF_PI) x = -PI - x;
    fx x2 = fmul(x, x), x3 = fmul(x2, x), x5 = fmul(x3, x2), x7 = fmul(x5, x2);
    return x - x3 / 6 + x5 / 120 - x7 / 5040;
}
```

精度实测：`fsin` 最大误差 1.66e-04，`fsqrt` 最大误差 1.53e-05（约 1 个最低位） 实测 D7 。定点数的代价：

代价	说明	常见处理
范围 vs 精度	Q16.16 若以厘米为单位只有 ±327 m；距离平方会溢出 32 位	平方、点积用 int64；大地图用 Q32.32（int64）或分块局部坐标
除法精度	小数除以大数会被截断成 0	调整运算顺序（先乘后除）；必要时临时提高精度
三角 / 开方	不能用 CRT	查表（表必须是常量，不能运行时用 `std::sin` 生成）或纯整数多项式
生态	物理、寻路、动画曲线都要定点版	逻辑层自研：简单几何碰撞、格子寻路；动画只用于表现

不用定点数行不行?

可以，但条件苛刻：所有端**同一编译器、同一版本、同一指令集、禁止 fast-math 和 FMA 收缩、自带超越函数。**单平台（比如同一主机平台）或全 PC 的项目有人这么做；跨 iOS / Android / PC 的移动 MOBA 基本都选定点数。另一个折中是软件模拟浮点（soft float），确定但慢一个数量级。

4.4 随机数：同一个种子，同样的调用次数

逻辑层随机数：xorshift64*（与 demos 中所有实验使用的同一个生成器）

```
struct LogicRng {                          // 只给逻辑层用；种子由服务器下发（HOK 文档：随帧下发）
    uint64_t s;
    explicit LogicRng(uint64_t seed) : s(seed * 0x9E3779B97F4A7C15ULL + 1) {}
    uint64_t Next() { s ^= s >> 12; s ^= s << 25; s ^= s >> 27; return s * 2685821657736338717
    int32_t Range(int32_t lo, int32_t hi) { return lo + (int32_t)(Next() % (uint64_t)(hi - lo)
};
```

种子一致只是必要条件。更隐蔽的 bug 是**调用次数不一致**：例如某个特效、UI 或“只在本地玩家执行”的逻辑也调了逻辑随机，于是本地端比别人多消耗一次随机数，从此所有随机结果都错位。规则是：**表现层永远不碰逻辑随机源**，需要随机就用自己的；凡是“只对本地玩家执行”的代码都不能改逻辑状态。HOK 把帧随机放在 C++ 逻辑层，种子随帧 hash 一起上报校验 HOK 文档 。

4.5 容器顺序与时间累加

实测 D5：同样 12 个实体 id，插入 `unordered_map`：默认桶数时遍历顺序是 `[1007, 3001, 2013, 4099, 10007, 7019, 6007, 5003, 8009, 9001, 11003, 12011]`；预留 1024 个桶并倒序插入后变成 `[12011, 11003, 10007, 9001, 8009, 7019, 6007, 5003, 4099, 3001, 2013, 1007]`；`std::map` 永远是升序。逻辑里“遍历所有单位结算伤害”如果依赖前者，一端扩容一次，结算顺序就变了。

实测 D6：每帧 `t += 1.0f/15` 累加 20 分钟：

编译选项	float 累加（s）	double 累加（s）	帧号 × 帧长（s）	float 误差（ms）
/O2 /fp:precise	1199.8835	1200.000000	1200.0000	-116.5
/O2 /fp:fast /arch:AVX2	1200.0017	1200.000000	1200.0000	1.7
/Od /fp:strict	1199.8835	1200.000000	1200.0000	-116.5

表 4-2 浮点累加时间的漂移（D6）。`/fp:precise` 下 20 分钟漂了约 -116 ms，而且不同编译选项漂移量不同——这正是 HOK 要求“逻辑层禁用墙钟、只用逻辑帧号、倍速用有理数”的原因。

4.6 蝴蝶效应与 desync 检测

把一个单位的 x 坐标改动 **1 个 ulp**（float 能表示的最小改动，约 0.0001 cm），再跑同样的模拟 **实测 D4**：

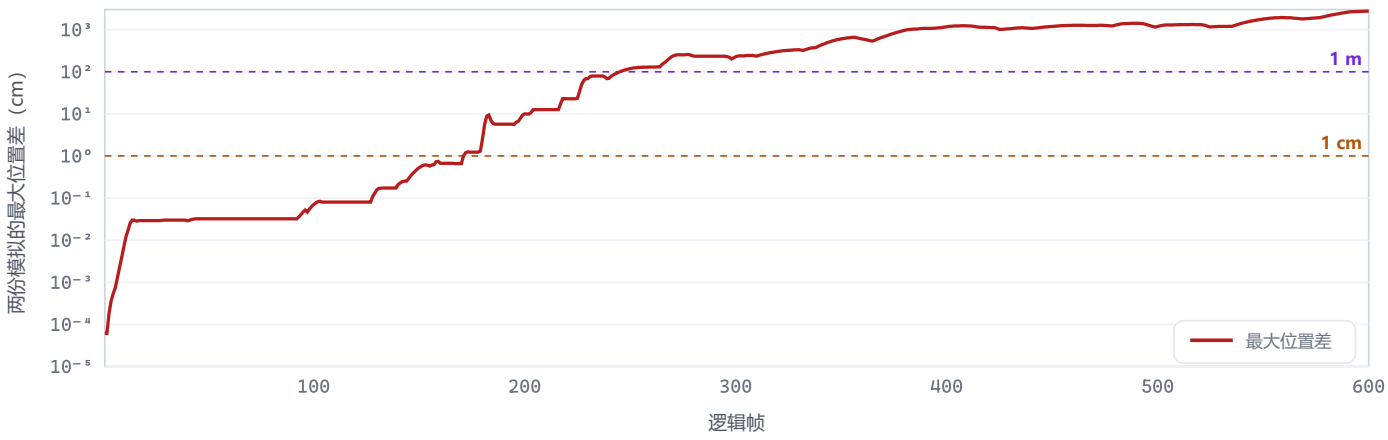


图 4-1 1 ulp 的扰动：hash 在第 1 帧就不同；差异通过排斥力在单位间传染、指数放大，第 172 帧超过 1 cm，第 246 帧超过 1 m——之后两份世界已经是两场不同的战斗。

这张图说明了两件事：**desync 无法自愈**（没有任何权威状态去纠正它）；**越早发现越好**（hash 第一帧就能看出来，而肉眼要到几十秒后）。工程上的检测与定位手段：

手段	做法	成本
帧 hash 上报比对	逻辑层周期性计算世界状态摘要，连同帧号、随机种子、KDA 等上报，服务器横向比对；HOK 用的就是这个，而不是“服务器养一个影子客户端” HOK 文档	极低，顺带反作弊
分层 hash	按系统（移动 / 技能 / Buff / 随机）或按实体分别算 hash，分歧时立刻知道是哪个系统	低
详细追踪开关	出问题时开启逐帧逻辑日志（输入、关键变量），两端日志 diff 找第一处不同	高，只在排查时开（HOK 有这个开关）
录像重放 + 二分	用录像在不同平台 / 版本重放，二分定位第一个分歧帧	离线，适合回归测试
确定性回归测试	CI 中跨编译器 / 平台跑同一份录像比对 hash（Riot 的 Delta Checker 就是从这类测试起步的 Riot 技术博客 ）	一次搭建，长期受益

4.7 HOK 的工程约束：把不确定的东西关在门外

HOK 的战斗逻辑整体放在不依赖 Unity 的 C++ GameCore 里 HOK 文档 ：定点数、可播种同步随机源、逻辑层单线程、禁用墙钟、倍速用有理数（分子 / 分母）、避免依赖不稳定遍历顺序。这些约束之所以能被**严格执行**，是因为逻辑层在物理上就访问不到引擎的浮点时间、随机数和容器——比代码规范和 Code Review 可靠得多。顺带的收益是同一份逻辑既能跑在手机上也能跑在服务器上（CS 模式，第 8 章）。

面试注意

“帧同步为什么要用定点数”是必考题。答题要点：① IEEE 754 基本运算本身确定，不确定来自编译器 / 指令集 / 数学库 / 运算顺序；② 跨平台无法锁死这些，所以用整数模拟小数；③ 定点数的代价（范围、精度、生态）以及怎么处理。能补一句“随机数调用次数也必须一致、表现层不能用逻辑随机”，说明你真踩过坑。

4.8 坑位清单

坑	症状	修正
定点查找表用 `std::sin` 在运行时生成	各平台的表本身就不同	表作为常量编译进去，或用整数算法生成
逻辑里用 `float` 做配置读取后再转定点	字符串 → float → 定点的舍入跨平台不同	配置表直接存定点整数，或用整数解析
按指针地址排序 / 用 `unordered_map` 遍历	偶发 desync，难复现	按稳定 id 排序
只对本地玩家执行的代码改了逻辑状态	本地端独有的分支 → 立刻分叉	本地分支只能改表现
表现层调用逻辑随机	随机序列错位	两套随机源
多线程更新单位	时好时坏	逻辑单线程，或确定性分片 + 固定合并顺序

本章速记

- 浮点不确定的来源：**重排 / FMA / 中间精度 / 超越函数库 / 归约顺序**；实测 `/fp:fast` `/arch:AVX2` 第 1 帧就分叉，定点三份一致。
- Q16.16：乘除借 int64，开方逐位法，三角函数纯整数多项式；注意范围、除法精度、生态。
- 随机：同种子 + 同调用次数；表现层不碰逻辑随机。容器：按稳定 id 排序。时间：只用帧号。
- desync 无法自愈：1 ulp 扰动第 246 帧变成米级差异；靠帧 hash 发现、分层 hash 和日志 diff 定位。

▶ **自测 1：同一台 PC 上，Debug 和 Release 版本的浮点结果会不同吗?**

▶ **自测 2：Q16.16 下计算两点距离，有什么坑?**

▶ **自测 3：发现线上有 0.1% 的对局 desync，怎么定位?**

CHAPTER 05

HOK 架构：三层分工与逻辑表现分离

C# 懂协议、懂表现，但对战斗一无所知；C++ 懂战斗，但不碰引擎、不收网络包。这条窄边界是整套帧同步能够长期维护的根本。

本章目标

- 画出 HOK 的三层架构（表现层 C# / 逻辑层 C++ GameCore / Relay 服务器），说清每层“不做什么”。
- 理解一条命令的三条命运：`OnReceive` 立即交付逻辑、`ExecLocal` 延迟表现、以及真正承载战斗表现的“结果事件”。
- 按顺序说出客户端主循环的七个阶段，以及“收在帧首、发在帧末”的原因。
- 理解 C# ⇄ C++ 共享内存 FIFO 的“预留 → 写入 → 提交”，以及单帧限流与延迟队列。
- 把这套分层与 Unreal（GAS 的 GameplayCue）、守望先锋（ECS）的同类设计对照起来。

30 秒面试版答案

HOK 分三层：Unity C# 只做输入采集、收发包、协议编解码和表现播放；独立的 C++ GameCore 做确定性战斗模拟（定点数、帧随机、单线程），不依赖引擎；Relay 服务器做帧号分配、汇总去重、定频广播和历史缓存。解出一条帧命令后立即写进 C++ 的共享内存（逻辑必须准点），表现另走一条路：战斗表现不是“收到命令就播”，而是 C++ 算完这一帧后把命中、伤害、死亡等**结果事件**推回 C#，由 AGE 时间轴系统执行；不依赖结果的反馈（技能指示器）在按键时就本地响应。帧号由 C++ 回报，C# 被逻辑拖着走，所以表现天然与逻辑同步。C# 与 C++ 之间是共享内存上的 FIFO 字节流，零序列化，单帧限流 1000 条防止卡顿雪崩。

5.1 三层全景



图 5-1 HOK 三层架构 HOK 文档 。每层的“不做什么”比“做什么”更重要：边界越窄，确定性越容易保证，层越容易替换（CS 模式就是把逻辑层搬到服务器）。

把战斗逻辑放进独立 C++ 层的四个收益：① 确定性（脱离引擎浮点、调度、容器）；② 可脱离 Unity 单独运行（服务器、自动化测试、录像校验都能跑）；③ 服务器复用同一份代码（CC / CS 双模式，第 8 章）；④ 性能可控（大批单位演算）。

5.2 一条命令的三条命运

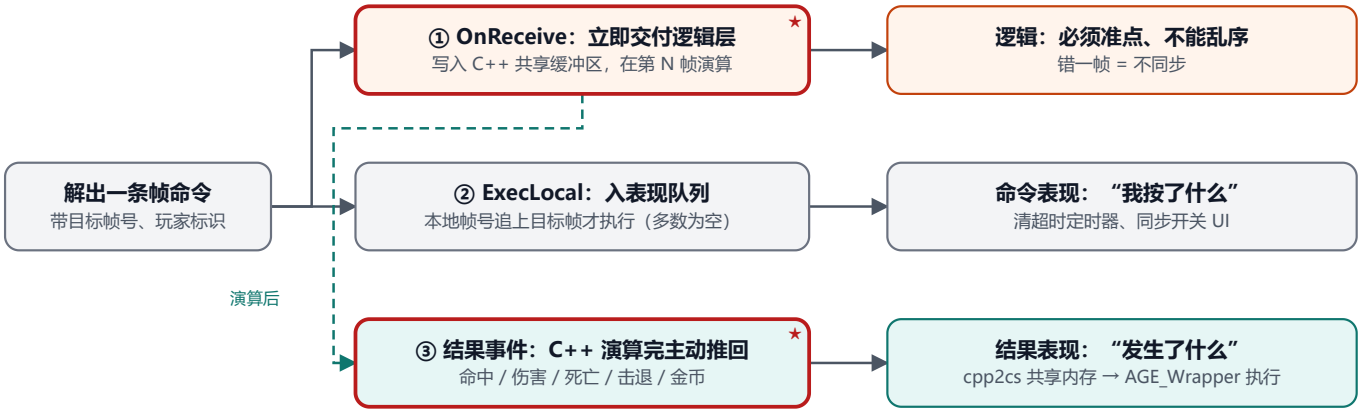


图 5-2 一条命令的三条命运 HOK 文档 。很多人以为表现靠路径 ②，事实正好相反：绝大多数命令的 `ExecLocal` 方法体是空的，技能特效、伤害飘字、死亡动画全部走路径 ③。

	逻辑（路径 ①）	命令表现（路径 ②）	结果表现（路径 ③）
对时间的要求	必须精确落在第 N 帧	大致跟上即可	跟着演算结果走
能否等待 / 乱序	都不能	都可以	可以延迟、可以限流
依赖演算结果	—	不依赖	**完全依赖**
例子	位移、技能、伤害结算	移除“等待服务器确认”的超时定时器、聊天输入框开关	命中特效、伤害飘字、死亡、音效
占比	全部命令	很小（多数为空）	绝大部分战斗表现

一个佐证：早期版本里“技能指示器显示 / 关闭”写在命令的 `ExecLocal` 里，后来被挪到了输入处——玩家一按键指示器就本地响应，根本不等帧回来 HOK 文档 。这就是**预表现**：不依赖演算结果的反馈，一律按键即响应；依赖结果的（中没中、扣多少），老实等 C++ 通知。

> **对照 Unreal：GameplayCue 是同一个思想**
>
> GAS 把“逻辑效果”（GameplayEffect 改属性）和“表现”（GameplayCue：粒子、音效、镜头抖动）严格分开：Cue 由效果的执行 / 添加 / 移除**触发**，在各端本地执行，可以走不可靠多播、可以被本地预测、丢了也不影响逻辑 UE 文档 。同样是“逻辑定义发生了什么，表现决定长什么样”。区别只在于 UE 的逻辑结果来自服务器复制，HOK 的来自本地确定性演算。

AGE：控制权在 C++，执行在 C#

AGE 是 HOK 的表现时间轴系统：一次技能编排成一条时间轴，挂着“持续段”和“瞬时事件”（播动画、触发粒子、震屏、改材质、切镜头、刷 HUD）。C++ 持有时间轴和帧同步时钟，进入 / 离开某段、变速时通过 cpp2cs 共享内存发出 `NtfAgePlayAnimDurEnter / Leave / SpeedModified` 等指令；C# 侧数十种 `AGEWrapXxx` 只负责实例化、按帧推进、收到结束指令后销毁。wrapper 的 id 由逻辑层的演出序号 + 类型编号组成，**局内不存在 C# 自行读演出文件播放的路径** HOK 文档 。代码量大说明“执行复杂”，不说明“控制权在 C#”。

5.3 帧号由谁说了算

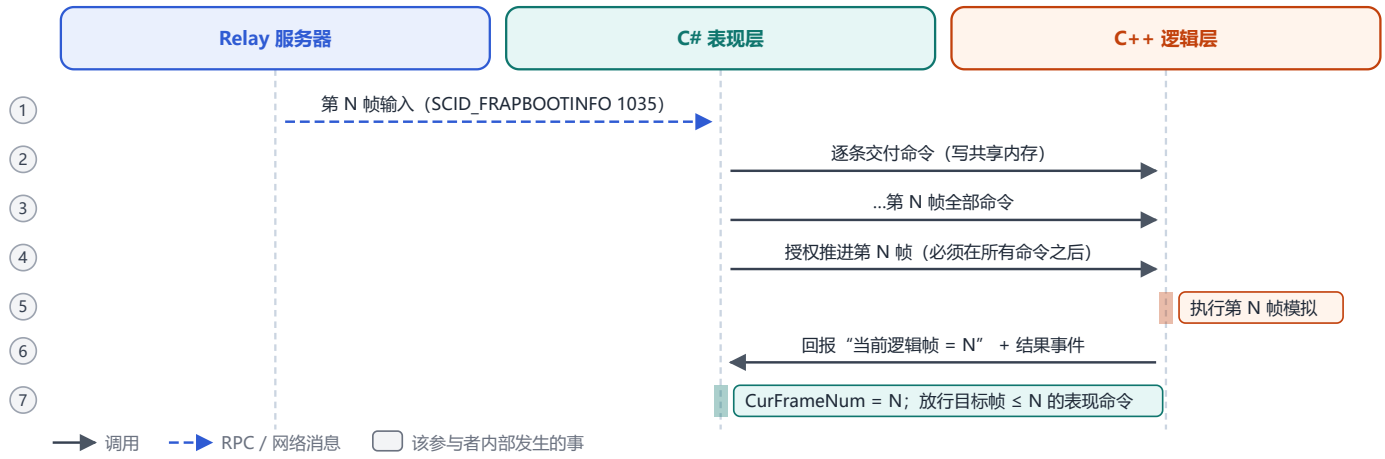


图 5-3 本地帧号不是 C# 自己累加的：C# 只负责“喂料”和“授权”，跑多快由 C++ 说了算 HOK 文档 。“授权推进”放在所有命令之后，否则第 N 帧会缺命令。

客户端逻辑帧率可以高于服务器广播帧率（例如服务器 15 帧/秒、客户端 30 帧/秒），帧号按 $\text{客户端帧号} = \text{服务器帧号} \times \frac{\text{服务器帧间隔}}{\text{客户端帧间隔}}$ 换算，表现插值发生在多出来的帧里。

5.4 一帧的生命周期：七个阶段



图 5-4 客户端主循环七阶段 HOK 文档 。② 的“抽干”和 ⑤ 的“限流”是一对：收包不能积压，处理事件却要削峰。

5.5 C# 与 C++ 如何对话：共享内存上的 FIFO

两层之间既不是普通的跨语言函数调用，也不是操作系统的管道，而是**“用共享内存实现的 FIFO 字节流”**：一次通信只有“取指针 + 写内存 + 提交游标”，没有序列化、系统调用和内核拷贝，成本是纳秒级 HOK 文档 。

预留 → 写入 → 提交（通用示意，非 HOK 源码）

```
// 单生产者（C#）/ 单消费者（C++ 逻辑层）的追加式字节流
struct CmdStream {
    uint8_t* buf; uint32_t cap;
    uint32_t write = 0;        // 已提交的写游标：消费者只看得到 [read, write)
    uint32_t pending = 0;      // 预留但尚未提交的位置
    uint32_t read = 0;

    void* Reserve(uint32_t n) {            // ① 预留 n 字节，返回裸指针
        if (pending + n > cap) Grow(pending + n);
        void* p = buf + pending; pending += n; return p;
    }
    void Commit() { write = pending; }     // ③ 提交：这一批数据对逻辑层“原子地”生效
    template <class F> void Drain(F&& exec) {   // 逻辑层推进时按顺序取出执行
        while (read < write) {
            uint32_t fn; memcpy(&fn, buf + read, 4);          // 4 字节函数编号
            read += 4 + exec(fn, buf + read + 4);             // exec 返回参数长度
        }
        read = write = pending = 0;                            // 本帧取空后复位
    }
    void Grow(uint32_t need);
};
// C# 侧：p = Reserve(4 + sizeof(Args)); *(int*)p = FN_MOVE; *(Args*)(p+4) = args;  // ② 不安全
//         ...  一帧所有命令写完 → Commit()
```

通道	承载	本地模式	服务器（CS）模式
主逻辑通道	逻辑层调用与事件	共享内存	**本地攒批（约 2 KB 或帧末）→** `CSID_GAMECORE_CLT_INPUTMSG` **网络包**
轻量逻辑通道	轻量逻辑调用	共享内存	共享内存
表现通道	表现插值数据	独立的“表现缓冲区”	同左
反向（C++ → C#）	结果事件	C# 启动时分配约 1 MB 非托管缓冲，每帧把地址交给 C++，C++ 直接填；填满则 C# 容量翻倍再拉，直到取空	

两个保护 HOK 文档：

- **单帧限流**：反向通道每帧最多处理 1000 条（服务器可配），超出留到下一帧——避免“事件多 → 这帧卡 → 下帧事件更多”的恶性循环。
- **延迟队列**：标记为“可以晚点处理”的事件进入独立的约 512 KB 缓冲错峰消费。

另有一条同步立即通道（C++ 直接回调 C#），只用于极少数必须立即得到结果的场景，并有明确约束（跨线程同步回调在部分编译后端有性能代价）。

发散：守望先锋的 ECS 与“确定性子集”

守望先锋（GDC 2017《Overwatch Gameplay Architecture and Netcode》）是状态同步，但同样把“需要按输入预测的系统”和“纯表现系统”分开：需要预测的 System 不用 `Update` 而用 `UpdateFixed`，在固定的 16 ms 命令帧上运行（比赛服 7 ms），这部分逻辑在客户端和服务器上用同一份代码、同一组输入演算，所以能在客户端预测、出错时回滚重放 GDC 2017。HOK 是把**整个战斗**做成确定性子集，守望先锋是只把**本地玩家相关的那部分**做成确定性子集——这是理解第 13 章 CMC 的钥匙。

面试注意

被问“帧同步里逻辑和表现怎么分离”，要答出三点：① 逻辑层独立（不依赖引擎，保证确定性）；② 表现由**演算结果**驱动，而不是由**输入命令**驱动（否则会出现“特效播了但其实没打中”）；③ 不依赖结果的反馈做预表现（按键即响应）。能补一句“表现可以限流、可以晚到，逻辑不行”就很完整了。

5.6 坑位

坑	后果	正确做法
收到“释放技能”命令就播技能特效	技能被打断 / 没蓝时特效仍然播了，十端表现不一致	等 C++ 推回“技能真正释放”的结果事件
C# 自己累加帧号	与逻辑帧号漂移，表现提前或滞后	帧号以 C++ 回报为准
授权推进放在交付命令之前	第 N 帧缺命令，直接不同步	一帧全部命令交付后再授权
收包只读一个	队列积压，越积越卡	每帧循环抽干
反向事件一次全处理	团战大招时单帧卡顿	单帧限流 + 延迟队列
表现层改逻辑状态	只有本端改了 → desync	表现只读逻辑结果

本章速记

- 三层：C#（输入 / 网络 / 表现，不做战斗判断）· C++ GameCore（确定性模拟，不碰引擎）· Relay（定序与可靠投递，不做战斗逻辑）。
- 三条命运：**OnReceive 立即入逻辑**、ExecLocal 延迟表现（多数为空）、**结果事件驱动战斗表现**（AGE：C++ 决策、C# 执行）。
- 帧号被 C++ 拖着走；七阶段：输入 → 抽干收包 → 窗口维护 → 推进 → 反向通知（限流）→ CS 流 → 帧同步器；帧末统一发包。
- C# ⇄ C++：共享内存 FIFO，预留 → 写入 → 提交；同一接口在 CS 模式下换成网络包。

▶ **自测 1：为什么技能指示器可以按键即显示，而伤害飘字必须等 C++?**

▶ **自测 2：反向通道为什么要限流？不限流会怎样?**

▶ **自测 3：GameplayCue 和 HOK 的结果事件有什么相同点和不同点?**

CHAPTER 06

HOK 下行：冗余广播、900 帧窗口、补发与追帧

“不等人”把丢包的代价全部留给了丢包者，所以下行需要一整套让丢包者自救的机制：冗余能补的冗余补，冗余补不了的主动要，要不回来的整段追。

本章目标

- 说清下行三层兜底的分工边界：丢帧 ≤ k 靠冗余，k < 丢帧 < 900 靠补发，超窗或断线靠 Recover。
- 理解冗余帧为什么“倒序 + 命中即截断”，并用实测和闭式解回答“一包带几帧够用”——尤其是突发丢包下为什么冗余会失灵。
- 能手画 900 帧接收窗口的状态变化：入窗即消费、找洞、区间合并请求、超窗转 Recover。
- 按时序讲完 Recover：问大厅 → 重连 Relay → 拉缓存分片 → 回放 → 追帧 → 上报完成；以及两层缓存、快速恢复。

30 秒面试版答案

下行三层兜底：第一层，服务器每个广播包都带最近 k 帧（倒序排列，客户端从新往旧解，遇到已处理的帧就停，平时只解一帧），挡住零散丢包；第二层，客户端 900 帧循环接收窗口发现“期待帧缺失但后面的帧已到”或 90 ms 没推进，就把缺失帧合并成区间请求补发（1036），请求频率两级：卡顿初期约 120 ms 一次，持续卡顿降到约 1000 ms 一次防请求风暴；第三层，帧号超出窗口或断线重连，走 Recover：先问大厅“我还在局里吗”，重连 Relay 后用 1090 请求缓存帧、1091 分片返回，顺序回放，缺口小于 15 帧算追上，上报 1093。补发回来的帧和正常帧走完全相同的路径，客户端不需要特殊逻辑。

6.1 三层兜底的边界



图 6-1 下行三层兜底 HOK 文档 。窗口负责“缓存排序去重”，补发与追帧只负责“把洞填上”——这个分工让补回来的帧不需要任何特殊处理。

6.2 第一层：一包多帧的冗余广播

服务器广播第 N 帧时，同一个 `SCID_FRAPBOOTINFO(1035)` 包里还带着 N−1、N−2…N−k+1 帧，于是同一帧会在连续 k 个包里重复出现。包内按**从新到旧**排列，客户端从最新帧开始遍历，遇到已经处理过的帧就**立即停止解包** HOK 文档 ：

倒序 + 命中即截断（示意）

```
void OnFramePacket(const FramePkt& pkt) {          // pkt.frames[0] 是最新帧，越往后越旧
    for (const FrameInfo& f : pkt.frames) {
        if (f.no < window.beg || window.Has(f.no))   // 已经消费过 / 已在窗口里
            break;                                    // 更旧的必然也处理过 → 停止解包
        window.Put(f);                                // 解包、入窗
    }
    window.ConsumeContiguous();                       // 入窗即消费（第 6.3 节）
}
// 不丢包时，每个包只解第一帧；只有真丢了包，才会往旧帧多解几帧。
// 如果冗余帧是正序排列的，这个 break 就不成立了——所以工程注释特意强调“必须倒序”。
```

一包带几帧够用？实测 + 闭式解 实测 E5

“需要补发”等价于“某一帧所在的连续 k 个包全丢了”。在两种丢包模型下仿真 40 万个包：

- **随机丢包**：每个包独立地以概率 p 丢失，k 个全丢的概率是 p^k。
- **突发丢包**（Gilbert-Elliott 两状态模型）：好状态不丢、坏状态全丢，平均丢包率 p，平均连丢长度 L。移动网络、Wi-Fi 切换、基站切换都属于这一类。

推导 • 突发丢包下“连续 k 包全丢”的概率

① $$\pi_B = \frac{p_{gb}}{p_{gb} + p_{bg}} = p$$ 坏状态的稳态概率 = 平均丢包率；$p_{bg} = 1/L$（每次离开坏状态的概率）

② $$P(\text{第 } n \text{ 包丢}) = \pi_B = p$$ 坏状态即丢包

③ $$P(\text{第 } n+1 \text{ 包也丢} \mid \text{第 } n \text{ 包丢}) = 1 - p_{bg} = 1 - \tfrac{1}{L}$$ 马尔可夫性：只取决于上一个状态

④ $$P_{\text{hole}}(k) = p\left(1 - \tfrac{1}{L}\right)^{k-1}$$ 连乘 $k-1$ 次；对比随机丢包的 p^k

随机丢包时每多带一帧，失败概率乘以 p（如 0.05）；突发丢包时只乘以 1−1/L（L=4 时是 0.75）——多带几帧几乎没用

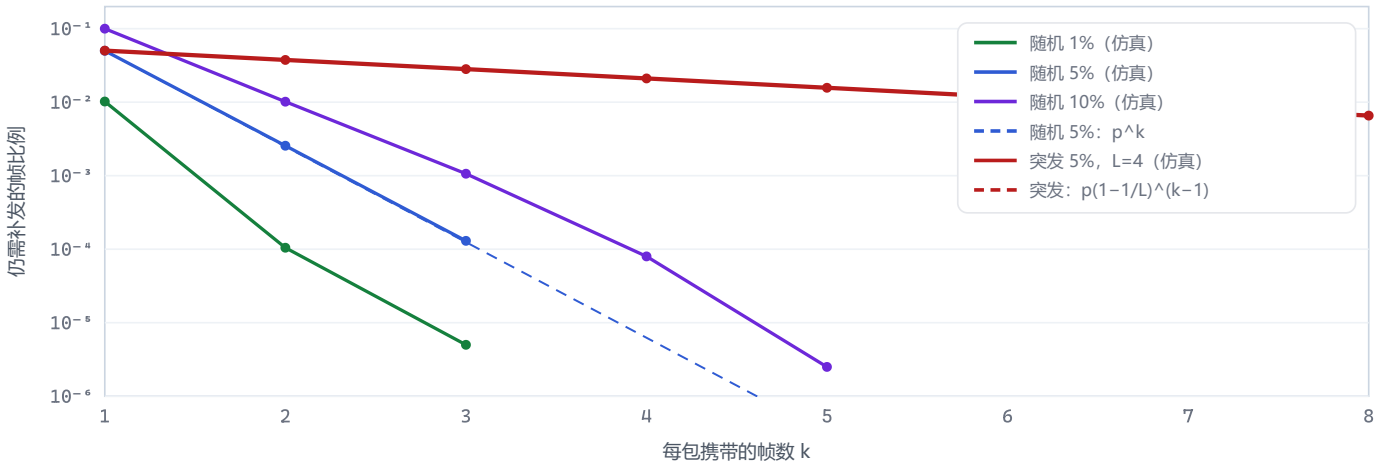


图 6-2 冗余长度 k 与仍需补发的比例（对数坐标；仿真 40 万包，点缺失表示一次都没发生）。随机丢包下 k=3 就把 5% 丢包压到约万分之一；突发丢包（同样 5% 平均丢包率）下 k=8 仍有约 0.7% 的帧需要补发——冗余挡不住连丢，这正是第二层“补发”存在的理由。

k	每包字节（估）	随机 1%	随机 5%	随机 10%	突发 5%（L=4）仿真	突发闭式解
1	60	1.02e-02	5.00e-02	1.00e-01	5.03e-02	5.00e-02
2	84	1.05e-04	2.55e-03	1.02e-02	3.77e-02	3.75e-02
3	108	5.00e-06	1.30e-04	1.06e-03	2.81e-02	2.81e-02
4	132	0.00e+00	0.00e+00	8.00e-05	2.10e-02	2.11e-02
5	156	0.00e+00	0.00e+00	2.50e-06	1.57e-02	1.58e-02
6	180	0.00e+00	0.00e+00	0.00e+00	1.17e-02	1.19e-02
7	204	0.00e+00	0.00e+00	0.00e+00	8.80e-03	8.90e-03
8	228	0.00e+00	0.00e+00	0.00e+00	6.58e-03	6.67e-03

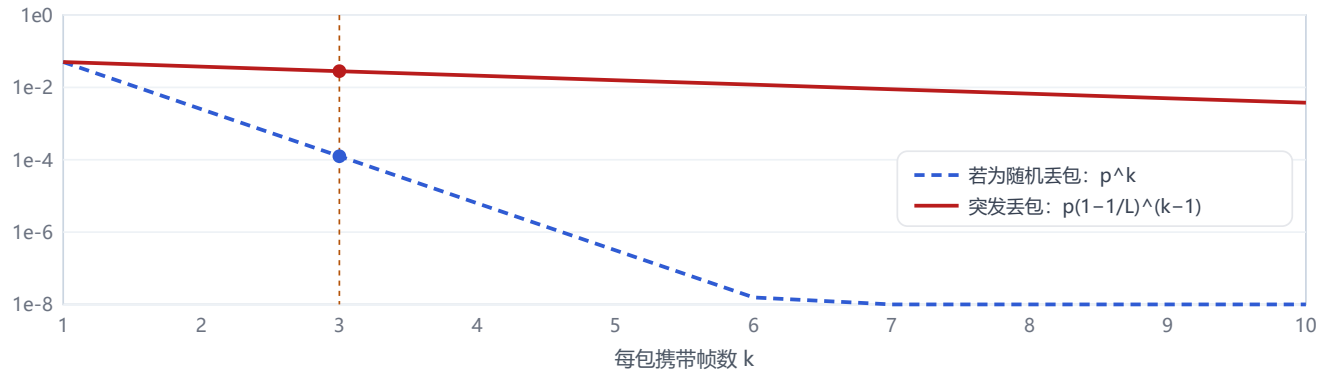
表 6-1 E5 数据。字节数按“包头 36 B + 每帧 24 B”估算；0 表示 40 万包里没出现过。构建脚本会断言突发丢包的仿真值与闭式解相差不超过 8%。

交互实验 · 一包带几帧够用?

平均丢包率 p **5.0%**

平均连丢长度 L（1 = 随机丢包） **4**

每包携带帧数 k **3** 逻辑帧率 15 帧/秒（HOK）



k = 3 时：

随机丢包　仍需补发比例 1.25e-4，每分钟约 0.11 帧

突发丢包　仍需补发比例 2.81%，每分钟约 25.31 帧

```
多带 1 帧：随机丢包失败率 ×0.050，突发丢包失败率 ×0.750
每个下行包约 108 B，15 帧/秒 → 1.58 KB/s（每多 1 帧 +0.35 KB/s）
```

结论：**冗余是对付零散丢包的利器，对付突发丢包几乎无效。**HOK 的 k 由服务器配置，配合“补发 + 追帧”兜底，而不是靠无限加大 k——k 每多 1 帧，每个下行包都要多带一帧的字节，而突发丢包时收益却只有 25%。

6.3 第二层的舞台：900 帧接收窗口

客户端维护一个长度 900（约 60 秒）的循环数组 `_receiveWindow`，用 `下标 = (帧号 - _basFrqNo) % 900` 映射，数组不移动、只滚动基址 **HOK 文档**：

变量	含义
`_begFrqNo`	**期待帧**：下一个必须处理的帧号；同时写进每个上行包头，充当上行 ACK
`_maxFrqNo`	已收到的最大帧号
`_basFrqNo`	循环数组基址（帧号 → 下标换算）
`FRQ_WIN_LEN`	窗口长度，固定 900

收到帧号 f 的三种命运：$f < \text{beg}$ → 已过期，丢弃；$\text{beg} \le f < \text{beg} + 900$ → 入窗，然后**立刻**从期待帧开始连续出窗、交付逻辑层，直到遇到空洞；$f \ge \text{beg} + 900$ → 中间有洞且超出窗口，转 Recover。每帧一次的“帧窗口维护”（主循环 ③）**不消费帧**，只做三件事：找洞决定是否补发、判断是否超时未推进、更新卡顿状态机——“入窗即消费，维护只管找洞、要洞、记账”。

下图用一个缩小到 16 帧的窗口演示完整过程。每一步的状态都是构建脚本按上面的规则**真实执行**出来的（`src/netfigs.py` 的 `win_run`）：



图 6-3 接收窗口逐步演示（窗口缩小为 10 帧便于观察，真实为 900；紫色虚线是窗口上界 beg+10）。注意 ② 的“倒序截断”、④ 后出现两个空洞、⑤ 的区间合并、⑥ 补发帧与正常帧走同一条路径、⑦ 超窗转 Recover。

补发的触发、请求与记账

要点	内容 HOK 文档
空洞判定	`_receiveWindow[beg] == null` 且 `_maxFrqNo > beg` 且 `_maxFrqNo < beg + 900`（第三条不满足就该走 Recover）
触发一：快速补洞	事件驱动：后面的帧到了、期待帧还缺 → 立即请求；有最小请求间隔做节流（后续包可能马上就到）
触发二：超时补发	距上次成功推进超过 90 ms → 请求“之后的都给我”；频率两级：卡顿初期约 120 ms 一次，超过紧急时长后约 1000 ms 一次
请求内容	扫描 `[beg, min(max+1, beg+900))`，把连续缺失合并成区间；区间数有上限，**最后一个区间留给“窗口之外”**，用 −1 作区间尾表示开放区间
补回的帧	仍用 1035 返回，照常入窗、出窗、交付——客户端没有任何“补发帧专用”逻辑
两次记账	请求时记“区间 + 时刻”，该区间的帧真正渲染出来后再记一次耗时——把“等网络补帧”和“本机处理慢”分开度量

为什么补发请求要降频?

持续卡顿通常意味着链路已经拥塞或断开。此时每 120 ms 发一次请求，请求本身就会和下行抢带宽，服务器每收到一次还要回一大段帧——“越卡越请求，越请求越卡”。两级频率是典型的**退避**（backoff）思想。第 3 章 E3 实测里，断流者在 2 秒断流窗口内卡了 2035 ms，多出来的几百毫秒正是退避的代价：用恢复速度换系统稳定。

6.4 从帧数据到逻辑推进

一帧 1035 的 payload 是这一帧内所有参与者的操作集合，分四类 HOK 文档：

类型	含义	例子
CC	客户端之间同步的通用帧命令	复杂操作、状态同步
CS	需要经过服务器处理的帧命令	技能释放、普攻、学技能、买卖装备
SC	服务器主动下发的帧命令	出生、传送、PVE 对象通知
通知类	非玩家操作的服务器通知	游戏结束、暂停、投降、掉线 / 重连、AI 接管

分派用“启动期反射注册 + 运行期查表”：启动时扫描带标注的命令类型和构造器，按编号注册；运行期按编号造命令对象（来自对象池，避免 GC），填入帧号与玩家标识，推入帧同步器：立即 `OnReceive` 交付逻辑层，同时排队等表现到期；一帧全部交付后才**授权推进。**消费每一帧时顺手做两件事：写入录像缓冲；记录已处理的数据长度（断线重连时“从哪补起”）。

实践中的坑（来自原文档）

协议处理函数的注册需要“类型被登记进列表”和“方法带协议标注”两个条件同时满足；只加标注、忘了登记类型，处理函数永远不会被调用，而且**不报错。**反射注册的好处是新增命令不改分派代码，坏处就是这种静默失败——建议启动时打印注册表并做自检。

6.5 第三层：Recover 追帧与断线重连

两种情况必须向服务器索取一段历史：帧号超出窗口（本地落后太多，旧数据已被循环覆盖），以及断线后重连。本质相同：**缺失帧数超过了本地窗口能容纳的范围。**

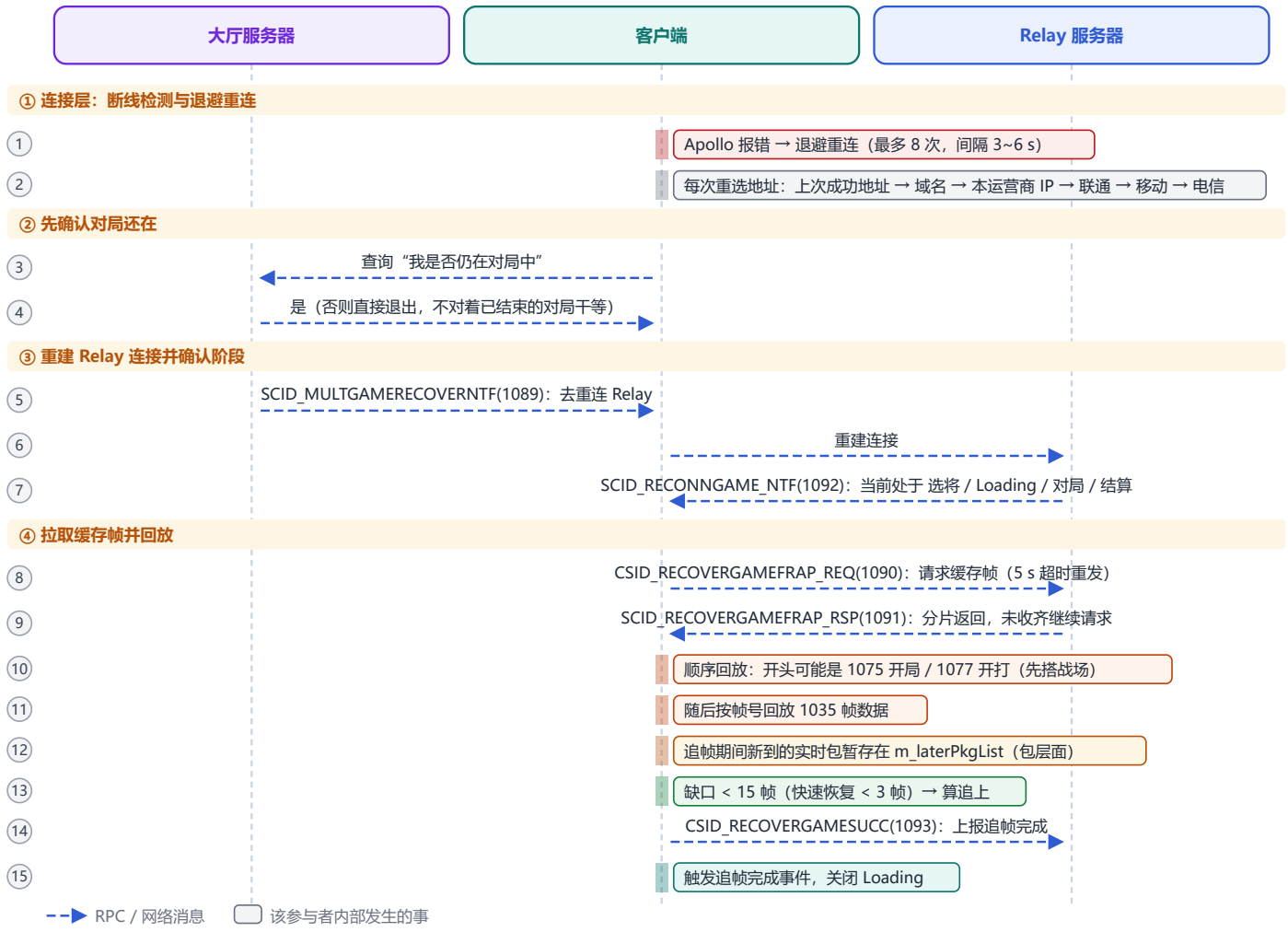


图 6-4 Recover 完整时序 HOK 文档 。注意第 ② 步：先问大厅“局还在不在”，避免玩家对着一个已经结算的对局等重连。



图 6-5 Recover 三态状态机。“处理缓存帧”期间屏蔽普通补发，避免两套机制互相干扰——一个经典的**模式互斥**设计。

缓冲	层次	存什么
`m_laterPkgList`	UDP **包**层面	追帧期间到达、尚未处理的完整网络包；追帧结束后统一走正常分发
`_receiveWindow`	**帧信息**层面	帧数据，无论来自实时包还是历史回放

两者不是“两个缓存”，而是两个不同的抽象层：历史回放往帧窗口里填，实时包在包层面排队，互不干扰。另外，收包后、分发前有一道**重连闸门**：处于恢复流程时，包被扣下暂存，等历史帧回放完再处理。

快速恢复是一个激进但有效的优化：追帧期间，反向通道**只放行“帧推进通知”**，特效、音效、UI 等事件全部跳过——追帧的唯一目标是让逻辑帧尽快跑上去，此时的表现毫无意义（甚至会特效刷屏）。代价是追帧期间画面朴素，换来更短的黑屏 HOK 文档 。这再次体现了第 5 章逻辑表现分离的价值：如果表现与逻辑耦合，这个开关根本做不出来。

追帧要多久？一个估算 推断

设断线 T_d 秒，逻辑帧率 $f = 15$，本机全速演算（跳过表现）的速度是 s 倍实时。追帧期间对局仍在推进，所以追上需要满足 $s \cdot t = fT_d + ft$，即 $t = \dfrac{T_d}{s-1}$。断线 30 秒、全速 20 倍实时时约 1.6 秒；如果不跳过表现、只能跑 3 倍实时，就要 15 秒——这就是快速恢复的意义。帧同步录像的“快进”、观战的“追直播”是同一个公式。

6.6 对照：状态同步怎么做这些事

问题	HOK 帧同步	Unreal 状态同步
零散丢包	一包多帧冗余	属性复制：丢了不重发旧包，下次发最新值；不可靠 RPC：丢了就丢了；可靠 RPC：重传
乱序	帧号 + 接收窗口重排	包序号；属性只认最新；可靠 RPC 在 Actor 通道内按序
较长丢包	主动补发区间	NAK 后把丢失包里的属性重新标脏，发送当前值
断线重连	拉缓存帧整段重放（需要时间追帧）	重新建立通道，服务器发一份当前状态（初始复制）即可；PlayerState 可被保留以恢复身份
代价	重连慢、必须保留全部历史输入	重连快；但无法“回到过去”（除非服务器另外录制）

面试注意

“帧同步怎么处理丢包”的满分结构：**先分方向**（上行丢操作、下行丢推进），**再分层级**（冗余 → 补发 → 追帧，各自的触发条件与边界），**最后讲取舍**（冗余对突发丢包无效、补发要退避、追帧要跳过表现）。能写出 p^k 与 $p(1-1/L)^{k-1}$ 的对比，说明你真的想过“k 设多大”。

6.7 坑位

坑	后果	修正
冗余帧正序排列	每个包都要解 k 帧，CPU 浪费 k 倍	倒序 + 命中即截断
只靠冗余、不做补发	突发丢包时频繁卡死	补发兜底（图 6-2）
补发请求不节流 / 不退避	弱网下请求风暴，越卡越卡	最小间隔 + 两级频率
补发帧走特殊路径	两套逻辑，状态容易不一致	补发帧照常入窗
追帧期间仍做普通补发	两套机制争抢同一段帧	状态互斥：处理缓存帧时屏蔽补发
追帧期间播放全部表现	追帧变慢、特效刷屏	快速恢复：只放行帧推进通知
重连前不确认对局状态	对着已结束的对局空等	先问大厅

本章速记

- 三层：**冗余（n ≤ k）→ 补发 1036（k < n < 900）→ Recover 1090/1091/1093（超窗 / 断线）**。
- 冗余倒序 + 命中即截断；随机丢包 p^k，突发丢包 $p(1-1/L)^{k-1}$——k=8 时突发 5% 仍有 0.7% 需补发。
- 窗口：900 槽循环数组，期待帧兼作上行 ACK；入窗即消费；维护只找洞、要洞、记账；区间合并 + 末区间 −1 开放。
- 补发两级频率（≈120 ms / ≈1000 ms）；Recover 三态 + 两层缓存 + 快速恢复只放行帧推进通知。

▶ **自测 1：为什么期待帧号可以当 ACK 用?**

▶ **自测 2：补发请求的最后一个区间为什么要留给“窗口之外”？**

▶ **自测 3：断线 10 秒和断线 3 分钟，恢复过程有什么不同?**

CHAPTER 07

HOK 上行：命令冗余、技能保序与确认

下行丢了有服务器冗余广播兜底，上行丢了就是丢了——服务器不会替你补。所以上行要自己做冗余，而且要区分“丢一次无所谓”的移动和“丢了就是真丢了”的技能。

本章目标

- 说清上行包里除了命令还带了什么（期待帧 ACK、命令序号），以及“只在战斗中才产生有效输入”的原因。
- 对比两种冗余：短冗余（当前 + 前两条，约 33 ms 重发）与技能全量冗余（所有未确认技能，约 200 ms 重发、1 s 超时）。
- 用实测回答：在随机丢包和突发丢包下，三种发送策略的送达率、延迟、带宽各是多少。
- 掌握技能序号的三个边界、三重保护和确认规则，以及追帧后序号 +100 的处理。

30 秒面试版答案

上行包除了命令，还带期待帧号（兼作下行 ACK）和递增的命令序号。冗余分两种：大多数关键命令（普攻、学技能、买装备）用**短冗余**——每个包带“当前 + 前两条”，约 33 ms 重发一次，代价低，能挡住零散丢包；三个主动技能用**全量冗余**——只要有技能没被确认，每约 200 ms 就把所有未确认技能重发一遍，按序号保序。确认的方式是客户端在下行帧里看到自己的技能被执行；序号等于期望值正常出队，大于则说明中间有技能丢了，小于则是重复。另有三重保护：同槽位重复点击去重（蓄力技豁免）、1 秒超时丢弃、队满丢最老；追帧结束后序号加一个大偏移（如 +100）把迟到的旧指令隔离掉。下行卡顿时主动降低上行重发频率，把带宽让给补帧。

7.1 从手指到上行包



图 7-1 一次操作的上行路径 HOK 文档 。非战斗状态（加载中、已结算）的输入直接丢弃，避免污染对局；AI 操作走 `CSID_AI_UPERMSG`；单机 / 训练模式把命令写进本地回环缓冲、直接推入本地帧同步器，同一套命令机制复用。

序列化分两种形态：简单命令（移动）直接填协议结构体字段，极省字节；复杂命令（多参数目标技能）先打包成二进制再塞进缓冲。一次发送通常会捎带若干条历史命令——这就是冗余。

7.2 两种冗余策略

对比项	短冗余	技能全量冗余
适用	普攻、学技能、买卖装备、蓄力技、联合技等关键命令；**普通移动不入队**（下一条移动会覆盖它）	三个主动技能（目标型 / 方向型 / 位置型）
队列	3 槽环形缓冲	协议上限（较大）
重发间隔	约 33 ms	约 200 ms
每次发送内容	当前 + 前 2 条	**所有未确认的技能**
次数限制	每条最多捎带若干次	直到确认，或 1 s 超时，或队满被挤掉
保序	无	有（技能序号）
开关	常开	服务器开局下发（关闭时技能走短冗余）

所有发包统一在帧末 flush：冗余重发定时到期 → 技能重发定时到期 → 待确认的可靠包超时重发 → 本帧新入队的普通消息。一个精巧的细节：**下行正在卡顿时，主动降低上行重发频率**——下行卡顿往往意味着拥塞，此时疯狂重发上行只会和补帧抢带宽 HOK 文档 。

7.3 实测：三种策略谁更可靠、谁更省 实测 E7

模型：上行每 33 ms 一个发送时隙（帧末 flush），同一时隙的包同生共死；单程 30 ms + 指数抖动（均值 20 ms）；10 分钟内 678 个技能（30% 是 80 ms 内的连招）。“确认”定义为技能到达服务器后约 60 ms 客户端在下行帧里看到它。两种信道：随机丢包 5%；突发丢包 10%、平均连丢 8 个时隙（约 264 ms，模拟 Wi-Fi 信号衰减）。

信道	策略	送达率	平均延迟 ms	p99 延迟 ms	每技能平均发送份数	生效顺序颠倒
随机丢包 5%	单发	93.36%	67	140	1.00	1
随机丢包 5%	短冗余（3 份）	100.00%	66	115	3.00	0
随机丢包 5%	技能全量冗余 + 序号	100.00%	79	310	1.26	0
突发丢包 10%（平均连丢 8 包 ≈264 ms）	单发	90.86%	67	142	1.00	1
突发丢包 10%（平均连丢 8 包 ≈264 ms）	短冗余（3 份）	92.18%	65	118	3.00	0
突发丢包 10%（平均连丢 8 包 ≈264 ms）	技能全量冗余 + 序号	99.11%	100	772	1.39	0

表 7-1 E7 汇总（demos/netsync_demo.cpp）。



图 7-2 技能送达率。随机丢包下短冗余就够了；突发丢包把 0 / 33 / 66 ms 发出的三份一起吞掉，短冗余只比单发好一点，全量冗余因为每 200 ms 重发一次、跨越了整段突发，仍能送达 99.1%。

从数据里读出的四个结论

1. **冗余的价值取决于丢包形态**：随机 5% 时短冗余已经 100.0%；突发丢包时同样 3 份只有 92.2%——和第 6 章下行冗余的结论完全一致：**挨在一起的副本会一起死。**
2. **全量冗余反而更省**：每个技能平均只发了 1.26 份（随机）/ 1.39 份（突发），远少于短冗余的 3 份——因为它“确认了就停”，只有真丢了才重发。这是**选择性重传**对**盲目重复**的胜利。
3. **代价是尾延迟**：全量冗余的 p99 延迟是 772 ms（突发）——重发间隔 200 ms，丢了就要多等一轮。所以技能用全量冗余，而对延迟最敏感、丢一次无所谓的移动根本不入队。
4. **序号消除了乱序**：单发时出现了 1 次“后按的技能先生效”（抖动大时连招的第二个包先到）；全量冗余的包内按序号排列，服务器按序号生效，颠倒次数为 0。

> **对照 Unreal：CMC 的 ServerMove 也是“带上旧的”**
>
> CharacterMovementComponent 的移动 RPC 是**不可靠**的，但客户端会把“重要的、尚未确认的旧移动”（例如带跳跃 / 加速度突变的那一步）一起打包发送（`ServerMoveOld` / 打包移动数据里的 OldMove），服务器用时间戳去重 `UE 源码` 。思路与 HOK 短冗余一致：**用不可靠通道 + 冗余换低延迟，而不是用可靠通道换队头阻塞。**第 12 章 E14 会量化可靠 RPC 的队头阻塞。

7.4 技能保序：与帧号无关的技能序号



图 7-3 技能序号的三个边界 HOK 文档 。技能序号单调递增，与逻辑帧号完全无关——它表达的是“玩家按下的顺序”，而不是“在哪一帧生效”。

三重保护

保护	规则	为什么
入队前去重	与上一条未确认技能同槽位 → 丢弃（视为重复点击）；**蓄力技豁免**（开始与结束本就同槽位）	紧张时狂点同一个按钮会产生大量无效指令
超时丢弃	入队超过 1 s 仍未确认 → 出队，计入“过期丢弃”	永远确认不了的技能会被每个包反复携带，白占带宽；且 1 s 前的技能再生效也没有意义
队满丢弃	未确认技能超过容量 → 丢最老的，计入“限长丢弃”	保证上行包不因冗余内容过多而超限

确认与出队



图 7-4 确认按序号顺序处理。确认的“信号”不是额外的 ACK 包，而是**在下行帧数据里看到自己的技能被执行**——帧同步里的“回执”天然存在。

打乱序号连续性的场景	问题	处理
断线追帧结束	追帧期间发出的技能可能在追帧完成后才到服务器，“迟到生效”	清空队列，序号加一个大偏移（如 +100），让迟到的旧指令被自然过滤
切换主控英雄	不同英雄的技能独立，序号语义断裂	清空队列，重新起算

两个处理是同一个思路：**与其精细地修补序号，不如用一个足够大的跳变把历史隔离掉**——TCP 用随机初始序号隔离旧连接的迟到报文，是同一类技巧。

面试注意

“上行丢包怎么办”是 HOK 帧同步的必问追问（下行有服务器冗余，上行没有）。回答要分命令类型：移动可以丢（下一条覆盖），关键命令短冗余，技能全量冗余 + 序号保序 + 确认。能说出“全量冗余反而比短冗余省带宽、代价是尾延迟”和“突发丢包会让紧挨着的副本一起丢”，就超出了大多数候选人。

7.5 坑位

坑	后果	修正
所有命令都做全量冗余	移动命令堆积，包体膨胀、尾延迟变大	按命令重要性分级
技能用可靠有序通道（TCP / 可靠 RPC）	一个包丢了，后面所有输入都被阻塞（队头阻塞）	不可靠 + 冗余 + 序号
冗余副本紧挨着发	突发丢包时一起丢（表 7-1）	重发间隔要跨越典型突发长度
没有超时	永远确认不了的技能永远占带宽	1 s 超时丢弃
去重时没豁免蓄力技	蓄力“松手”被当成重复点击丢掉	按技能类型豁免
追帧后沿用旧序号	迟到的旧技能突然生效	序号大偏移隔离历史

本章速记

- 上行包 = 命令 + **期待帧（ACK）** + **命令序号**；非战斗输入丢弃；帧末统一 flush；下行卡顿时上行主动退让。
- 短冗余：当前 + 前 2 条、约 33 ms；全量冗余：所有未确认技能、约 200 ms、1 s 超时、按序号保序。
- E7：突发丢包下短冗余 92.2%、全量 99.1%；全量平均只发 1.39 份，但 p99 772 ms。
- 序号三边界 nBeginSeq / nMaxSeq / nCurSkillSeq；确认 = 在下行帧里看到自己；= 出队、> 有空洞、< 重复；追帧后 +100 隔离。

▶ **自测 1：为什么普通移动命令不进冗余队列?**

▶ **自测 2：全量冗余的重发间隔设成 33 ms 会怎样?**

▶ **自测 3：服务器怎么知道技能是重复的?**

CHAPTER 08

HOK 保障与延展：网络质量、一致性校验、录像与 CS 模式

前面的机制解决“能通”；这一章解决“通得怎么样、不好时怎么知道哪里不好”，以及帧同步那份“输入序列”还能派什么用场。

本章目标

- 理解为什么“发包时刻 − 收包时刻”测出来的 ping 会偏大，并用实测看清偏差的量级与来源。
- 能讲出卡顿归因的四个维度、卡顿三态状态机，以及双链路（Wi-Fi + 蜂窝）的触发条件与代价控制。
- 说清一致性校验为什么是“帧 hash 上报比对”而不是“服务器养影子客户端”，以及录像“与实战走同一条路”的价值。
- 理解 CC / CS 两种模式、同一接口两种实现的设计，并与 LoL、UE 的回放方案对照。

30 秒面试版答案

ping 不能用主线程时刻算，因为包到达后会在网络库缓冲里等主线程来取，测出来的值混入了排队时间，并随帧率和卡顿放大；HOK 用网络线程打的到达时间戳（还分协议栈层和解密重组后两级）。卡顿从四个维度归因：帧推进抖动分 10 档统计、卡顿三态状态机（超过 300 ms 没新帧进入卡顿，最近收包小于 70 ms 进入恢复）、包到达到被处理的滞留时间（区分网络慢还是本机慢）、卡顿期间有效帧的到达序列（零散丢包还是整段断流）。Wi-Fi 变差时可以 Wi-Fi + 蜂窝双发，先到者生效。一致性靠各端周期性上报帧 hash 横向比对，成本极低、顺带反作弊。录像就是输入序列，回放与实战走完全相同的分发链路。另有 CS 模式：同一份 C++ 逻辑跑在服务器，客户端收结果流，上层代码通过统一接口无感切换。

8.1 ping：一个容易测错的指标

回顾第 5 章：Apollo 有自己的网络线程负责收包、解密、组包，但 C# 侧是**主线程同步轮询**取包。于是包到达后要在缓冲区里等 0~N 个渲染帧才被主线程取走。用主线程时刻计算 ping，测出来的是“网络往返 + 排队等主线程”的时间 HOK 文档 。实测（真实 RTT 均值约 55 ms，ping 在帧末发出） 实测 E8 ：

渲染帧率	网络线程时间戳 ms	主线程时刻 ms	平均多算 ms	p95 多算 ms
30 fps	54.8	67.5	12.7	16.5
30 fps + 3% 长帧（+100 ms）	55.1	70.9	15.8	26.9
60 fps	55.0	67.3	12.3	16.3
60 fps + 3% 长帧（+100 ms）	54.9	76.5	21.6	97.4
120 fps	54.7	59.9	5.2	8.0
120 fps + 3% 长帧（+100 ms）	55.1	72.8	17.7	93.8

表 8-1 E8：同一组网络往返，两种计时方式。平均偏差最多约一个渲染帧，并且与“RTT 除以帧长的余数”有关（ping 在帧末发出、在下一个帧首被取走）；一旦有长帧，p95 偏差可达 97 ms——玩家看到的就是“一卡 ping 就飙”。

所以 HOK 优先用 SDK 在网络线程打的到达时间戳，并提供两级：**底层协议栈时间戳**（更贴近网卡，精度最高）与**协议层时间戳**（解密与重组之后），可云控切换。注意这两个时间戳与主线程 `Time` 是**不同的时间基准**，不能混用。配套优化：有其它上行包时可以少发 ping（上行包本身带期待帧，已有 ACK 语义）；ping 包顺带回传丢帧数、抖动、补包统计；显示值做加权平滑。

对照 Unreal

UE 的 `APlayerState` 会复制一个压缩过的 ping 值（用 `GetPingInMilliseconds()` 读取），连接层还有 `UNetConnection::AvgLag` 等统计；UE 5.1 起新增了 `NMT_Ping` 控制消息与 `NetPing` 相关实现 UE 源码 。UE 的网络收包发生在 `UNetDriver::TickDispatch`（游戏线程 tick 开头），同样存在“包在套接字缓冲里等一帧”的问题，所以 UE 的 ping 也会受服务器 / 客户端帧率影响——这是面试里可以主动指出的细节。

8.2 卡顿归因：把“卡”拆开看

维度	做法 HOK 文档	回答的问题
① 帧推进抖动	每帧记录“实际间隔 − 66 ms”，按 10 档分桶（110 / 200 / … / 900 / 2000 ms+），随结算上报；保留具体卡顿帧号与时长（有上限）	是普遍轻微抖动，还是偶发严重卡顿?
② 卡顿状态机	三态：正常 / 卡顿 / 恢复中；进入卡顿时建档（前后收发包量、空洞长度、网络制式、持续时长），随 ping 回传	一次卡顿事件的完整画像
③ 处理延迟	单独统计“包到达 → 被主线程处理”的滞留时间	**慢的是网络，还是本机?**
④ 帧到达序列	卡顿期间有效帧的到达序列	零散丢包，还是整段断流?

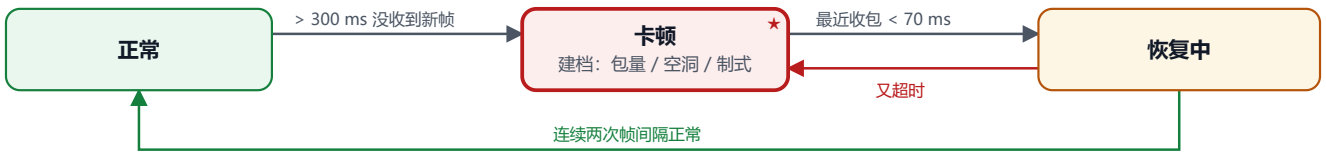


图 8-1 卡顿三态状态机 HOK 文档 （阈值可配）。“恢复中”这个中间态避免了状态在正常 / 卡顿之间抖动——典型的**迟滞（hysteresis）**设计。

8.3 双链路、加速器、后台收发、UDP 假死

机制	要点 HOK 文档
双链路（Wi-Fi + 蜂窝）	触发：主动补帧请求超阈值（下行在丢）/ 移动命令上行后迟迟收不到回包（上行有问题）/ ping 或原始延迟超阈值。开启后通过 UDP 控制字通知通道层，同一份数据两条链路都发，先到者生效。约束：30 s 冷却、低功耗模式降级、服务器可下发关闭；结算时上报单 / 双链路时长与切换次数评估收益
加速器协同	建连时把目标 IP / 端口注册给第三方加速 SDK（UDP 白名单）；局内持续上报实测延迟、均值 / 方差 / 丢包率
切后台	Unity 主循环降频或暂停时，启动独立网络线程继续收发，与主线程用锁互斥（抢不到锁就跳过本轮）；能在后台处理的（维持连接、心跳）立即处理，涉及表现与场景的攒到回前台再消费
UDP 假死	发包正常、一个包都收不到、连接状态却“正常”；判定：距上次收包超时且 ping 序号仍在推进 → 主动重建 UDP 连接并上报

双链路的本质是“空间冗余”

第 6、7 章的冗余是**时间冗余**（同一份数据在不同时刻再发一次），突发丢包会让挨得近的副本一起丢；双链路是**空间冗余**（同一份数据走两条独立的物理链路），两条链路同时突发的概率约为两者的乘积——这正是对付“Wi-Fi 信号衰减但还没断”这种突发丢包的正确工具。代价是流量和功耗，所以要有触发条件和冷却。

8.4 一致性校验：帧 hash 上报比对



图 8-2 一致性校验 HOK 文档 。常见误解是“服务器养一个不发言的影子客户端当标准答案”——HOK 不是这么做的。比对 hash 不需要服务器跑第二份逻辑，成本极低，并顺带完成反作弊：某个客户端的 hash 长期与多数不一致，很可能就是篡改者。

还有可选的**详细追踪**开关：开启后记录更细的演算轨迹，用于排查棘手的不同步（代价是性能与存储，仅在需要时开）。第 4 章的蝴蝶效应实验说明了为什么 hash 要“周期性”而不是“结束时”：1 ulp 的差异在第 1 帧就体现在 hash 上，而肉眼可见要到第 246 帧之后，越早发现，越容易定位到第一个分歧帧。

hash 能防什么、防不了什么

能防：改本地数值（攻击力、血量、CD）——改了就和其他九端不一致。防不了：**透视 / 全图**（客户端本来就有全量数据，只是读出来显示，不改状态）、**脚本 / 宏**（输入合法）。这两类要靠客户端反作弊、行为分析，或者 CS 模式（客户端拿不到完整逻辑）。“帧同步不能防作弊”和“帧同步完全能防作弊”都是错的。

8.5 录像：与实战走同一条路

帧同步的录像就是“把输入序列存下来”。HOK 更彻底：帧数据到达后第一时间写入录像缓冲，开局 / 开打 / 结算等关键协议也写入；回放时读录像、按协议号分派，遇到帧数据就喂给帧窗口——之后走**完全相同的分发链路**（交付逻辑层 → 授权推进 → 播放表现）HOK 文档。回放不是“另一套播放器”，而是“把实战的输入源换成文件”：录像能忠实复现实战中的 bug、不需要维护第二套逻辑、观战也复用同一机制（输入源换成实时转发）。由于逻辑在本地，观战可以自由切视角、看任意玩家，服务器不需要额外计算。

	HOK（帧同步录像）	Unreal（DemoNetDriver 回放）	英雄联盟（Chronobreak）
录什么	输入序列 + 关键协议	服务器发给客户端的复制数据流（以“观战连接”的视角录制）	服务器的全部输入、对局设置与配置（Server Network Recording）
体积	极小	较大（与复制带宽同量级）	小
能否跳转	只能从头 / 从快照重放	周期性检查点（checkpoint）支持快速跳转	确定性服务器重放到任意时刻，用于“回档”
前提	逻辑确定性 + 同版本	复制系统本身	把服务器改造成确定性（Riot 2016~2017 的工程）
来源	HOK 文档	UE 文档	Riot 技术博客

英雄联盟的例子很有启发：它是状态同步游戏，却为了“电竞比赛中出 bug 时回到某个时间点重开”（Project Chronobreak，2017 年首次在 LCS 赛场使用）把**服务器**做成了确定性的——录下服务器输入，重放时不限速，回到 40 分钟通常不到 3 分钟。而这项确定性工作最初是为了做可录制输入驱动的快速回归测试工具 Delta Checker Riot 技术博客 。**确定性不是帧同步的专利，而是一种可以单独购买的工程能力。**

8.6 另一种形态：CS 模式

开局协议 `SCID_MULTGAME_BEGINLOAD(1075)` 的 `bCsMode` 决定本局是 CC 还是 CS HOK 文档 ：

	CC（帧同步）	CS
战斗逻辑在哪	本地逻辑层全量运行	服务器逻辑层运行，本地只跑轻量部分
服务器做什么	帧号分配、汇总去重、定频广播、缓存	直接算完整逻辑，下发结果
下行内容	各玩家输入（1035）	演算结果的通知流（切成连续 PIECE 分片）
接收窗口 / 补发 / 追帧	需要	由服务器的可靠有序流替代（分片滑动窗口 + 区间 ACK + 缺口重传）
优势	带宽低、录像便宜、服务器成本低	天然防作弊（客户端拿不到完整逻辑）、客户端算力要求低

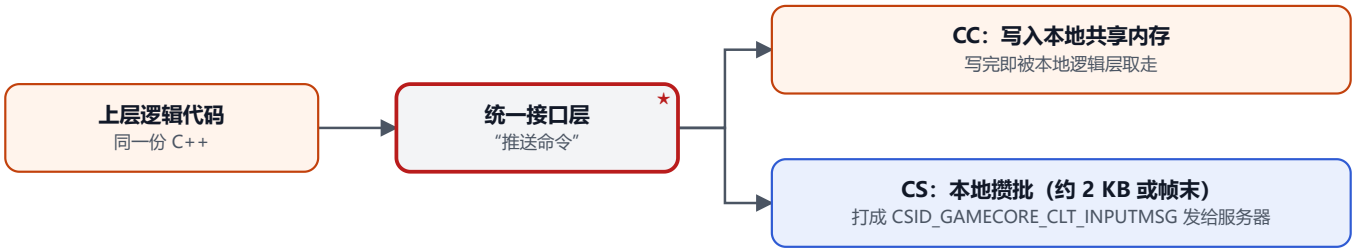


图 8-3 同一接口，两种传输实现：业务代码零改动 **HOK 文档** 。这是把“传输方式”抽象成可替换实现的收益，也是第 5 章“逻辑层不依赖引擎”埋下的伏笔。

“滑动窗口 + 按序消费 + 缺口重传”的思想在 CC 与 CS 下是一致的，只是承载内容不同：CC 下是输入，CS 下是结果。两种模式由服务器在开局时决定：常规 5v5 用 CC；对防作弊要求极高、低端机、部分娱乐玩法可用 CS。

CS 模式其实就是“服务器跑确定性逻辑的状态同步”

把 CS 模式放到第 1 章的谱系里看：逻辑在服务器、客户端收结果，这就是状态同步；但因为逻辑本身是确定性的，它同时保留了“录输入即可重放”的能力。HOK 的 CC / CS 双模式、LoL 的确定性服务器、守望先锋的确定性命令帧，说明的是同一件事：**帧同步与状态同步不是两个阵营，而是“权威放在哪”和“传什么”两个旋钮的不同组合。**第 14 章会把这两个旋钮展开。

面试注意

三个高频追问：“帧同步怎么检测不同步”（周期性 hash 上报比对，不是影子客户端）；“帧同步能防作弊吗”（能防改数值，防不了透视和脚本）；“帧同步录像为什么小”（只存输入，回放复用实战路径，代价是依赖确定性和版本）。答第三个时顺带对比 UE 的 DemoNetDriver 录的是复制数据流，体积大但不依赖确定性。

本章速记

- ping 用网络线程到达时间戳；主线程时刻会多算最多约一帧，长帧时 p95 偏差 97 ms（E8）。
- 卡顿四维：抖动 10 档分桶 / 三态状态机（300 ms 进卡顿、70 ms 进恢复）/ 处理延迟 / 到达序列；双链路 = 空间冗余。
- 一致性：帧 hash 上报横向比对，顺带反作弊；防得住改数值，防不住透视和脚本。
- 录像 = 输入序列，回放与实战同路径；CS 模式 = 同一份确定性逻辑跑在服务器，统一接口切换传输实现。

▶ **自测 1：为什么不让服务器跑一个影子客户端来做一致性校验?**

▶ **自测 2：卡顿状态机为什么需要“恢复中”这个中间态?**

▶ **自测 3：CS 模式下，第 6 章的接收窗口和补发还需要吗?**

CHAPTER 09

回滚网络：把 Lockstep 推到极限

格斗游戏把“等待”逼到了不可接受的地步：一帧 16.7 ms，目押窗口只有一两帧。回滚的答案是“不等了，先猜”——代价是保存每一帧的世界、并随时准备重写过去。

本章目标

- 写出回滚的核心循环：存快照 → 预测缺失输入 → 推进 → 真值到达 → 回到分歧帧 → 重算到当前帧。
- 用实测和公式说清“输入延迟 d 与回滚深度”的关系，理解为什么现代格斗游戏用“回滚 + 2~3 帧输入延迟”。
- 知道回滚的四个代价：画面跳变、音频无法回滚、更严苛的确定性、成倍的 CPU 预算——以及各自的缓解方式。
- 把回滚思想迁移到状态同步：守望先锋、Rocket League、UE 网络物理、CMC 都在用。

30 秒面试版答案

回滚（Rollback，GGPO 2006 年提出、2019 年以 MIT 协议开源）是 P2P 帧同步的一种：每帧保存游戏状态快照；缺远端输入时先预测（通常重复对方上一帧输入）立即推进，本地操作零等待；远端真实输入到达后若与预测不同，就加载分歧那一帧的快照，用正确输入重算到当前帧，画面瞬间修正。回滚深度约等于“单程延迟 + 抖动”折算的帧数减去输入延迟，所以现代格斗游戏会加 2~3 帧输入延迟，用几十毫秒换回滚深度和跳变的大幅下降。代价：每帧快照与多帧重算（模拟必须远快于一帧）、画面跳变、音效收不回来、对确定性要求更高；而且预测窗口用完之后仍要等待，木桶效应只是被推迟了。适合 2 人、状态小、延迟极敏感的格斗；不适合 10 人、状态大、移动网络的 MOBA。

9.1 一段简史

时间	事件	来源
1990s	严格 Lockstep：星际、魔兽 3、帝国时代	公开资料
2000s	延迟型 Lockstep：早期格斗游戏联机，用固定输入延迟吸收抖动	公开资料
2006	Tony Cannon（EVO 联合创始人）不满于当年在线格斗的延迟体验，开发 GGPO，年底发布首版	GGPO 资料
2011–2012	商用授权版本被《街头霸王 III 3rd Strike Online Edition》《Skullgirls》等采用	GGPO 资料
2019-10	GGPO 以 MIT 协议开源（github.com/pond3r/ggpo）	GGPO 资料
2020 至今	回滚成为主流格斗游戏标配；同期 MOBA 收敛到“定频广播不等人”——两条路线并行，各自针对不同约束	公开资料 · HOK 文档

9.2 核心循环



图 9-1 一次回滚。只要预测对了（玩家操作大多是连续的），就什么都不会发生；预测错了，代价是一次“重写过去”和一次画面跳变。

回滚核心循环（与 demos/netsync_demo.cpp 中 run_p2p 的逻辑一致，删去了统计代码）

```
// 每个渲染帧调用一次；frame = 本地已推进到的帧号，d = 输入延迟（帧）
void RollbackTick() {
    // ① 收网络包：远端第 n 帧的真实输入到达
    for (auto& [player, n, input] : net.Receive()) {
        known[player][n] = input;
        if (n < frame && used[player][n] ≠ input)          // 当时用的是预测值，而且猜错了
            firstMismatch = std::min(firstMismatch, n);
    }
    // ② 回滚：回到最早的分歧帧，用（新的）最佳输入重算到当前帧
    if (firstMismatch < frame) {
        LoadState(snapshot[firstMismatch]);                // 快照保存的是“执行第 n 帧之前”的状
        for (int n = firstMismatch; n < frame; ++n) {
            snapshot[n] = SaveState();
            Simulate(InputsFor(n));                        // 已知就用真值，未知仍用预测值
        }
        firstMismatch = INT_MAX;
    }
    // ③ 推进：本地输入排到 d 帧之后；预测窗口还没用完就继续往前走
    known[me][frame + d] = SampleLocalInput();
    if (frame - MinConfirmedFrameOfOthers() > kMaxPrediction)   // 窗口用完：只能等（木桶效应回来
        return;
    snapshot[frame] = SaveState();
    Simulate(InputsFor(frame));                            // InputsFor：已知用真值，未知用“上一
    ++frame;
    net.SendRecentInputs(me, /*带最近若干帧，抗丢包*/ 24);
}
```

GGPO 的 API 长什么样

游戏只需要实现几个回调：`save_game_state` / `load_game_state`（快照）、`advance_frame`（回滚重算时由 GGPO 驱动推进一帧）、`free_buffer`、`on_event`；主循环里调 `ggpo_add_local_input` 提交本地输入、`ggpo_synchronize_input` 取本帧所有玩家输入（含预测值）、`ggpo_advance_frame` 通知推进；`ggpo_set_frame_delay` 设置输入延迟 GGPO 源码。注意 GGPO 完全不关心你的游戏逻辑，它只要求“给定输入就能确定性地推进一帧、并能存取完整状态”——这就是回滚的全部前提。

还有一个容易忽略的机制：**时间同步**。如果一方的时钟持续领先，它会一直在“预测对方”，对方却一直在“等它的输入”，回滚全落在领先方。GGPO 会估算双方的帧领先量，让领先的一方主动等待几帧，把预测负担平摊到两边 GGPO 源码。

9.3 实测：输入延迟 d 换回滚深度 实测 E9

模型：2 人、60 帧/秒（16.7 ms）、每帧有 20% 概率改变输入（相当于每秒 12 次操作变化），丢包 1%，单程 = RTT/2、指数抖动均值 1 + 0.04·RTT ms；回滚最多预测 8 帧。对每个 RTT 扫描输入延迟 d = 0~4，并对照“不回滚、只靠输入延迟”的纯延迟型。

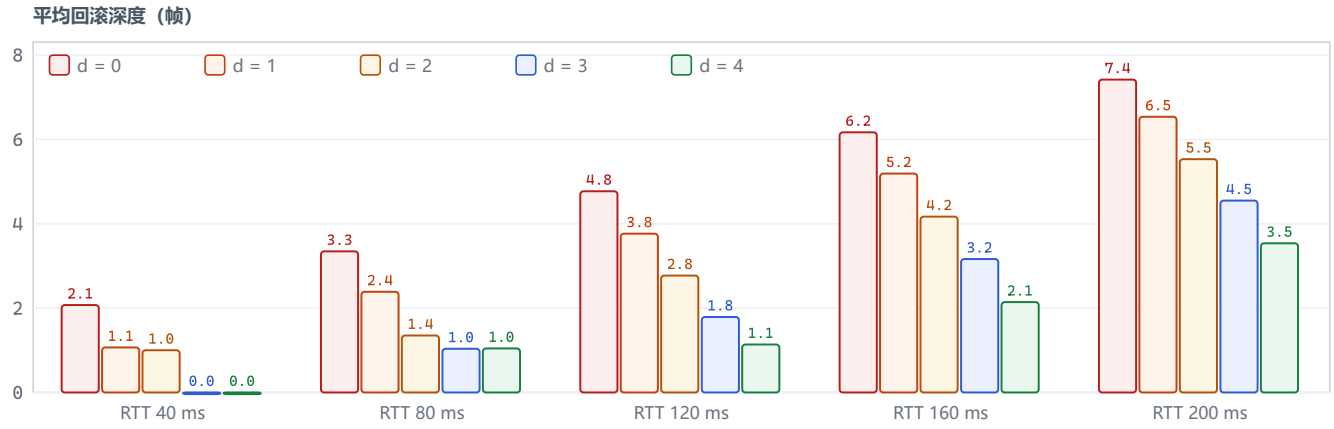


图 9-2 平均回滚深度：每加 1 帧输入延迟，深度大约少 1 帧；当 d 覆盖了“单程延迟 + 抖动”，回滚就几乎消失（RTT 40 ms、d = 3 时为 0）。

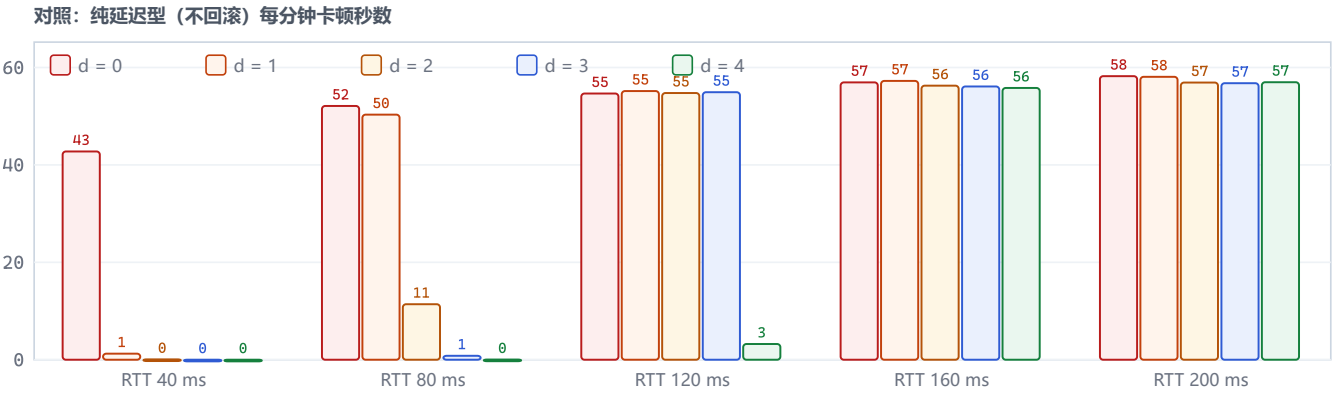


图 9-3 同样的网络、不回滚：只要 d 没覆盖单程延迟，几乎每一帧都在等对方输入，一分钟里有几十秒是卡住的（游戏实际以低于 60 fps 的速度运行）。回滚版本在同样条件下的卡顿几乎为 0（RTT 200 ms、d = 0 时也只有 243 ms/分，因为深度偶尔超过 8 帧窗口）。

回滚深度可以这样估算：远端在第 $n-d$ 帧采样的输入要排到第 n 帧执行，送到本地需要单程延迟 $\frac{\mathrm{RTT}}{2}$ 加抖动 J；本地执行第 n 帧时它还没到，就只能预测，等它到了再回滚：

推导 · 回滚深度的估算

① 输入到达本地的时刻 $\approx t_{n-d} + \frac{\mathrm{RTT}}{2} + J$ — 远端在执行第 $n-d$ 帧时采样，并假设两端帧时钟大致对齐

② 本地执行第 n 帧的时刻 $= t_n = t_{n-d} + dT$ — 每帧 $T = 16.7$ ms

③ 需要预测的帧数 $\approx \dfrac{\mathrm{RTT}/2 + J}{T} - d$ — 两者之差除以帧长

④ $D_{\text{rollback}} \approx \dfrac{\mathrm{RTT}/2 + J}{T} - d + 1$ — 再加约 1 帧的离散化误差（输入在帧首采样、在帧中到达）；结果 ≤ 0 时几乎不回滚

RTT ms	输入延迟 d	d 折合 ms	回滚 次/秒	平均深度（实测）	估算	p95 深度	重算 帧/秒
80	0	0	12.2	3.34	3.7	4	40.6
80	1	17	12.1	2.39	2.7	3	28.8
80	2	33	11.8	1.35	1.7	2	15.9
80	3	50	4.1	1.04	0.7	1	4.3
80	4	67	0.2	1.05	0.0	1	0.2
160	0	0	11.7	6.17	6.2	7	72.4
160	1	17	11.8	5.19	5.2	6	61.1
160	2	33	11.5	4.17	4.2	5	47.9
160	3	50	11.4	3.16	3.2	4	36.1
160	4	67	11.3	2.14	2.2	3	24.3

表 9-1 E9 节选（RTT 80 / 160 ms）。回滚次数基本等于“对方输入发生变化的频率”（约 12 次/秒），输入延迟减少的是**每次回滚的深度**，直到 d 足够大时回滚才消失。

为什么不把 d 设得很大、干脆不回滚？因为 d 是**所有人的操作都要等的**：d = 4 帧就是 67 ms 的固定延迟，对格斗游戏的“目押”是不可接受的。现代格斗游戏的做法是 **2~3 帧基础延迟 + 回滚**：几十毫秒的延迟在容忍范围内，却把大多数网络下的回滚深度压到 1~3 帧，跳变几乎看不出来 HOK 文档 公开资料 。不少游戏还允许玩家在设置里自己调这个值（GGPO 时代起就有）。

9.4 回滚的四个代价

代价	表现	缓解
画面跳变	角色瞬移、已经播出的攻击被“收回”、特效突然消失	限制回滚深度；对**表现层**做位置平滑（逻辑仍然精确）；只在真正分歧时回滚；动画从“重算后的帧”继续播放而不是从头
音频无法回滚	该响的没响、不该响的响了、同一音效播两次	音频不参与回滚：按 (帧号, 事件 id) 去重，重算时已播过的不再播；预测时播出的错误音效通常只能接受
确定性更严苛	同一段帧被反复重算，任何不确定都会暴露成“回滚后对不上”	定点数、确定性随机、状态完整序列化（漏存一个变量 = 重算结果错）
CPU 预算成倍	深度 D 时一个渲染帧里要模拟 D + 1 帧	见下面的预算推导；状态要小、模拟要快

推导 • 回滚的 CPU 预算

① $T_{\text{frame}} \geq (D_{\max}+1)\,t_{\text{sim}} + D_{\max}\,t_{\text{load/save}} + t_{\text{render}}$ 最坏情况：一帧内回滚 $D_{\max}$ 帧再推进 1 帧

② $t_{\text{sim}} \leq \dfrac{T_{\text{frame}} - t_{\text{render}}}{D_{\max}+1}$ 忽略快照开销

③ $t_{\text{sim}} \leq \dfrac{16.7-8}{8+1} \approx 0.97\ \text{ms}$ 60 fps、渲染占 8 ms、最多回滚 8 帧

所以回滚要求“逻辑模拟比一帧快一个数量级”，格斗游戏（2 个角色 + 判定框）做得到，MOBA（几十单位 + 上百 Buff）做不到——这是 HOK 不选回滚的硬约束之一

实测里 RTT 160 ms、d = 0 时每秒要额外重算 72 帧，相当于在 60 帧/秒之外又多跑了 72 帧的逻辑；d = 2 时降到 48 帧。输入延迟不仅减少跳变，也直接降低 CPU 负担。

9.5 回滚在状态同步里的化身

“保存历史 → 回到分歧点 → 用正确数据重算”并不是帧同步独有的。状态同步里只要做了**客户端预测**，就一定有它：

系统	回滚的范围	“真值”从哪来	来源
GGPO / 格斗游戏	整个游戏状态	远端玩家的真实输入	GGPO
Unreal CMC	只回滚**本地角色的移动**	服务器发回的权威位置（ClientAdjustPosition），然后重放未确认的 SavedMoves	UE 源码（第 13 章）
守望先锋	本地玩家相关的、由输入驱动的 System 状态	服务器对这些命令帧的权威结果	GDC 2017
Rocket League	车与球的物理状态	服务器物理状态；客户端回滚后重新模拟	GDC 2018
UE 网络物理（Resimulation）	物理对象	服务器状态与本地历史不一致超过阈值时触发重模拟；`NetworkPhysicsComponent` 负责输入 / 状态历史	UE 文档（5.4 起，Resimulation 为实验性）

区别在于“真值”：帧同步回滚等的是**别人的输入**，状态同步预测等的是**服务器的结果**。但数据结构完全一样：一个按帧（或时间戳）索引的历史环形缓冲 + 一个“从某帧开始重算”的函数。理解了这一点，第 10、13 章的“预测与和解”就只是换了个名字。

面试注意

被问“王者荣耀为什么不用回滚”时，从回滚的四个前提逐条对照：人数（2 vs 10）、状态规模（快照成本）、每帧重算成本（CPU 预算推导）、网络环境（移动网络抖动会让回滚持续触发、画面持续抽动）。再补一句“回滚并没有消灭木桶效应，只是推迟到超过预测窗口才暴露”，这道题就答满了。

9.6 坑位

坑	后果	修正
快照漏存某个状态（如随机数生成器、技能冷却的小数部分）	回滚后与对方对不上 → desync	状态集中管理（ECS / 单块内存），整体拷贝；回归测试：每帧都强制回滚一次再比对 hash（同步测试模式）
表现层在预测帧里触发了不可撤销的事件（成就、结算、网络请求）	“幽灵事件”	不可撤销的副作用只在**确认帧**（所有人的输入都已知）上触发
输入延迟 d = 0	回滚频繁且深、CPU 吃紧	2~3 帧基础延迟，或按网络动态调整
不做时间同步	一方长期领先，回滚全落在它身上	估算帧领先量，领先方等待
预测策略过于“聪明”	猜错时跳变更大	最常用的仍是“重复上一个输入”，简单且错误代价可控

本章速记

- 回滚 = 每帧快照 + 预测缺失输入 + 真值不符时加载分歧帧快照重算到当前；GGPO 2006 年提出、2019 年 MIT 开源。
- 深度 $\approx (\mathrm{RTT}/2 + J)/T - d + 1$：每加 1 帧输入延迟深度少约 1 帧；回滚次数 ≈ 对方输入变化频率。
- 四个代价：跳变、音频、确定性 + 完整序列化、CPU（$t_{sim} \le (T - t_{render})/(D+1)$，60 fps 回滚 8 帧时约 1 ms）。
- 预测窗口用完仍要等；同样的“历史 + 重算”结构就是 CMC / 守望先锋 / Rocket League / UE 网络物理的预测和解。

▶ **自测 1：回滚的“同步测试模式”（sync test）是什么？为什么有用？**

▶ **自测 2：RTT 120 ms、60 帧/秒、d = 2，估算平均回滚深度。**

▶ **自测 3：为什么说 CMC 的“纠正后重放 SavedMoves”就是回滚?**

PART III

状态同步篇：以 Unreal 为主线

先讲与引擎无关的三件套（插值、预测与和解、延迟补偿），再进入 Unreal：网络架构与复制管线、属性复制与 RPC、移动与技能预测，以及 Iris、Mover、网络物理这些新系统。

CHAPTER 10

状态同步通论：插值、预测与和解、延迟补偿

状态同步的客户端同时活在三个时间里：看别人是在看过去，看自己是在看未来，开枪打别人时，服务器要替你回到过去。

本章目标

- 画出“客户端的三个时间”：插值的他人（过去）、预测的自己（未来）、服务器的现在。
- 用实测选插值延迟：快照间隔、抖动、丢包如何决定“该往回看多少毫秒”。
- 写出预测 + 和解（reconciliation）的核心循环，用实测说明“直接覆盖”为什么会橡皮筋。
- 说清延迟补偿的回溯量为什么是 RTT + 插值延迟、为什么要设上限、“隔墙杀”与“先开枪优势”从哪来。

30 秒面试版答案

状态同步里服务器权威，客户端用三件套弥补延迟：① **插值**：别人的角色按“服务器时间 − 插值延迟（常见 100 ms 左右，约两个快照间隔）”渲染，在两份快照之间插值，丢一份快照也不卡，代价是看到的别人在过去；② **预测 + 和解**：自己的输入立即在本地模拟（带序号保存），服务器回传“处理到第几号输入 + 权威状态”后，客户端回到这个状态、重放之后未确认的输入，预测对了就无感，错了只纠正差值，不会被拉回一个 RTT 之前；③ **延迟补偿**：服务器收到开火命令时，把其他玩家的碰撞体回溯到射手当时看到的时刻（约 RTT + 插值延迟之前）做命中判定，再恢复；回溯要设上限，否则高延迟玩家会“隔墙杀”低延迟玩家。带宽靠增量压缩、量化、兴趣管理和优先级调度控制。

10.1 客户端的三个时间



图 10-1 状态同步客户端的三个时间。守望先锋的描述是：客户端“领先服务器半个 RTT 加一个缓冲帧” GDC 2017 。HOK 的帧同步里没有这种分裂：所有单位都在同一个时间（收到第 N 帧时），代价是自己的操作也要等一个来回。

10.2 插值：看别人是在看过去

服务器以固定频率（例如 20 Hz，每 50 ms 一份）下发快照。客户端不直接画最新快照，而是把它们放进缓冲区，渲染“估计的服务器时间 − 插值延迟 D”这一时刻的状态：找到前后两份快照线性插值；如果后一份还没到，只能外推（或停住）。D 越大越不容易缺快照，但画面越旧。Source 引擎的经典默认值是 100 ms 插值延迟（`cl_interp` 0.1，即 20 Hz 更新时两个快照间隔），这样丢一份快照仍有数据可插 Valve 开发者文档 。

实测：插值延迟选多少 实测 E10

模型：服务器 20 Hz 快照，下行 40 ms + 指数抖动（均值 2 / 15 / 40 ms 三档），丢包 2%；目标做三角波横移（每 1.4 s 折返一次，带急转），客户端 60 fps 渲染。

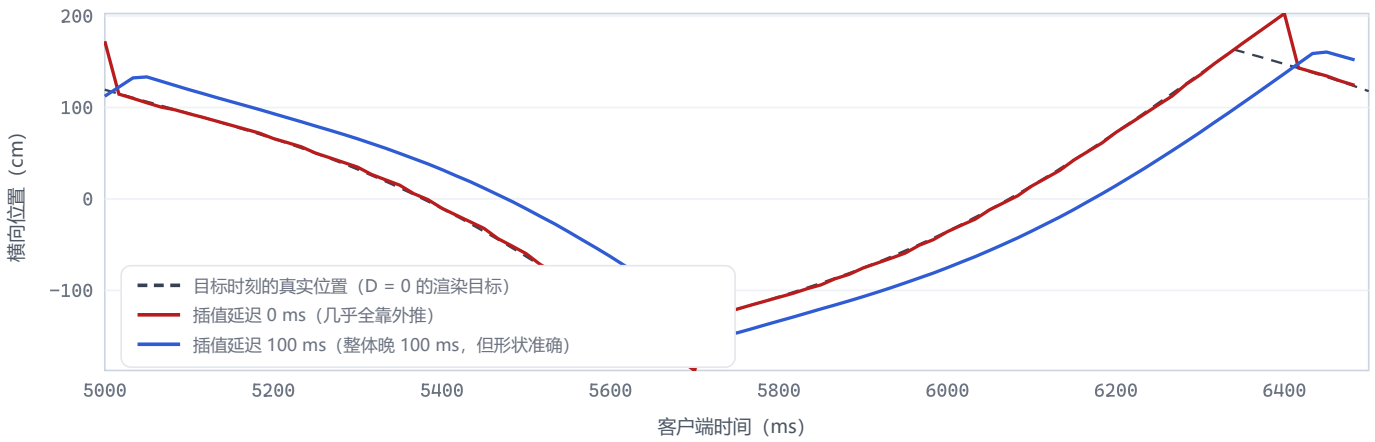


图 10-2 抖动均值 15 ms 时的一段轨迹。D = 0 时最新快照总是“还没到”，只能按速度外推，在折返处冲过头再被拉回（红线的尖角）；D = 100 ms 时画面整体晚 100 ms，但曲线形状与真实运动一致。

抖动均值 ms	插值延迟 D ms	需要外推的帧	平均误差 cm	p99 误差 cm	画面落后 ms（40 + J + D
2	0	100.0%	2.5	40.1	42
2	50	1.7%	0.2	1.0	92
2	100	0.0%	0.2	0.9	142
2	150	0.0%	0.2	0.9	192
2	200	0.0%	0.2	0.9	242
15	0	100.0%	4.2	57.6	55
15	50	26.5%	0.6	9.4	105
15	100	0.9%	0.2	0.9	155
15	150	0.0%	0.2	0.8	205
15	200	0.0%	0.2	0.8	255
40	0	100.0%	6.9	80.0	80
40	50	55.7%	1.7	37.9	130
40	100	10.2%	0.4	9.4	180
40	150	0.5%	0.3	1.4	230
40	200	0.0%	0.2	1.2	280

表 10-1 E10。误差是“画出来的位置”与“该渲染时刻的真实位置”之差。抖动越大，需要的 D 越大：抖动 40 ms 时 D = 100 仍有 10.2% 的帧要外推，D = 150 才降到 0.5%。

推导・插值延迟的经验下界

① $D \geq \Delta_{\text{snap}} \cdot (1 + n_{\text{loss}}) + J_{p99}$ 快照间隔 × (1 + 允许连丢的份数) + 抖动的高分位

② $D \geq 50 \times (1 + 1) + J_{p99}$ 20 Hz 快照、容忍连丢 1 份

③ $D \approx 100\ \text{ms} + J_{p99}$ 这就是 Source 默认 100 ms 的来历；移动网络抖动大，D 要更大或做自适应

自适应插值延迟：持续统计快照到达间隔的分布，网络好时缩小 D（画面更新），变差时平滑放大（拉伸时间轴，不跳变）

10.3 外推：Dead Reckoning

外推（按最后已知的速度 / 加速度往前推）能让画面“不落后”，但运动一旦改变方向就会冲过头，再被拉回（图 2-2 的外推线、图 10-2 的红线）。它适合**运动可预测**的对象：载具、投射物、惯性大的物理物体；不适合会急转的人形角色。实践中常见的做法是：**以插值为主，缺快照时短时间外推（设上限，如 250 ms），超时就停住**；外推出错后不直接跳到正确位置，而是在几帧内把“显示位置”平滑收敛到“逻辑位置”（Unreal 的 SimulatedProxy 网格平滑就是这一类，第 13 章）。

10.4 预测与和解：看自己是在看未来

如果自己的角色也等服务器回来才动，每次按键都要多一个 RTT 的延迟。客户端预测（client-side prediction）让本地输入立即生效；服务器仍然权威，它回传“我处理到你第几号输入了，此时你在这里”。客户端拿到后有两种做法：

预测 + 和解（通用写法，与 demos/netsync_demo.cpp E11 的方案 2 一致）

```
struct PendingInput { uint32_t seq; Input in; float dt; };
std::deque<PendingInput> pending;            // 已发出、服务器尚未确认的输入
State predicted;

void OnLocalInput(Input in, float dt) {
    uint32_t seq = ++nextSeq;
    predicted = Simulate(predicted, in, dt);  // ① 立即在本地模拟：零延迟手感
    pending.push_back({seq, in, dt});
    SendToServer(seq, in, dt /*, 以及最近几条未确认输入作冗余 */);
}

void OnServerState(uint32_t ackSeq, const State& authoritative) {
    while (!pending.empty() && pending.front().seq ≤ ackSeq)
        pending.pop_front();                  // ② 丢掉服务器已经处理过的输入
    State s = authoritative;                  // ③ 回到服务器的权威状态（“加载快照”）
    for (auto& p : pending) s = Simulate(s, p.in, p.dt);   // ④ 重放尚未确认的输入（“重算”）
    ApplyCorrection(predicted, s);            // ⑤ 预测对了：s == predicted，什么都不发生；错了：
    predicted = s;
}
// 错误做法（“直接覆盖”）：predicted = authoritative —— 丢掉了 pending，
// 角色每次都被拉回到“一个 RTT 之前”的位置，然后再往前走：经典的橡皮筋。
```

实测：三种做法 实测 E11

模型：60 Hz 输入，移动速度 600 cm/s，按“右 1.2 s → 停 0.4 s → 左 1.2 s → 停 0.4 s”循环；RTT ≈ 100 ms，上行丢包 5%（每包带最近 3 个移动作冗余），服务器 30 Hz 回传确认；**服务器在 3.0~3.5 s 判定玩家被定身**（客户端不知道）。纵轴是“画面上的位置 − 服务器对同一组输入的权威结果”。

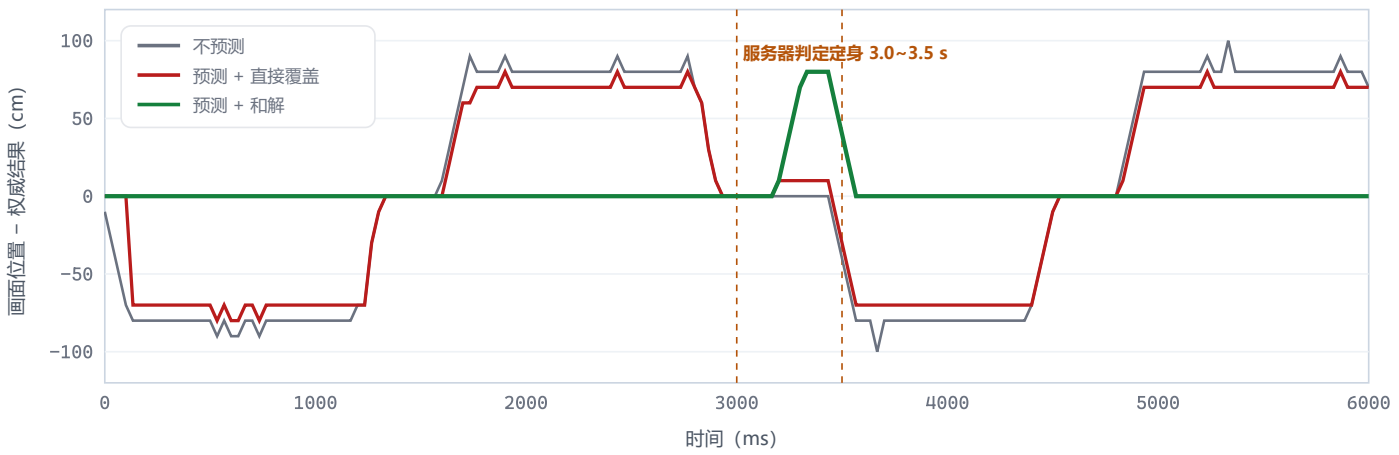


图 10-3 三种做法的偏差曲线。和解（绿线）在移动过程中始终为 0——预测完全正确；只有定身期间本地预测“多走了”，偏差上升到一个 RTT 左右的位移后被服务器纠正回来。

做法	输入到画面延迟 ms	纠正次数（>1 cm）	平均纠正 cm	最大纠正 cm
不预测（等服务器）	122	0	0.0	0.0
预测 + 直接覆盖	17	70	16.1	70.0
预测 + 和解（重放未确认移动）	17	8	18.8	20.0

表 10-2 E11。“直接覆盖”纠正了 70 次（移动中几乎每次确认都要拉回），和解只纠正了 8 次——全部发生在服务器真的与预测不同的定身期间。

和解的关键是**输入带序号、服务器回传“处理到第几号”**。这正是 HOK 上行包里“命令序号”、Unreal CMC 里“时间戳”扮演的角色。被定身这种“预测必错”的情况无法避免，能做的是让纠正**平滑**（把视觉位置在几帧内插值到正确位置），以及在玩家能感知的地方给出合理的表现（被控时的特效、音效）。

预测什么、不预测什么

只预测**由本地输入直接决定、且失败代价可接受**的东西：自己的移动、开火的枪口特效与弹道、技能的起手动画。不预测或谨慎预测：伤害数值、死亡、拾取物归属（错了要撤销会很尴尬）、与别人强交互的结果（推人、抓取）。GAS 把这一点做成了框架：能被预测的东西都挂在一个 Prediction Key 上，服务器拒绝时整体撤销（第 13 章）。

10.5 延迟补偿：开枪时服务器替你回到过去

射手看到的敌人是插值后的“过去的敌人”（图 10-1 左侧）。如果服务器用自己“现在”的敌人位置判定命中，射手只能“提前量瞄准”，延迟越高越打不中。延迟补偿（lag compensation）的做法是：服务器为每个玩家保存最近一段时间（例如 1 s）的碰撞体历史；收到开火命令时，把**其他玩家**回溯到“射手开火时屏幕上看到的那一刻”，判定命中，再恢复。

推导 · 应该回溯多少

① $t_{\text{shooter view}} = t_{\text{fire}} - \left(\frac{\text{RTT}}{2}_{\downarrow} + D\right)$ 射手开火时看到的是“下行延迟 + 插值延迟”之前的敌人

② $t_{\text{server recv}} = t_{\text{fire}} + \frac{\text{RTT}}{2}_{\uparrow}$ 命令到达服务器

③ $\Delta_{\text{rewind}} = t_{\text{server recv}} - t_{\text{shooter view}} = \text{RTT} + D$ 服务器要回溯的量 = 往返时间 + 插值延迟（Source 引擎的说法：命令执行时间 − 延迟 − 插值时间）

实际实现常让客户端在命令里带上“我渲染的是哪个时刻”，服务器据此回溯并做合理性校验，而不是只靠估计的 RTT

实测（目标 600 cm/s 横移、随机折返，命中半径 34 cm，射手瞄得很准、只有 6 cm 手抖，插值延迟 100 ms）实测 E12：

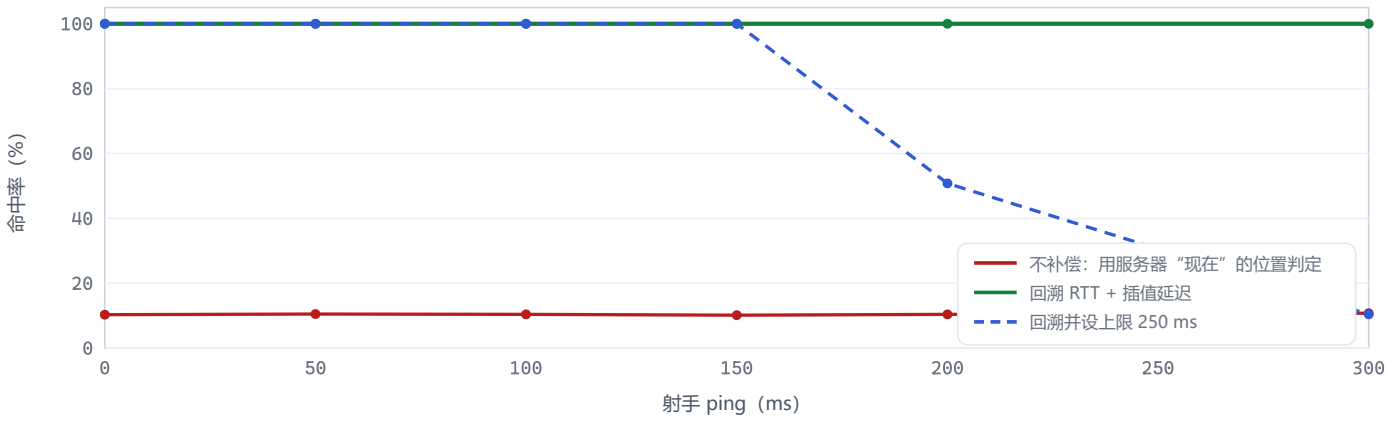


图 10-4 延迟补偿的效果。不补偿时，即使 ping 为 0，插值延迟本身就让命中率只剩约 10%；回溯后（模型里 RTT 估计准确）命中率回到 100%。设了 250 ms 上限后，ping 超过约 150 ms 的射手会逐渐失去补偿。

回溯的代价落在**被打的人**身上：低 ping 的受害者已经躲到墙后，高 ping 射手的命令到达后服务器回溯到过去，判定命中——受害者体验到“我明明躲进去了还是死了”（隔墙杀）。所以要设回溯上限（Source 的 `sv_maxunlag` 默认 1 秒 Valve 开发者文档；很多竞技游戏设得更低），把“高 ping 不公

平”限制在一个范围内。另一个副作用是**先手优势（peeker's advantage）**：从掩体后探出的人先看到对方（对方看到的是插值后的过去），提高 tick 率、降低插值延迟可以缓解——这是 Valorant 选择 128 tick 服务器的动机之一 Riot 技术博客 。

10.6 带宽：增量、量化、兴趣、优先级

手段	做法	例子
增量压缩	只发和“对方已确认的基线”不同的字段；基线没确认就继续以旧基线为参照	Quake 3 的快照增量；UE 属性复制按连接保存影子状态，只发变化的属性
量化	位置到厘米 / 0.1 厘米、角度压到 8 / 16 位、法线压缩	UE 的 `FVector_NetQuantize` / `NetQuantize10` / `NetQuantize100` / `NetQuantizeNormal`
兴趣管理（AOI）	只发对这个客户端“相关”的对象：距离、视野、队伍、所有权	UE 的相关性（`IsNetRelevantFor`、`NetCullDistanceSquared`）、Replication Graph 的空间网格
优先级 + 频率	重要的发得勤，不重要的发得少；带宽满了按优先级排队	UE 的 `NetUpdateFrequency` + `NetPriority` × 等待时间（第 11 章 E13）
休眠	长期不变的对象停止检查，变化时再唤醒	UE 的 Net Dormancy

第 1 章的 E1 已经量化过：增量 + AOI 能把 100 个单位的下行从 48.5 KB/s 压到 2.9 KB/s。

> **面试注意**
> “状态同步怎么解决延迟”的标准答案就是三件套：插值（别人）、预测与和解（自己）、延迟补偿（命中）。高频追问：① 插值延迟设多少、为什么（快照间隔 × 2 + 抖动）；② 预测错了怎么办（回到权威状态 + 重放未确认输入，不是直接覆盖）；③ 延迟补偿回溯多少、有什么副作用（RTT + 插值延迟；隔墙杀、先手优势；设上限）。

10.7 坑位

坑	后果	修正
和解时直接用服务器状态覆盖	橡皮筋（表 10-2）	回到权威状态后重放未确认输入
插值延迟固定且偏小	移动网络下频繁外推、跳动	按抖动统计自适应，平滑调整
对别人的角色也做预测	别人的输入你不知道，预测必然频繁出错	别人一律插值（外推只用于可预测对象）
延迟补偿无上限	高 ping 玩家隔墙杀	设回溯上限，并校验客户端声称的时刻
只回溯位置不回溯姿态	动画不同步导致头部判定框错位	回溯完整的碰撞体姿态（或保存骨骼命中盒历史）
预测了不可撤销的东西	“我捡到了”然后又没了	只预测可撤销的表现，关键结果等服务器

本章速记

- 三个时间：别人在 t − (RTT/2 + D)，服务器在 t，自己在 ≈ t + RTT/2。
- 插值：$D \gtrsim \Delta_{snap}(1 + n_{loss}) + J_{p99}$，20 Hz 时约 100 ms；E10：抖动 40 ms 时 D = 100 仍有 10.2% 帧外推。
- 预测 + 和解：输入带序号，服务器回 ack + 权威状态，客户端回到权威状态并重放未确认输入；E11 纠正 70 → 8 次。
- 延迟补偿：回溯 RTT + D；代价是隔墙杀与先手优势，要设上限。带宽：增量 + 量化 + AOI + 优先级 + 休眠。

▶ **自测 1：为什么不对其他玩家做预测，而是插值?**

▶ **自测 2：一个 FPS 服务器 tick 30 Hz、快照 30 Hz，插值延迟大概设多少?**

▶ **自测 3：和解时如果服务器的权威位置和预测只差 2 cm，要不要纠正?**

CHAPTER 11

Unreal 网络架构与复制管线

Unreal 的复制系统回答的是一个调度问题：每个服务器 tick、每条连接、成百上千个 Actor，带宽有限，先发谁、发什么、发给谁。

本章目标

- 画出 NetDriver / NetConnection / Channel / PackageMap 的结构，说清 Bunch、可靠性在哪一层实现。
- 熟练回答 NetMode、Role / RemoteRole、所有权（Owner → PlayerController → Connection）三组概念，以及 Gameplay 框架各类存在于哪一端。
- 按顺序讲出 `ServerReplicateActors` 的流程：频率门控 → 休眠 → 相关性 → 优先级排序 → 饱和截断。
- 用实测理解“优先级 × 等待时间”为什么能避免饿死；知道 Replication Graph 和 Iris 分别解决了什么。

30 秒面试版答案

Unreal 默认是服务器权威的状态同步。服务器的 NetDriver 为每个客户端维护一个 NetConnection，连接里有控制通道和每个复制 Actor 一条 ActorChannel；数据以 Bunch 为单位打进 UDP 包，可靠 Bunch 由通道负责重传和排序，对象引用靠 PackageMap 把 UObject 映射成 NetGUID。每个 Actor 在各端有 Role：服务器上是 Authority，拥有它的客户端上可以是 AutonomousProxy（本地控制、做预测），其他客户端上是 SimulatedProxy（插值播放）。所有权沿 Owner 链找到 PlayerController，进而确定连接——Server RPC 只能从拥有者连接发起。每个网络 tick，服务器对需要复制的 Actor 做：按 NetUpdateFrequency 门控、跳过休眠、按连接判相关性、按“NetPriority × 距上次复制时间”（再按距离和视野加权）排序、依次复制直到带宽饱和，没发出去的下一 tick 优先级更高。大规模用 Replication Graph 把相关性做成空间网格等节点，UE5 的新复制系统 Iris 把过滤、优先级改成显式 API，并与游戏线程数据解耦以并行化。

11.1 网络栈的结构

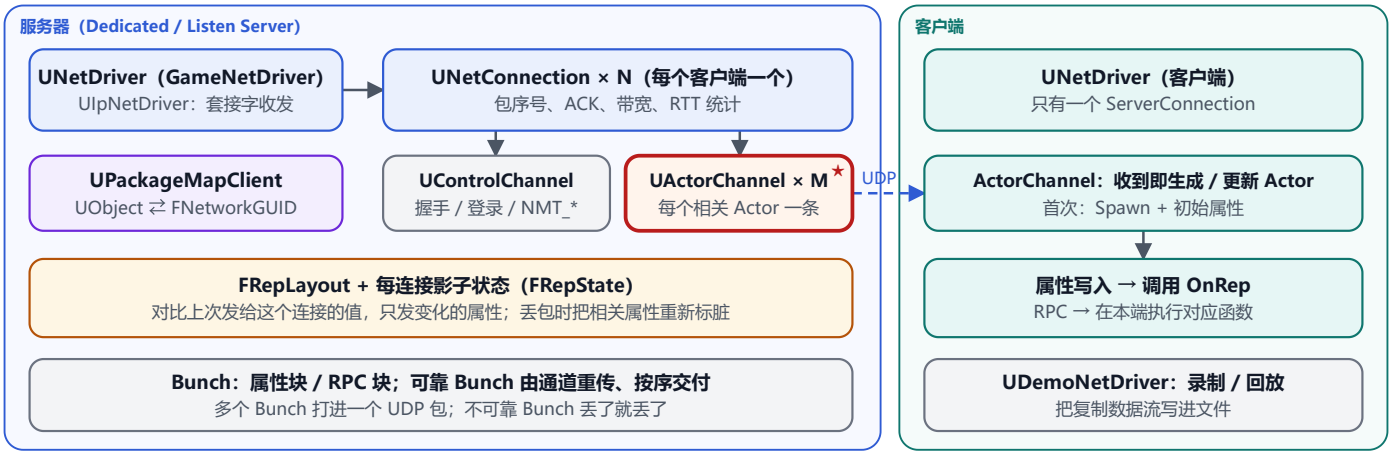


图 11-1 Unreal 网络栈 **UE 源码** 。可靠性不是“整个连接要么可靠要么不可靠”，而是 **Bunch 级**的：同一个包里可以既有可靠的 RPC，也有不可靠的属性更新。

11.2 NetMode、Role 与所有权

NetMode	含义
`NM_Standalone`	单机，不联网（同时是“服务器”和“客户端”）
`NM_DedicatedServer`	专用服务器：无本地玩家、不渲染
`NM_ListenServer`	主机：既是服务器又有一个本地玩家（主机玩家没有延迟，存在主机优势）
`NM_Client`	连接到服务器的客户端

Role（`GetLocalRole()`）	出现在哪	该做什么
`ROLE_Authority`	服务器上的所有 Actor；客户端本地生成、不复制的 Actor 也是	跑权威逻辑、改复制属性、发 Client / Multicast RPC
`ROLE_AutonomousProxy`	拥有它的客户端上（被本地 PlayerController 控制的 Pawn、PlayerController 自己）	采集输入、本地预测、发 Server RPC
`ROLE_SimulatedProxy`	其他客户端上	插值 / 平滑播放，不做权威判断
`ROLE_None`	不复制的对象	—

服务器上 Pawn 的 RemoteRole 是 AutonomousProxy，但复制给**非拥有者**连接时会被降级为 SimulatedProxy——同一个 Actor 在不同客户端上的 Role 不同 UE 文档 。所有权：沿 `GetOwner()` 链一路往上找到 PlayerController，它的 `NetConnection` 就是这个 Actor 的“拥有者连接”。它决定了三件事：Server RPC 能否发出（只有拥有者客户端能对该 Actor 调用 Server RPC）、`COND_OwnerOnly` 等条件复制发给谁、`bOnlyRelevantToOwner` 的 Actor 复制给谁。



图 11-2 Gameplay 框架各类存在于哪一端 UE 文档 。面试常问“客户端能不能拿到 GameMode”（不能，GameMode 只在服务器）、“别人的 PlayerController 在我这吗”（不在，PlayerController 只复制给拥有者）。详细的蓝图联机分析见 UE_Coop_Shooter 那本书。

11.3 每个网络 tick 发生了什么

服务器在 `UNetDriver::TickFlush` 中调用 `ServerReplicateActors`（使用 Replication Graph 时由它接管），网络 tick 频率受 `NetServerMaxTickRate` 限制（BaseEngine.ini 中 IpNetDriver 默认 30）

UE 源码 / 配置 。流程按官方文档“Detailed Actor Replication Flow”整理：

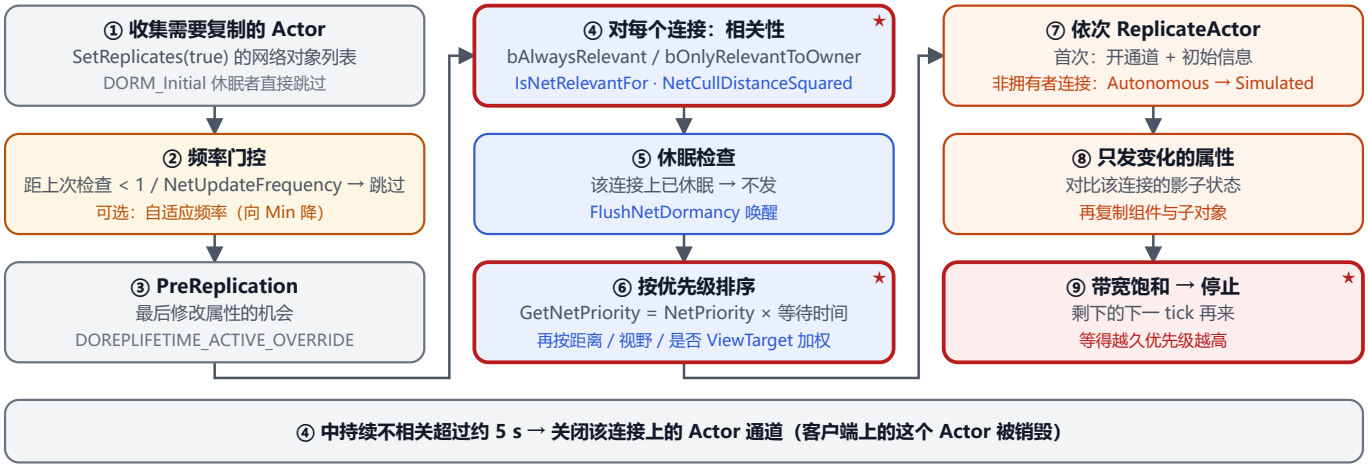


图 11-3 传统复制管线（`ServerReplicateActors`） UE 文档 。三列依次是“这个 Actor 这 tick 要不要考虑” “对这个连接要不要发、先发谁” “发什么、发到哪为止”。

属性 / 函数	作用	默认值 / 备注
`NetUpdateFrequency`	每秒最多检查几次是否需要复制	`AActor` 默认 100；`APlayerState` 在构造函数里设得很低（源码为 1）——挂 GAS 的项目通常调到 100；UE 5.5 起改为 `SetNetUpdateFrequency()` / `GetNetUpdateFrequency()`，直接访问已弃用
`MinNetUpdateFrequency`	开启自适应频率时的下限	默认 2
`NetPriority`	带宽竞争时的权重	Actor 1.0，Pawn / PlayerController 更高（3.0）
`NetCullDistanceSquared`	超过这个距离（平方）不相关	默认 225000000（即 150 米）
`bAlwaysRelevant` / `bOnlyRelevantToOwner` / `bNetUseOwnerRelevancy`	相关性的三个开关	GameState、PlayerState 总是相关；PlayerController 只对拥有者相关；武器常跟随拥有者的相关性
`NetDormancy`	休眠：长期不变就不再检查	`DORM_Initial` 适合关卡里静止的可破坏物；改变前调 `FlushNetDormancy()`
`ForceNetUpdate()`	下一 tick 立即考虑复制（无视频率门控）	低频 Actor 偶尔要立刻同步时用

11.4 实测：带宽饱和时先发谁 实测 E13

模型：30 Hz 网络 tick，每 tick 预算 1200 B；10 个英雄（优先级 3、希望 100 Hz、每次 60 B）、20 个投射物（2、30 Hz、30 B）、100 个可拾取物 / 机关（1、10 Hz、20 B）、10 个 PlayerState（1、1 Hz、40 B）。总需求明显超出预算。三种调度：UE 式（优先级 × 距上次发送时间）、静态优先级（永远先发优先级高的）、轮询。

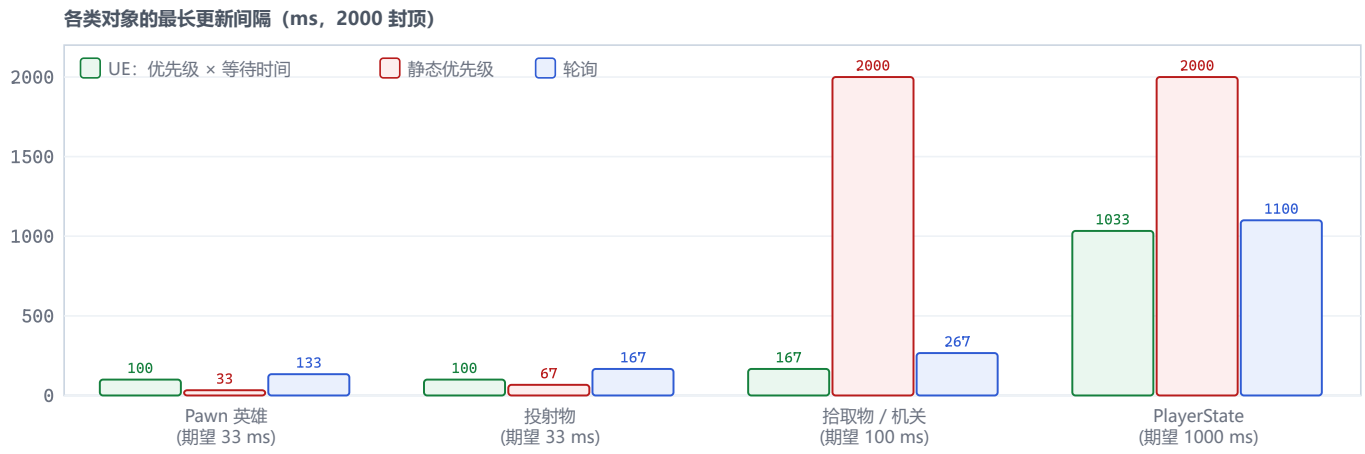


图 11-4 静态优先级会**饿死**低优先级对象：拾取物最长 60000 ms 没更新（40 个从未发出），PlayerState 最长 15000 ms；UE 式调度里等待时间会把低优先级对象的分数“养”上来，所有类别的最长间隔都有界。

调度	对象	期望间隔 ms	实际平均 ms	最长 ms	带宽占比	从未发出
UE：优先级 × 等待时间	Pawn 英雄	33	54	100	31.2%	0
UE：优先级 × 等待时间	投射物	33	67	100	25.0%	0
UE：优先级 × 等待时间	可拾取物 / 机关	100	131	167	42.6%	0
UE：优先级 × 等待时间	PlayerState	1000	1001	1033	1.1%	0
静态优先级	Pawn 英雄	33	33	33	50.0%	0
静态优先级	投射物	33	38	67	43.4%	0
静态优先级	可拾取物 / 机关	100	274	60000	6.3%	40
静态优先级	PlayerState	1000	1875	15000	0.3%	0
轮询	Pawn 英雄	33	55	133	30.6%	0
轮询	投射物	33	68	167	24.7%	0
轮询	可拾取物 / 机关	100	129	267	43.6%	0
轮询	PlayerState	1000	1007	1100	1.1%	0

表 11-1 E13 明细。轮询在这个模型里与 UE 式接近，但它不区分重要性；UE 式让英雄的平均间隔（54 ms）略好于轮询（55 ms），最长间隔 100 ms 对 133 ms。

“优先级 × 等待时间”的本质

它是**老化（aging）**调度：一个对象越久没被服务，分数就线性增长，迟早会超过任何高优先级对象，所以不会饿死；而高优先级对象的分数涨得更快，所以平均更新更勤。操作系统的进程调度、网络的加权公平队列都是同一类思想。HOK 的帧同步里没有这个问题——它每帧只发输入，不需要在对象之间分配带宽。

11.5 规模化：Replication Graph 与 Iris

	传统 ServerReplicateActors	Replication Graph	Iris
相关性计算	每个 Actor × 每个连接调用 `IsNetRelevantFor`（虚函数）	Actor 按类路由到节点（空间网格 `GridSpatialization2D`、总是相关、按连接总是相关……），每个连接直接从节点收集列表，结果可跨连接复用	过滤器（Filter）API：`NetObjectFilter`、网格过滤器等，替代 `IsNetRelevantFor`
优先级	`GetNetPriority`（虚函数）	节点内自定义	优先级器（Prioritizer）API，如 `SphereNetObjectPrioritizer`
属性复制	FRepLayout 对比影子状态	同左	复制状态与游戏线程数据分离，NetSerializer 量化后做增量；以 Push Model 为主（不推送时退化为按频率轮询）
RPC 与属性顺序	没有固定保证	同左	RPC 默认在同一连接的状态数据之后发送
适用	中小规模	大世界、大逃杀（Fortnite 使用）	新一代系统；与 Replication Graph 不兼容
状态	默认	插件，生产可用	UE 5.1 引入，截至 5.6 官方文档仍标注为 Experimental

来源：**UE 文档 / 源码**（Replication Graph 文档、《Getting Started With Iris》《Migrate to Iris》）。Iris 的迁移要点：开启已注册子对象列表 `bReplicateUsingRegisteredSubObjectList = true`；非 Actor / 组件的子对象要实现 `RegisterReplicationFragments`；自定义 `NetSerialize` 的结构体需要 Iris 版 NetSerializer。

版本提示

Iris、Mover、网络物理都是 UE5 中仍在快速演进的系统，面试时说“我了解它们的设计目标和当前状态（实验性 / Beta），生产项目仍以传统复制 + CMC 为主”比背 API 更稳妥。引用版本号时说清楚“截至我看的 5.x 文档”。

面试注意

高频题：“Role 有哪几种、AutonomousProxy 和 SimulatedProxy 区别”“为什么客户端调 Server RPC 没反应”（Actor 不归这个客户端所有，比如对场景里的门直接调 Server RPC——要通过自己的 PlayerController / Pawn 转发）“NetUpdateFrequency 和 tick 什么关系”（门控上限，实际还受 NetServerMaxTickRate 和带宽限制）“带宽不够怎么办”（相关性、休眠、频率、优先级、量化、Replication Graph）。

11.6 坑位

坑	后果	修正
对不归自己所有的 Actor 调 Server RPC	RPC 被丢弃，日志里有“No owning connection”类警告	经 PlayerController / 自己的 Pawn 转发，或设置 Owner
在客户端读 GameMode	空指针	需要客户端可见的数据放 GameState / PlayerState
PlayerState 上挂了频繁变化的数据却没调频率	更新明显滞后	调高 NetUpdateFrequency 或改用 ForceNetUpdate
大量静止 Actor 不休眠	服务器每 tick 白白检查	DORM_Initial / DormantAll，改变前 Flush
bAlwaysRelevant 滥用	大世界里带宽爆炸	用距离 / 队伍 / Replication Graph 节点控制
依赖“属性先到、RPC 后到”	传统复制下顺序不保证，偶发空引用	RPC 里带上需要的数据，或在 OnRep 中处理

本章速记

- NetDriver → NetConnection（每客户端）→ Control / Actor 通道；Bunch 级可靠性；PackageMap 映射 NetGUID；FRepLayout + 影子状态做增量。
- Role：Authority / AutonomousProxy / SimulatedProxy；所有权沿 Owner 链找 PlayerController → 连接，决定 Server RPC 与 OwnerOnly 复制。
- 管线：频率门控 → PreReplication → 相关性 → 休眠 → 优先级（NetPriority × 等待时间，距离 / 视野加权）→ 复制变化属性 → 饱和停止。
- E13：静态优先级会饿死低优先级对象，老化调度不会；规模化用 Replication Graph，新一代 Iris（5.1 引入、仍为实验性）。

▶ **自测 1：Listen Server 上，主机玩家的 Pawn 的 Role 是什么？客户端玩家的 Pawn 在主机上是什么？**

▶ **自测 2：一个 Actor 的 NetUpdateFrequency 设成 200，服务器 tick 30 Hz，它每秒最多复制几次？**

▶ **自测 3：为什么 Replication Graph 能比传统方式快很多？**

CHAPTER 12

Unreal 属性复制与 RPC：状态与事件

属性复制同步的是“现在是什么样”，RPC 传递的是“刚才发生了什么”。用错一个，就会出现“中途加入的人看不到门是开的”或者“满屏重复的受击特效”。

本章目标

- 写出属性复制的完整声明（UPROPERTY、GetLifetimeReplicatedProps、条件、RepNotify、Push Model），知道 OnRep 在哪端触发。
- 用实测理解属性复制的三个语义：只保证最终一致、中间值会被合并、丢包后发最新值。
- 掌握 RPC 的三种类型 × 可靠性 × “从哪端调用、Actor 归谁所有 → 在哪执行”表，以及可靠 RPC 的队头阻塞与缓冲溢出。
- 能根据数据的性质选择：属性 / 不可靠多播 / 可靠 RPC / GameplayCue / 计数器 + OnRep。

30 秒面试版答案

属性复制：在 UPROPERTY 上标 `Replicated` 或 `ReplicatedUsing=OnRep_X`，在 `GetLifetimeReplicatedProps` 里用 `DOREPLIFETIME(_CONDITION)` 注册，可加条件（OwnerOnly、SkipOwner、SimulatedOnly、InitialOnly 等）。服务器按连接保存影子状态，只发变化的属性；它只保证**最终一致**：短时间内的多次修改会被合并成一次，丢包后不重发旧包而是发最新值；中途加入的客户端也能拿到当前值。C++ 里 OnRep 只在客户端值变化时调用，服务器要自己手动调。RPC 分 Server（拥有者客户端 → 服务器）、Client（服务器 → 拥有者客户端）、NetMulticast（服务器 → 所有相关端），可靠 RPC 保证到达且在 Actor 通道内有序，但会队头阻塞、缓冲满了会断线；不可靠 RPC 可能丢。原则：持续状态用属性，一次性表现用不可靠多播或 GameplayCue，关键一次性事件才用可靠 RPC，而且尽量用“计数器属性 + OnRep”代替。

12.1 属性复制：声明与条件

UE C++：复制属性的完整写法（示例，未在本机编译）

```cpp
// MyCharacter.h
UPROPERTY(ReplicatedUsing = OnRep_Health)
float Health = 100.f;

UPROPERTY(Replicated)
int32 Ammo = 30;                        // 只有自己需要知道弹药数 → OwnerOnly

UPROPERTY(ReplicatedUsing = OnRep_BurstCounter)
uint8 BurstCounter = 0;                 // 开火计数器：每开一枪 +1，用来驱动别人端的开火特效

UFUNCTION() void OnRep_Health(float OldHealth);     // 可以带一个参数接收旧值
UFUNCTION() void OnRep_BurstCounter();

// MyCharacter.cpp
void AMyCharacter::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) cor
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    FDoRepLifetimeParams Params;
    Params.bIsPushBased = true;                                    // Push Model：只有标脏才检查
    DOREPLIFETIME_WITH_PARAMS_FAST(AMyCharacter, Health, Params);

    Params.Condition = COND_OwnerOnly;
    DOREPLIFETIME_WITH_PARAMS_FAST(AMyCharacter, Ammo, Params);

    Params.Condition = COND_SkipOwner;                             // 开枪者自己本地预测过了，不需
    DOREPLIFETIME_WITH_PARAMS_FAST(AMyCharacter, BurstCounter, Params);
}

void AMyCharacter::ApplyDamage_Server(float Damage)             // 只在服务器上调用
{
    const float Old = Health;
    Health = FMath::Max(0.f, Health - Damage);
    MARK_PROPERTY_DIRTY_FROM_NAME(AMyCharacter, Health, this);  // Push Model 下必须标脏
    OnRep_Health(Old);                                          // C++ 中服务器不会自动调 OnRep
}
```

复制条件（`ELifetimeCondition`）	发给谁	典型用途
`COND_None`	所有相关连接（默认）	血量、状态
`COND_InitialOnly`	只在初始复制时发一次	出生时确定、之后不变的配置（队伍、皮肤）
`COND_OwnerOnly`	只发给拥有者	弹药、背包、技能冷却等私有数据
`COND_SkipOwner`	发给除拥有者外的所有人	拥有者本地预测过的东西（开火计数器、预测的动画状态）
`COND_SimulatedOnly` / `COND_AutonomousOnly`	只发给模拟代理 / 只发给自主代理	只有旁观者需要的平滑数据 / 只有控制者需要的校正数据
`COND_SimulatedOrPhysics`	模拟代理或物理模拟的对象	物理相关数据
`COND_InitialOrOwner`	初始复制 + 之后只给拥有者	—
`COND_Custom`	由 `DOREPLIFETIME_ACTIVE_OVERRIDE` 在 PreReplication 中动态开关	按游戏状态决定是否复制
`COND_ReplayOrOwner` / `COND_ReplayOnly` / `COND_SkipReplay` 等	与回放（DemoNetDriver）相关	只为回放录制的数据
`COND_Dynamic` / `COND_Never`	运行时可改的条件 / 永不复制（UE5 新增）	—

OnRep 语义	说明 UE 文档 / 源码
在哪端触发	**C++：只在客户端**收到新值时触发，服务器修改不会自动调用；**蓝图 RepNotify：服务器上通过蓝图设置变量时也会调用**——两者不一致是常见坑
何时触发	默认 `REPNOTIFY_OnChanged`：收到的值与本地不同才触发；`REPNOTIFY_Always`：每次收到都触发（客户端本地改过值、服务器再发同样的值时有用）
多个属性	同一 Actor 一次更新里的多个属性会先全部写入、再依次调用各自的 OnRep，所以 OnRep 里读到的其他属性是“同一批”的新值
对象引用	复制的是 NetGUID；客户端还没有那个对象时指针为空，等对象到达、映射完成后会再次触发 OnRep

数组与结构体：`TArray` 按元素做增量；大数组（背包、Buff 列表）用 `FFastArraySerializer`，元素变化时 `MarkItemDirty`，客户端有 `PostReplicatedAdd / PostReplicatedChange / PreReplicatedRemove` 逐元素回调（GAS 的激活效果列表就是这么做的）。结构体默认按成员增量复制；实现了 `NetSerialize`（`WithNetSerializer` 特征）就作为整体原子复制，常用来做量化（如 `FVector_NetQuantize10`）。

12.2 属性复制的三个语义

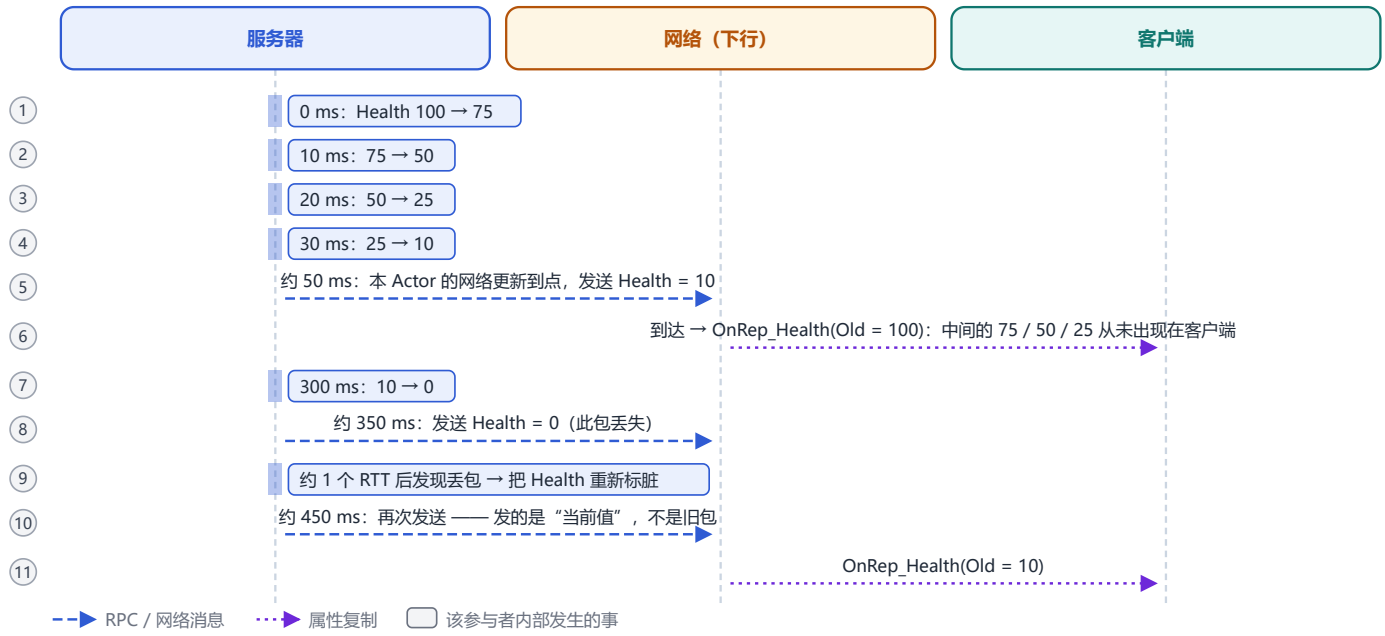


图 12-1 属性复制只保证**最终一致**：两次网络更新之间的多次修改被合并；丢包后发送的是最新值。所以不能用“OnRep 的次数”去数事件次数（受击次数、开枪次数），要用计数器或事件通道。

实测：同一组受击，属性 vs RPC 实测 E14

模型：0 / 10 / 20 / 30 ms 连续 4 次受击（100 → 75 → 50 → 25 → 10），300 ms 再受击一次（→ 0）。属性：NetUpdateFrequency = 10（每 100 ms 一次更新，相位随机），下行丢包 20%，单程 40 ms，丢包后约 90 ms 发现并在下次更新补发最新值。RPC：每次受击发一个多播；可靠 RPC 丢包后 100 ms 重传、按序交付。2 万次试验：

通道	客户端收到什么	延迟	适合
复制属性 Health	平均只触发 2.22 次 OnRep（5 次变化中的中间值大多被合并），但**最终值一定正确**	最终值平均在 420 ms 到达（第 5 次受击在 300 ms）	持续状态；中途加入也能拿到
不可靠多播 RPC	平均收到 4.00 / 5 个事件	最低	可以丢的表现（受击特效、音效）
可靠多播 RPC	5 个都收到、按序	平均 90 ms（前一个丢了，后面的即使先到也要等：队头阻塞）	必须到达、次数要准的关键事件，且频率低

这次试验里 OnRep 触发次数	占比 %
1	0.8
2	76.0
3	23.2

表 12-1 OnRep 次数分布：大多数试验只触发 2 次（4 次连击合并成 1 次 + 最后 1 次），只有更新时刻恰好落在连击中间时才会触发 3 次。

12.3 RPC：类型、可靠性、在哪执行

UE C++：三种 RPC（示例，未在本机编译）

```
UFUNCTION(Server, Reliable, WithValidation)          // 拥有者客户端 → 服务器
void ServerFire(FVector_NetQuantize Origin, FVector_NetQuantizeNormal Dir, float ClientTime);

UFUNCTION(Client, Reliable)                          // 服务器 → 拥有者客户端
void ClientRejectFire(uint8 Reason);

UFUNCTION(NetMulticast, Unreliable)                  // 服务器 → 服务器自己 + 所有相关客户端
void MulticastHitFX(FVector_NetQuantize Where);

bool AMyWeapon::ServerFire_Validate(FVector_NetQuantize Origin, FVector_NetQuantizeNormal Dir,
{
    return Dir.IsNormalized();                       // 返回 false 会把这个客户端踢掉：只用于
}
void AMyWeapon::ServerFire_Implementation(FVector_NetQuantize Origin, FVector_NetQuantizeNorma
{
    if (!FireRateBucket.TryConsume(GetWorld()->GetTimeSeconds())) { ClientRejectFire(1); retur
    // ... 服务器权威的弹药扣减、延迟补偿命中判定（第 10 章）
}
```

在哪调用	RPC 类型	Actor 归调用方连接所有	Actor 归其他客户端 / 服务器 / 无主
服务器	NetMulticast	服务器 + 所有（相关的）客户端	
	Server	在服务器上执行（相当于本地调用）	
	Client	归某个客户端所有 → 在那个客户端执行	无主 / 服务器所有 → 在服务器执行
客户端	Server	**在服务器执行**	**被丢弃**
	Client	在调用方客户端本地执行	
	NetMulticast	只在调用方客户端本地执行（不会发到网络上）	

表 12-2 RPC 调用与执行端 UE 文档 。最常见的 bug 是第五行：客户端对“不归自己所有”的 Actor 调 Server RPC，直接被丢弃。

可靠性	保证	代价 / 风险
Reliable	一定到达；同一 Actor 通道内按调用顺序执行	丢包时后面的可靠 RPC 要等它（E14 平均 90 ms）；可靠缓冲有上限，持续高频发送会溢出并**断开连接**
Unreliable	不保证到达、不保证顺序	带宽紧张或丢包时会丢；适合高频、可丢的数据（移动、表现）

多播与相关性

NetMulticast 只发给“当前与该 Actor 相关”的连接；之后才变得相关、或中途加入的客户端**收不到之前的多播**。所以“门打开了”“宝箱被开过”这种持续状态必须用属性（新来的人能拿到当前值），多播只用于“此刻播一下就行”的表现。传统复制系统下，同一 Actor 的属性与 RPC 到达顺序也没有固定保证（Iris 改为 RPC 在状态数据之后发送） UE 文档 。

12.4 怎么选：状态、事件，还是计数器

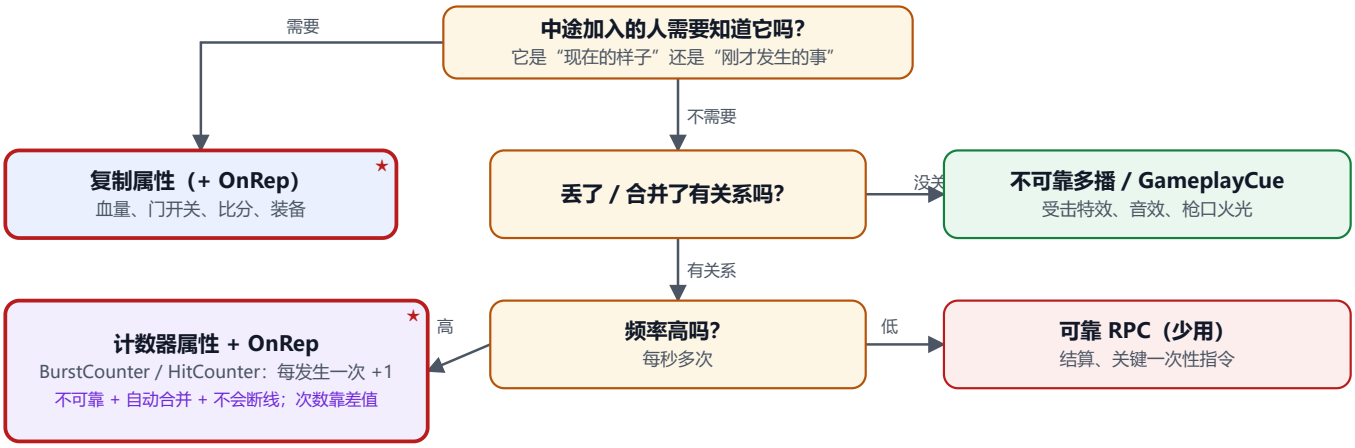


图 12-2 数据通道的选择。计数器模式（ShooterGame 的 `BurstCounter` 就是这样：服务器每次开火 +1，模拟代理在 OnRep 里播放开火表现）兼顾了“次数可推算”和“不阻塞、可合并”。

对照 HOK

帧同步里没有“属性复制 vs RPC”的选择题：网络上只有输入，状态与事件都是本地演算出来的（第 5 章的结果事件）。HOK 的上行技能命令需要“必须到达、按序”，它没有用可靠通道，而是用“不可靠 + 全量冗余 + 序号”（第 7 章）——和这里“计数器属性 + OnRep”是同一个思路：**用幂等的、可合并的表示代替可靠有序的消息流。**

面试注意

高频题：“属性复制和 RPC 的区别、各自适用场景”（状态 vs 事件、最终一致 vs 逐条、新加入者能否拿到）；“OnRep 在服务器会调用吗”（C++ 不会、蓝图会）；“Reliable 有什么风险”（队头阻塞、缓冲溢出断线，Tick 里发 Reliable 是大忌）；“为什么客户端调 Server RPC 没执行”（所有权）。能讲出 E14 的“5 次变化只触发 2 次 OnRep”，说明你理解了属性复制的合并语义。

12.5 坑位

坑	后果	修正
用 OnRep 次数统计事件次数	次数偏少（被合并）	计数器差值，或事件通道
持续状态用多播	中途加入 / 后来相关的人看不到	属性
Tick 里发 Reliable RPC	弱网下可靠缓冲溢出，连接被关闭	高频数据用不可靠，或合并成属性
C++ 服务器改了属性却期待 OnRep 执行	服务器端（Listen Server 主机）没表现	服务器手动调用 OnRep 或抽出公共函数
Push Model 下忘记标脏	属性永远不复制	修改处统一 `MARK_PROPERTY_DIRTY`，或封装 Setter
_Validate 里做业务校验（射速等）	网络抖动导致的正常玩家被踢	_Validate 只查明显非法；业务校验在 _Implementation 里拒绝并回滚
大数组整体复制	每次改一个元素都可能触发大量比较 / 发送	FFastArraySerializer

本章速记

- 属性：`Replicated / ReplicatedUsing` + `DOREPLIFETIME(_CONDITION / _WITH_PARAMS_FAST)`；条件 OwnerOnly / SkipOwner / InitialOnly …；Push Model 要标脏。
- 语义：最终一致、中间值合并、丢包发最新值；E14：5 次变化平均 2.22 次 OnRep。C++ OnRep 只在客户端触发。
- RPC：Server（只能由拥有者客户端发起）/ Client / NetMulticast（只发给相关连接）；Reliable 有序但队头阻塞（E14 90 ms）且可能溢出断线。
- 选择：状态 → 属性；可丢表现 → 不可靠多播 / GameplayCue；高频且要计数 → 计数器 + OnRep；低频关键 → 可靠 RPC。

▶ **自测 1：子弹打中墙留下弹孔，用什么同步？**

▶ **自测 2：为什么开火计数器要用 COND_SkipOwner?**

▶ **自测 3：Listen Server 上，主机玩家受伤时 C++ 的 OnRep_Health 会执行吗?**

CHAPTER 13

Unreal 的预测：CMC、GAS、Mover 与网络物理

第 10 章的“预测 + 和解”落到 Unreal 里有三个实现：角色移动交给 CharacterMovementComponent，技能交给 GAS 的 Prediction Key，而 Mover 和网络物理是 Epic 把它们通用化的下一代尝试。

本章目标

- 按时序讲完 CMC 的一次移动：SavedMove → 本地执行 → ServerMove → 服务器重放与误差判定 → ClientAckGoodMove / ClientAdjustPosition → 客户端重放未确认移动。
- 会给 CMC 加一个可预测的自定义移动（冲刺），知道 SavedMove / 压缩标志 / CanCombineWith 各自的作用。
- 说清 GAS 的预测：Prediction Key、LocalPredicted、哪些东西能预测、被拒绝时怎么撤销、为什么不会重复扣两次。
- 了解 Mover、Network Prediction Plugin、Chaos 网络物理（预测插值 / 重模拟）的定位与成熟度。

30 秒面试版答案

CMC 是 Unreal 的角色移动预测：自主代理每帧把输入（加速度、跳跃等压缩标志）和时间戳存成 SavedMove，本地立刻执行，再通过不可靠的 ServerMove RPC 发给服务器（顺带一个重要的旧移动作冗余）；服务器用客户端时间戳算出时间差、重放这次移动，比较客户端声称的位置，误差在阈值内就回 ClientAckGoodMove，超出就回 ClientAdjustPosition；客户端收到纠正后设到权威位置，再把时间戳之后的 SavedMoves 全部重放——这就是第 10 章的和解。其他端的角色是模拟代理，靠复制的移动数据插值并平滑网格。GAS 用 Prediction Key 做技能预测：LocalPredicted 技能在客户端立即激活，生成一个 key 随 ServerTryActivateAbility 发给服务器，期间产生的 GE、GameplayCue、蒙太奇都挂在这个 key 上；服务器拒绝则整体撤销，接受则用复制下来的 key 对账、去掉本地预测的副本避免重复。Mover 是更模块化的新移动框架，网络物理提供预测插值和重模拟，二者在 UE5 中仍处于实验性 / Beta 阶段。

13.1 CMC 的一次移动



图 13-1 CharacterMovementComponent 的网络流程 **UE 源码** 。和第 10 章 E11 的“预测 + 和解”逐项对应：SavedMoves = 未确认输入队列，时间戳 = 输入序号，ClientAdjustPosition = 权威状态，重放 = 和解。

CMC 概念	第 10 章的通用概念	说明
`FSavedMove_Character` / `SavedMoves`	pending 输入队列	保存加速度、压缩标志、时间戳、起止状态；对象池分配
TimeStamp	输入序号	同时用于服务器计算 DeltaTime；服务器会校验时间戳（防加速）
ServerMove / ServerMovePacked	上行输入	不可靠；带一个“重要的旧移动”（OldMove）作冗余；UE 4.26 起默认使用打包的移动数据容器（新移动 + 待合并移动 + 旧移动）
`MAXPOSITIONERRORSQUARED`（GameNetworkManager）	纠正阈值	位置误差平方超过它才纠正，避免浮点误差导致每帧纠正
ClientAdjustPosition / ClientVeryShortAdjustPosition	权威状态	服务器在发送客户端调整时（每帧最多一次）发出，带时间戳
ClientUpdatePositionAfterServerUpdate	和解重放	把纠正时间戳之后的 SavedMoves 重新执行一遍

调试工具

`p.NetShowCorrections 1` 在发生纠正时画出客户端与服务器的胶囊体位置（`p.NetCorrectionLifetime` 控制显示时长）；`net PktLag=100`、`net PktLoss=5`、`net PktLagVariance=30` 模拟延迟 / 丢包 / 抖动；Network Insights（Unreal Insights 的网络页）看每个 Actor 的复制带宽。纠正频繁出现时，先查“客户端与服务器的移动逻辑是否一致”（例如只在客户端改了 MaxWalkSpeed）。

13.2 给 CMC 加一个可预测的冲刺

最常见的错误：客户端按下冲刺时直接改 `MaxWalkSpeed`，再用一个 Server RPC 通知服务器改。结果是客户端先跑快、服务器晚一个 RTT 才跑快，这段时间里每一步都和服务器对不上，被连续 ClientAdjustPosition 拉回（第 10 章“橡皮筋”）。正确做法是把“是否冲刺”放进 SavedMove，让它和移动输入**一起**被预测、发送、重放：

UE C++：可预测的冲刺（压缩标志写法，示例，未在本机编译）

```
class FSavedMove_MyChar : public FSavedMove_Character {
public:
    uint8 bWantsToSprint : 1;
    virtual void Clear() override { FSavedMove_Character::Clear(); bWantsToSprint = 0; }
    virtual uint8 GetCompressedFlags() const override {
        uint8 F = FSavedMove_Character::GetCompressedFlags();
        if (bWantsToSprint) F |= FLAG_Custom_0;                 // 随 ServerMove 一起上行
        return F;
    }
    virtual bool CanCombineWith(const FSavedMovePtr& New, ACharacter* C, float MaxDelta) const
        if (bWantsToSprint != ((FSavedMove_MyChar*)New.Get())->bWantsToSprint) return false;
        return FSavedMove_Character::CanCombineWith(New, C, MaxDelta);
    }
    virtual void SetMoveFor(ACharacter* C, float Dt, const FVector& Accel, FNetworkPredictionD
        FSavedMove_Character::SetMoveFor(C, Dt, Accel, Data);
        bWantsToSprint = Cast<UMyCMC>(C->GetCharacterMovement())->bWantsToSprint;   // 记录这一
    }
    virtual void PrepMoveFor(ACharacter* C) override {          // 重放（和解）时恢复这一步的输入
        FSavedMove_Character::PrepMoveFor(C);
        Cast<UMyCMC>(C->GetCharacterMovement())->bWantsToSprint = bWantsToSprint;
    }
};

void UMyCMC::UpdateFromCompressedFlags(uint8 Flags) {           // 服务器从标志位恢复输入
    Super::UpdateFromCompressedFlags(Flags);
    bWantsToSprint = (Flags & FSavedMove_Character::FLAG_Custom_0) != 0;
}
float UMyCMC::GetMaxSpeed() const {                             // 两端用同一个函数、同一个输入算
    return (bWantsToSprint && IsMovingOnGround()) ? SprintSpeed : Super::GetMaxSpeed();
}
// 还需要：FNetworkPredictionData_Client_MyChar::AllocateNewMove() 返回 FSavedMove_MyChar，
// 并在 UMyCMC::GetPredictionData_Client() 里创建它。
```

压缩标志只有 4 个自定义位（`FLAG_Custom_0~3`）；需要更多数据（例如冲刺能量、瞄准方向）时，UE 4.26 起可以继承 `FCharacterNetworkMoveData` 自定义打包的移动数据 **UE 源码** 。这一节的原则对任何预测系统都成立：**影响模拟结果的一切输入，都必须进入“被保存、被发送、被重放”的那份数据**——和帧同步“所有输入都要进帧命令”完全一样。

13.3 模拟代理：别人的角色怎么平滑

其他客户端上的角色不跑预测，它们通过 Actor 的 `ReplicatedMovement`（位置、旋转、速度，可量化）接收服务器状态，CMC 在两次更新之间按速度模拟、收到新状态时做 **SmoothCorrection**：胶囊体直接到新位置，**网格（Mesh）**保留一个偏移、在若干帧内平滑收敛（`NetworkSmoothingMode`：Disabled / Linear / Exponential）。这就是第 10 章“逻辑位置精确、显示位置平滑”的实现：碰撞与判定用胶囊，玩家看到的是平滑后的网格。

为什么 CMC 不像 Source 那样做固定延迟的快照插值?

CMC 的模拟代理默认更接近“收到即用 + 模拟外推 + 平滑”，没有一个全局的“渲染过去 100 ms”的插值时间轴，所以延迟更低，但在高抖动下更容易看到修正。需要严格插值（例如电竞回放、观战）的项目会自己做快照缓冲。延迟补偿也不是引擎内置的：Unreal 没有通用的“服务器回溯命中盒”，射击游戏通常自己维护历史（或者像很多项目一样，客户端判定命中、服务器做合理性校验）`推断`（基于引擎源码中无通用回溯模块）。

13.4 GAS 的预测：Prediction Key

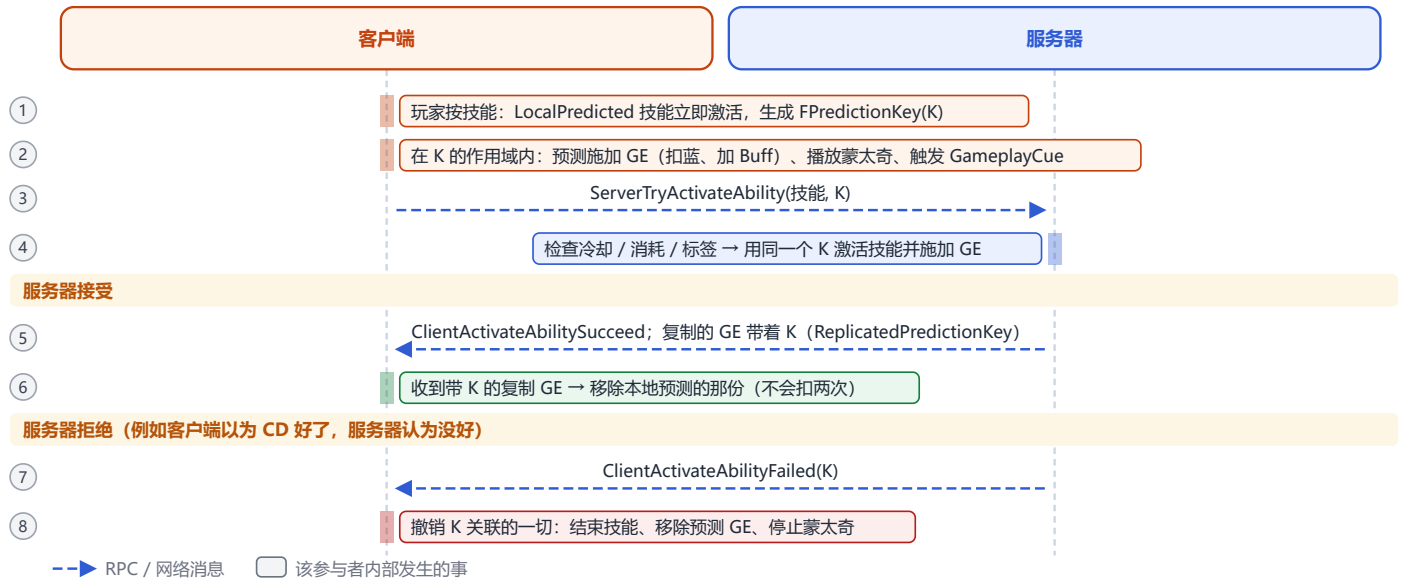


图 13-2 GAS 的预测流程 `UE 源码（GameplayPrediction.h 注释）`。Prediction Key 就是第 10 章“输入序号”在技能系统里的化身：所有预测副作用都挂在 key 上，才能整体对账或整体撤销。

能预测（随 Prediction Key）	不预测 / 有限制
技能激活；由预测技能触发的事件；GE 施加（属性修改、标签修改）；GameplayCue（随 GE 或单独执行）；蒙太奇；移动（交给 CMC）	GE 的移除；周期性效果（DOT）；预测施加的瞬时 GE 在客户端被当作“无限时长”处理，以便确认或撤销；伤害等关键结果通常只在服务器结算

Net Execution Policy	含义	适用
`LocalPredicted`	客户端预测激活，服务器确认	玩家主动技能（默认推荐）
`LocalOnly`	只在本地客户端运行	纯表现、本地 UI 交互
`ServerInitiated`	服务器激活，再在拥有者客户端运行	服务器触发的技能
`ServerOnly`	只在服务器运行	被动、AI、纯逻辑

需要客户端提供瞄准数据的技能（例如 `WaitTargetData`）会把目标数据随 key 通过 `ServerSetReplicatedTargetData` 发给服务器；服务器在 _Implementation 里校验它是否合理（与第 10 章“延迟补偿要校验客户端声称的时刻”是同一个问题）。

13.5 下一代：Mover、Network Prediction Plugin、网络物理

系统	定位	状态（截至查阅的资料）
Network Prediction Plugin（NPP）	通用的“输入 / 同步状态 / 辅助状态 + 固定 tick + 回滚重模拟”框架，把 CMC 那套写法抽象成模板	实验性；社区反馈其开发重心已转向 Chaos 物理后端
Mover（Mover 2.0）	CMC 的长期替代：移动模式（Movement Mode）+ 叠加移动（Layered Move）+ 修饰器，模块化、可用于非 Actor；网络预测由后端提供（NPP 或 Chaos）	5.4 起以实验性插件提供，5.6~5.7 期间进入 Beta（不同资料说法不一，以官方 release notes 为准）；官方仍在做破坏性 API 调整
Chaos 网络物理	物理复制模式：预测插值（Predictive Interpolation，在插值时间线上做局部预测）、重模拟（Resimulation，状态偏差超过阈值时回滚重算）；`NetworkPhysicsComponent` 管理输入 / 状态历史、输入冗余与衰减	5.4：预测插值为 Beta、重模拟为实验性；项目设置里有“支持的最大 RTT”“触发重模拟的误差阈值（厘米）”等参数；物理复制 LOD 在 5.7 中仍在完善

来源：**UE 文档 / 公开路线图 / 官方论坛**。这些系统的共同点是把 GGPO 式的“历史 + 回滚重算”带进了服务器权威的状态同步——第 9 章的结论在这里再次成立：**预测必然伴随回滚，回滚必然需要历史和确定性的重算**（物理的重算只需要“足够接近”，不需要逐位一致，因为服务器会持续纠正）。

面试注意

“CMC 怎么做网络同步”几乎必考，答题顺序：自主代理保存 SavedMove 并本地执行 → 不可靠 ServerMove（带时间戳 + 旧移动冗余）→ 服务器重放并按误差阈值决定 Ack 还是 Adjust → 客户端纠正后重放未确认移动 → 模拟代理插值 + 网格平滑。追问“怎么加冲刺 / 翻滚”时，一定要说“把输入放进 SavedMove（压缩标志或自定义移动数据），不要用 RPC 改速度”。GAS 问到预测时，关键词是 Prediction Key、LocalPredicted、撤销与去重。

13.6 坑位

坑	后果	修正
用 RPC 改 MaxWalkSpeed 做冲刺	冲刺开始 / 结束时连续纠正、橡皮筋	输入进 SavedMove，两端同一函数算速度
只在客户端改了影响移动的参数（重力、摩擦、根运动）	持续纠正	参数也要复制，或由输入驱动
自定义 SavedMove 没实现 CanCombineWith	状态切换的那一步被合并掉，服务器看不到	状态不同的移动禁止合并
根运动蒙太奇只在一端播放	位置分歧	根运动蒙太奇要在预测端和服务器端同时播放（GAS 的 PlayMontageAndWait 会处理）
GAS 里在客户端直接改属性	服务器不知道、马上被复制覆盖	通过预测 GE 修改
预测 GE 施加了伤害	服务器拒绝时“伤害撤销”很怪	伤害等关键结果只在服务器结算，客户端只预测表现

本章速记

- CMC：SavedMove（加速度 + 标志 + 时间戳）→ 本地执行 → 不可靠 ServerMove（+ 旧移动）→ 服务器重放、误差阈值 → Ack / Adjust → 客户端重放未确认移动；模拟代理插值 + 网格平滑。
- 自定义移动：输入进 SavedMove（FLAG_Custom_0~3 或自定义移动数据），实现 SetMoveFor / PrepMoveFor / CanCombineWith / UpdateFromCompressedFlags。
- GAS：Prediction Key 挂住所有预测副作用；LocalPredicted；拒绝整体撤销，接受按 key 去重；GE 移除、周期效果不预测。
- Mover / NPP / Chaos 网络物理：把“历史 + 回滚重算”通用化；仍在实验性 / Beta 阶段。

▶ **自测 1：CMC 为什么用不可靠 RPC 发 ServerMove?**

▶ **自测 2：客户端开了加速器（时间流速变快），CMC 怎么防?**

▶ **自测 3：GAS 预测扣蓝，服务器也扣了一次，为什么玩家不会看到扣两次?**

PART IV

选型、混合与面试

把两条线并到一起：两个旋钮看懂所有方案，一张全维对照表做选型，几个公开案例校准直觉，最后是在 Unreal 里做帧同步的工程清单和 50 道追问。

CHAPTER 14

选型、混合方案与业界案例

帧同步和状态同步不是两个阵营，而是两个旋钮的不同位置：权威放在哪，网上传什么。

本章目标

- 用“权威位置 × 传输内容”两个旋钮给任意同步方案定位，包括 HOK 的 CC / CS、LoL、守望先锋。
- 掌握一张 14 个维度的全维对照表，能针对需求说出取舍。
- 能把 HOK 的每个机制映射到 Unreal 中的对应物，反之亦然。
- 知道如果要在 Unreal 里做帧同步，需要绕开哪些引擎设施、自己补哪些模块。

30 秒面试版答案

选型先问能不能做确定性（定点数、单线程、不用引擎物理），不能就只有状态同步；能的话再看单位规模、录像观战需求、网络环境、反作弊要求和对延迟的敏感度：单位多、要录像、移动网络、服务器成本敏感 → 定频广播帧同步（HOK）；2 人、状态小、帧级延迟敏感 → 回滚；透视敏感、单位少、要强权威 → 状态同步 + 预测 + 延迟补偿（Unreal 默认）。现实中的方案大多是混合的：LoL 是状态同步但服务器做成了确定性以支持回档，守望先锋是状态同步但输入驱动的系统用固定命令帧做预测回滚，HOK 同时有 CC 帧同步和 CS 服务器演算两种模式。

14.1 两个旋钮



图 14-1 两个旋钮下的方案地图。左上是帧同步，右下是经典状态同步，但真实游戏经常落在两格之间：HOK 在左上（CC）与右上（CS）之间切换，LoL 在右下却借用了左上的确定性。

14.2 全维对照表

维度	定频广播帧同步（HOK）	回滚（GGPO）	状态同步（UE 默认）
一致性来源	确定性 + 相同输入	确定性 + 相同输入（事后修正）	服务器权威状态
带宽	∝ 玩家 × 操作频率，与单位数无关（E1）	同左，P2P	∝ 相关单位 × 变化频率；靠增量 / AOI / 优先级压缩
自己的操作延迟	自己 → 服务器往返（E3：约 112 ms），靠预表现掩盖	约等于输入延迟 d（E3：约 68 ms）	本地预测，几乎为 0
看到别人	与自己同一时间	当前帧，可能跳变	插值后的过去（约 RTT/2 + 100 ms）
一人网卡的影响	只影响自己	短：别人轻微跳变；长：全员等待	只影响自己（服务器照常模拟他）
服务器成本	低（不跑逻辑）	无 / 极低	高（每局跑完整逻辑 + 复制）
防改数值	帧 hash 比对	同左（通常无服务器，需额外设计）	天然（权威在服务器）
防透视	难（客户端有全量数据）	难	可做（按可见性裁剪复制）
断线重连	拉历史输入重放 + 追帧	通常不支持中途加入	拉一份当前状态即可
录像 / 观战	极小、免费	极小	录复制数据流，较大
确定性要求	逐位一致（定点数）	更严苛（反复重算）	无（预测部分“足够接近”即可）
引擎物理 / 动画驱动逻辑	不能用于逻辑	不能	可以
开发 / 调试难点	desync 定位、抗丢包、追帧	状态序列化、跳变、音频	带宽、所有权 / Role、预测纠正、命中公平

适合	MOBA、RTS、多人战场	格斗、1v1 动作、体育	FPS、MMO、开放世界、大多数 UE 项目

14.3 HOK ⇄ Unreal 概念映射

问题	HOK（帧同步）	Unreal（状态同步）
时间节拍	服务器帧号（66 ms）	服务器网络 tick（NetServerMaxTickRate）+ 每 Actor NetUpdateFrequency
上行	帧命令 `CSID_GAMING_UPERMSG`（UDP）	Server RPC（移动走 CMC 的 ServerMove）
上行确认	在下行帧里看到自己的命令；期待帧号	ClientAckGoodMove / 复制状态；可靠 RPC 由通道确认
抗丢包	下行一包多帧、上行短冗余 / 全量冗余、补发、追帧	属性发最新值、可靠 Bunch 重传、ServerMove 带旧移动
乱序	900 帧窗口	包序号；可靠 RPC 通道内有序
表现分离	结果事件 → AGE（C++ 决策、C# 执行）	GameplayCue、OnRep 驱动表现、不可靠多播
预测	不预测逻辑；指示器等“预表现”	CMC、GAS Prediction Key、网络物理重模拟
一致性 / 反作弊	帧 hash 比对	服务器权威 + RPC 校验
带宽调度	无需（只有输入）	相关性、优先级 × 等待时间、休眠、Replication Graph / Iris
重连	Recover：1090 / 1091 / 1093，追帧	重新连接 + 初始复制；PlayerState 保留
录像	输入序列，回放与实战同路径	DemoNetDriver 录复制流，检查点跳转
网络质量	网络线程时间戳 ping、卡顿四维归因、双链路	PlayerState ping、NetConnection 统计、Network Insights

14.4 公开案例速览

游戏	方案	值得记住的点	来源
王者荣耀	定频广播帧同步（CC）+ CS 模式	三层冗余、900 帧窗口、逻辑表现分离、hash 校验	HOK 文档
星际争霸 / 魔兽争霸 3	严格 Lockstep	“某玩家正在拖慢游戏”；录像就是输入文件	公开资料
英雄联盟	状态同步 + 确定性服务器	为可录制输入的测试（Delta Checker）和赛场回档（Chronobreak）把服务器做成确定性	Riot 技术博客
守望先锋	状态同步 + 命令帧预测回滚	16 ms 命令帧（比赛 7 ms）；客户端领先服务器约半个 RTT + 一个缓冲帧；每个包带全部未确认输入；丢包时服务器让客户端加速推进命令帧以扩大服务器缓冲	GDC 2017
Rocket League	服务器权威物理 + 客户端预测回滚	对车和球都做物理预测，收到服务器状态后回滚重算	GDC 2018
Valorant	状态同步，128 tick 服务器	高 tick 率缩小先手优势与延迟补偿误差	Riot 技术博客
主流格斗游戏	回滚 + 小输入延迟	GGPO 及其后继；可调输入延迟	公开资料
Fortnite	Unreal 状态同步 + Replication Graph	大逃杀规模的相关性计算	UE 文档

守望先锋的“时间膨胀”值得多看一眼

状态同步的服务器也需要输入缓冲：客户端输入到达服务器的时机有抖动，服务器在每个命令帧需要“刚好有一份输入可用”。守望先锋的做法是：服务器发现缓冲不够（丢包 / 抖动），就通知客户端**略微加快**命令帧的推进速度，让输入提前一点到达；网络变好再放慢回去 GDC 2017 。这和 HOK 下行“窗口 + 补发”、格斗“输入延迟 d”、GGPO“时间同步”解决的是同一个问题：**用一个小的、可调的缓冲吸收抖动。**

14.5 在 Unreal 里做帧同步：工程清单

Unreal 的设施几乎全部围绕状态同步设计。要做 HOK 式帧同步，基本思路是**把 UE 当成“表现层 + 输入层 + 传输层”，逻辑层自己写**——就像 HOK 把 Unity 当表现层一样 `推断`：

模块	做法	要避开的坑
逻辑层	独立的纯 C++ 模块（不依赖 UObject / 引擎时间），定点数数学库、确定性随机、数组式实体存储（ECS 风格），固定帧推进	不要在逻辑里用 `float` / `FVector`、`TMap` 遍历、`FMath::Rand`、`GetWorld()->GetTimeSeconds()`、蓝图数学节点
物理 / 碰撞	逻辑层自研定点碰撞（圆、胶囊、网格寻路）	Chaos 物理不是逐位确定的，不能参与逻辑；根运动、动画通知不能驱动逻辑
表现层	Actor 只是“视图”：逻辑层产出结果事件（生成、移动、播放技能、受击），Actor 订阅并插值；类似 HOK 的 AGE / UE 的 GameplayCue	表现层不能改逻辑状态；Actor 不开复制（或只复制与逻辑无关的东西）
输入	Enhanced Input 采集 → 量化成帧命令（方向量化到整数、技能槽 + 目标）→ 命令序号	不要把浮点摇杆值直接送进逻辑
传输	方案 A：独立 UDP（自研或 KCP）连 Relay 服务；方案 B：复用 NetDriver，用 PlayerController 上的不可靠 RPC 发命令、自己做冗余与序号	不要用 Reliable RPC 发每帧命令（队头阻塞 + 缓冲溢出）
Relay 服务	独立服务（定帧、汇总、广播、缓存、补发），或无渲染的 UE 专用服务器只跑转发逻辑	服务器不等人；期待帧做 ACK
校验与录像	逻辑层周期性算 hash 上报；录像 = 输入序列 + 版本号	逻辑层版本必须与录像绑定
CI	跨编译器 / 平台重放同一份录像比对 hash；回滚类项目再加同步测试模式	只在一个平台测试等于没测

反过来，如果项目已经深度依赖 Unreal 的 Gameplay 框架（CMC、GAS、蓝图逻辑、Chaos），想“改成帧同步”几乎等于重写逻辑层——这正是 GGPO 资料里反复提到的“回滚无法后装”的问题：**确定性与逻辑表现分离必须在项目第一天就决定。**

14.6 常见混合设计

帧同步 + 服务器抽查

CC 模式为主，服务器对可疑对局（hash 少数派、举报）离线重放录像复核，兼顾成本和反作弊。

状态同步 + 确定性子模拟

弹道、技能时间轴、伤害公式写成确定性的，服务器只同步“开火事件 + 种子”，各端自己算轨迹，大幅减少投射物复制。

状态同步 + 局部预测回滚

只对本地玩家相关的系统（移动、技能起手）做预测回滚，其他一律插值（守望先锋、CMC、GAS）。

帧同步 + 关键事件可靠通道

帧命令走不可靠 UDP + 冗余，开局 / 结算 / 投降等走 TCP 或可靠通道（HOK 的 UDP + TCP 双通道）。

面试注意

设计题（“让你做一个 3v3 竞技游戏的网络同步”）的答题框架：① 先问约束（人数、单位规模、品类、平台与网络、反作弊等级、是否要录像 / 观战 / 断线重连、引擎）；② 用两个旋钮给出方案并说明理由；③ 讲三个关键机制（例如状态同步就讲预测和解、插值、延迟补偿，帧同步就讲确定性、冗余补发、追帧）；④ 讲度量与兜底（ping、卡顿归因、desync / 纠正统计、弱网测试）。不要一上来就说“用帧同步”或“用 UE 默认的就行”。

本章速记

- 两个旋钮：**权威在哪 × 网上传什么**；HOK CC / CS、LoL、守望先锋都是旋钮的中间位置。
- 14 维对照：带宽、延迟、影响域、服务器成本、反作弊（改数值 vs 透视）、重连、录像、确定性、物理、调试难点、适用品类。
- HOK ⇄ UE 一一映射：帧号 ⇄ 网络 tick；冗余 / 窗口 ⇄ 属性最新值 / 可靠 Bunch；AGE ⇄ GameplayCue；预表现 ⇄ CMC / GAS 预测。
- UE 里做帧同步：自写确定性逻辑层，UE 只做表现 / 输入 / 传输；确定性必须在第一天决定。

▶ **自测 1：为什么说“英雄联盟证明了确定性不是帧同步的专利”？**

▶ **自测 2：一个 4 人合作 PVE 游戏，成本敏感、不强调反作弊、大量怪物，怎么选？**

▶ **自测 3：在 Unreal 里用 Reliable RPC 每帧发帧命令有什么问题?**

CHAPTER 15

面试追问 50 题与速记卡

每题先给能直接说出口的答案，括号里是展开时去哪一章找论据和数字。

15.1 一棵追问树

面试官最常见的追问路径

帧同步和状态同步的区别？（Ch1）
- 帧同步为什么要确定性？浮点为什么不确定？（Ch4）
  - 定点数怎么实现乘除 / 开方 / 三角？范围和精度怎么取舍？（Ch4.3）
  - 出了 desync 怎么发现、怎么定位？（Ch4.6 · Ch8.4）
- 一个人卡了，其他人会卡吗？（Ch3）
  - 严格 Lockstep / 定频广播不等人 / 回滚各自怎样？（Ch3.3 实测）
  - 回滚为什么王者不用？（Ch9.4 · Ch3.4）
- 丢包怎么办？（Ch6 · Ch7）
  - 下行：冗余 → 补发 → 追帧，各自的边界？k 设多大？（Ch6.2 推导）
  - 上行：短冗余 vs 技能全量冗余？技能顺序怎么保证？（Ch7）
  - 断线重连怎么做？追帧要多久？（Ch6.5）
- 逻辑和表现怎么分离？（Ch5）
- 状态同步怎么解决延迟？（Ch10）
  - 插值延迟设多少？（Ch10.2）
  - 预测错了怎么办？（Ch10.4 · Ch13.1 CMC）
  - 延迟补偿回溯多少？隔墙杀？（Ch10.5）
  - Unreal：Role / 所有权 / 复制管线 / 属性 vs RPC / GAS 预测（Ch11 · Ch12 · Ch13）

15.2 基础概念（1–10）

▶ **1. 用一句话说清帧同步和状态同步的本质区别。**

▶ **2. 帧同步一定比状态同步省带宽吗？**

▶ 3. 为什么实时游戏多用 UDP 而不是 TCP?

▶ 4. 抖动和延迟哪个更伤体验?

▶ 5. 为什么逻辑要固定步长?

▶ 6. 渲染插值有什么代价?

▶ 7. 客户端怎么和服务器对时?

▶ 8. “权威”是什么意思?

▶ 9. 录像在两种方案里有何不同?

▶ 10. 帧同步能防作弊吗?

15.3 帧同步与 HOK（11–25）

▶ 11. 帧同步的三块基石?

▶ 12. 王者荣耀一个人卡了，其他人会卡吗?

▶ 13. 回滚是不是就没有木桶效应了?

▶ 14. 为什么帧同步要用定点数?

▶ 15. 除了浮点，还有哪些确定性陷阱?

▶ 16. desync 怎么发现和定位?

▶ 17. HOK 为什么让 C++ 独立承担逻辑?

▶ 18. 一条帧命令的“三条命运”?

▶ 19. 为什么技能指示器可以不等帧，伤害飘字必须等?

▶ 20. 下行冗余为什么倒序排列?

▶ 21. 一包带几帧冗余合适?

▶ 22. 900 帧接收窗口做什么？期待帧有什么双重身份?

▶ 23. 补发请求怎么设计?

▶ 24. 断线重连的完整流程?

▶ 25. 上行技能怎么保证不丢、不重、不乱?

15.4 状态同步与 Unreal（26–40）

▶ 26. 状态同步怎么解决延迟?

▶ 27. 插值延迟设多少?

▶ 28. 预测错了怎么办？为什么不能直接覆盖?

▶ 29. 延迟补偿回溯多少？有什么副作用?

▶ 30. Unreal 的 Role 有哪些?

▶ 31. 为什么客户端调 Server RPC 没执行?

▶ 32. GameMode、GameState、PlayerController、PlayerState 分别在哪一端?

▶ 33. 服务器每个网络 tick 的复制流程?

▶ 34. 为什么优先级要乘以等待时间?

▶ 35. 属性复制有哪些语义?

▶ 36. OnRep 在服务器会调用吗?

▶ 37. Reliable RPC 有什么风险?

▶ 38. 开枪特效怎么同步?

▶ 39. CMC 怎么做网络同步?

▶ 40. GAS 怎么预测技能?

15.5 设计与综合（41–50）

▶ 41. 在 CMC 上加冲刺，怎么做才不会被拉回?

▶ 42. 王者荣耀为什么不用回滚?

▶ 43. 格斗游戏为什么要加 2~3 帧输入延迟?

▶ 44. 怎么测 ping 才准?

▶ **45. 玩家反馈“卡”，你怎么定位?**

▶ **46. 让你在 Unreal 里做帧同步，怎么做?**

▶ **47. 设计一个 5v5 MOBA 的网络同步方案。**

▶ **48. 设计一个 5v5 战术射击游戏的网络同步方案。**

▶ **49. 帧同步和状态同步的“回滚”有什么共同点?**

▶ **50. 如果只能记一句话?**

15.6 速记卡

HOK 一句话

服务器定帧汇总 + 三层冗余抗丢包 + 900 帧窗口定序 + 本地确定性模拟 + 逻辑表现彻底解耦。

三条路线

收齐才走（严格）/ 到点就发（不等人）/ 先猜后改（回滚）；只有“不等人”把一人卡顿关在他自己身上。

下行三层

n ≤ k 冗余；k < n < 900 补发 1036；超窗 / 断线 Recover 1090 / 1091 / 1093。

冗余公式

随机丢包 p^k；突发丢包 $p(1-1/L)^{k-1}$——挨得近的副本会一起丢。

上行两种冗余

短冗余：当前 + 前 2 条、约 33 ms；技能全量：所有未确认、约 200 ms、1 s 超时、序号保序。

确定性清单

定点数 · 同种子同调用次数 · 有序容器 · 只用帧号 · 单线程 · 不用引擎物理 · 版本 / 配置校验。

状态同步三件套

插值看别人（约 100 ms）· 预测 + 和解看自己（回到权威 + 重放）· 延迟补偿打别人（回溯 RTT + D，设上限）。

UE 复制管线

频率门控 → 相关性 → 休眠 → 优先级 × 等待时间 → 变化属性 → 饱和停止。

属性 vs RPC

状态用属性（最终一致、可合并、新人可见）；可丢表现用不可靠多播；高频计数用计数器 + OnRep；可靠 RPC 少用。

CMC 五步

SavedMove → 本地执行 → ServerMove（+旧移动）→ Ack / Adjust → 重放未确认移动。

回滚深度

$\approx (\mathrm{RTT}/2 + J)/T - d + 1$；CPU：$t_{sim} \le (T - t_{render})/(D+1)$。

两个旋钮

权威在哪 × 网上传什么；HOK CC / CS、LoL、守望先锋都是中间位置。

APPENDIX

附录

协议与参数速查、Unreal API / 宏 / CVar 速查、术语表，以及对照原文档的勘误与补充。

APPENDIX · A

HOK 协议与参数速查

来自《HOK 帧同步机制详解》附录 A / B，补充了“本书在哪一章讲”和“对应实测”。对外交流时只讲机制，不讲协议号。

A.1 协议

协议号	协议名	方向	用途	本书
1034	`SCID_FRAPBOOT_SINGLE`	S→C	单帧补发	Ch6.3
1035	`SCID_FRAPBOOTINFO`	S→C	**驱动帧下发（含冗余帧）**，补发帧也走它	Ch6.2 · Ch6.4
1036	`CSID_REQUESTFRAPBOOTSINGLE`	C→S	帧补发请求（区间，末区间可为开放区间）	Ch6.3
1037	`CSID_REQUESTFRAPBOOTTIMEOUT`	C→S	超时触发的补发请求	Ch6.3
1075	`SCID_MULTGAME_BEGINLOAD`	S→C	开局（`bCsMode` 决定 CC / CS）	Ch8.6
1076	`CSID_MULTGAME_LOADFIN`	C→S	加载完成	—
1077	`SCID_MULTGAME_BEGINFIGHT`	S→C	开打	Ch6.5
1089	`SCID_MULTGAMERECOVERNTF`	S→C	大厅通知重连 Relay	Ch6.5
1090	`CSID_RECOVERGAMEFRAP_REQ`	C→S	请求缓存帧（进入 Recover）	Ch6.5
1091	`SCID_RECOVERGAMEFRAP_RSP`	S→C	分片返回缓存帧	Ch6.5
1092	`SCID_RECONNGAME_NTF`	S→C	告知重连所处阶段	Ch6.5
1093	`CSID_RECOVERGAMESUCC`	C→S	上报追帧完成	Ch6.5
—	`CSID_GAMING_UPERMSG` / `CSID_AI_UPERMSG`	C→S	玩家 / AI 帧命令上行（UDP）	Ch7
—	`CSID_RELAYSVRPING`	↔	ping（兼带网络统计回传）	Ch8.1
—	`CSID_RELAYHASHCHECK`	C→S	帧 hash 一致性上报	Ch8.4
—	`SCID_FRAPPARAM_NTF`	S→C	窗口 / ping 参数下发	Ch6
—	`SCID_SYNCGAMECORE_FRAPINFO`	S→C	CS 模式 / 录像帧信息	Ch8.6
—	`CSID_GAMECORE_CLT_INPUTMSG`	C→S	CS 模式下攒批的逻辑输入	Ch5.5 · Ch8.6

A.2 参数

参数	默认值	来源	本书的相关实测 / 推导
服务器 / 客户端帧间隔	66 ms（约 15 帧/秒）	逻辑层开局下发	E2 插值滞后、E3 三路线
接收窗口长度	900 帧（约 60 秒）	常量	图 6-3（缩小演示）
补发超时阈值	90 ms	服务器下发	E3 模型同参数
紧急 / 惰性上报频率	120 ms / 1000 ms	服务器下发	E3 断流恢复时间
短冗余重发间隔	33 ms	服务器下发	E7
技能全量冗余重发间隔 / 指令超时	200 ms / 1000 ms	服务器下发	E7
补发区间数上限	协议上限 − 1（留 1 给窗口外）	协议常量	图 6-3 ⑧
ping 间隔	0.5 s	服务器下发	E8
卡顿 / 恢复判定阈值	300 ms / 70 ms	可配	图 8-1
追帧完成阈值	15 帧（快速恢复 3 帧）	常量	Ch6.5 追帧时长估算
Recover 请求超时 / 连接重连上限	5 s / 8 次	常量	图 6-4、6-5
反向通道单帧处理上限	1000 条	服务器可配	Ch5.5
双链路冷却	30 s	—	Ch8.3

APPENDIX · B

Unreal 网络 API / 宏 / CVar 速查

按“写代码时会查”的顺序整理；版本相关项已标注。

类别	名称	说明
属性	`UPROPERTY(Replicated)` / `ReplicatedUsing=OnRep_X`	复制属性 / 带通知
	`GetLifetimeReplicatedProps` + `DOREPLIFETIME` / `_CONDITION` / `_CONDITION_NOTIFY` / `_WITH_PARAMS_FAST`	注册；条件见 Ch12.1；`REPNOTIFY_Always` 每次都通知
	`FDoRepLifetimeParams::bIsPushBased` + `MARK_PROPERTY_DIRTY_FROM_NAME`	Push Model（需开启 `Net.IsPushModelEnabled`）
	`FFastArraySerializer` / `MarkItemDirty` / `PostReplicatedAdd/Change` / `PreReplicatedRemove`	大数组增量复制与逐元素回调
	`FVector_NetQuantize` / `NetQuantize10` / `NetQuantize100` / `NetQuantizeNormal`	量化向量
RPC	`UFUNCTION(Server | Client | NetMulticast, Reliable | Unreliable, WithValidation)`	实现 `_Implementation`；校验 `_Validate`（返回 false 会断开该客户端）
	所有权	`SetOwner` / `GetOwner` / `GetNetConnection`；Server RPC 只能由拥有者连接发起
	可靠缓冲	可靠 RPC 堆积超过上限会关闭连接；不要在 Tick 里发 Reliable
Actor 复制设置	`bReplicates` / `SetReplicates`	开启复制
	`SetReplicateMovement` / `ReplicatedMovement`	复制位置 / 旋转 / 速度
	`NetUpdateFrequency` / `MinNetUpdateFrequency`	默认 100 / 2；UE 5.5 起用 `SetNetUpdateFrequency()` 等访问器
	`NetPriority` / `GetNetPriority()`	带宽竞争权重 × 等待时间
	`bAlwaysRelevant` / `bOnlyRelevantToOwner` / `bNetUseOwnerRelevancy` / `IsNetRelevantFor()`	相关性

	`NetCullDistanceSquared`	默认 225000000（150 m）
	`NetDormancy` / `FlushNetDormancy()` / `ForceNetUpdate()`	休眠与强制更新
	`PreReplication` / `DOREPLIFETIME_ACTIVE_OVERRIDE`	每次复制前动态开关属性（配合 `COND_Custom`）
	`bReplicateUsingRegisteredSubObjectList` / `AddReplicatedSubObject`	注册式子对象复制（Iris 必需）
Role / 模式	`GetLocalRole()` / `GetRemoteRole()` / `HasAuthority()`	Authority / AutonomousProxy / SimulatedProxy
	`IsLocallyControlled()`	是否由本机玩家控制（Listen Server 主机判断常用）
	`GetNetMode()`	Standalone / DedicatedServer / ListenServer / Client
	`AGameStateBase::GetServerWorldTimeSeconds()`	近似服务器时间（未扣单程延迟，Ch2.4）
CMC	`FSavedMove_Character`：`Clear / SetMoveFor / PrepMoveFor / GetCompressedFlags / CanCombineWith`	自定义可预测移动
	`UpdateFromCompressedFlags` / `FLAG_Custom_0~3` / `FCharacterNetworkMoveData`	服务器侧恢复输入；更多数据用自定义移动数据（4.26+）
	`AGameNetworkManager::MAXPOSITIONERRORSQUARED` / 时间差异检测	纠正阈值、防加速
	`NetworkSmoothingMode`	模拟代理平滑：Disabled / Linear / Exponential
GAS	`FPredictionKey` / `ReplicatedPredictionKey`	预测对账与撤销
	`EGameplayAbilityNetExecutionPolicy`	LocalPredicted / LocalOnly / ServerInitiated / ServerOnly

	`ServerTryActivateAbility` / `ClientActivateAbilitySucceed / Failed` / `ServerSetReplicatedTargetData`	激活与目标数据
调试	`net PktLag=N` / `PktLagVariance=N` / `PktLoss=N` / `PktOrder=1` / `PktDup=N`	模拟延迟、抖动、丢包、乱序、重复
	`p.NetShowCorrections 1` / `p.NetCorrectionLifetime`	画出 CMC 纠正
	`stat net` / Network Insights（`-NetTrace=1 -trace=net`）	带宽、包、每 Actor 复制开销
	`NetServerMaxTickRate`（IpNetDriver，默认 30）	服务器网络 tick 上限
	`net.Iris.UseIrisReplication`	运行时启用 Iris（需编译时 `bUseIris` 与插件）
	Replication Graph（插件）	大规模相关性；与 Iris 不兼容
	Physics Prediction 设置（项目设置 → Physics）	启用网络物理预测、最大支持 RTT、重模拟误差阈值（5.4+，实验性 / Beta）

APPENDIX · C

术语表

中英对照，按出现顺序。

术语	英文	一句话
帧同步	Lockstep / Frame Synchronization	各端跑相同确定性逻辑、靠相同输入达成一致
状态同步	State Synchronization / Replication	服务器权威计算，复制状态与事件
确定性	Determinism	相同输入 → 逐位相同的输出
不同步	Desync	各端结果分歧并逐帧放大，无法自愈
定点数	Fixed-point（Q16.16 / Q32.32）	用整数表示小数，处处一致
严格 Lockstep / 延迟型	Deterministic Lockstep / Delay-based	收齐才推进 / 用输入延迟吸收抖动
定频广播不等人	Fixed-rate relay, no waiting（Time Bucket）	服务器到点就发，谁没到就没有谁
回滚	Rollback（GGPO）	先预测推进，真值不符时回到分歧帧重算
冗余帧 / 帧空洞	Spare Frame / Frame Hole	一包多帧抗丢包 / 期待帧缺失而后续已到
突发丢包	Burst Loss（Gilbert-Elliott）	丢包成串出现，冗余副本会一起丢
追帧	Recover / Catch-up	拉服务器缓存帧批量重放，追上实时
预表现	Pre-presentation	不依赖演算结果的反馈按键即响应
快照插值	Snapshot Interpolation	渲染过去一段时间的状态，在快照之间插值
外推 / 航位推算	Extrapolation / Dead Reckoning	按速度往前推，转向时会过冲
客户端预测 / 和解	Client-side Prediction / Reconciliation	本地先动；收到权威状态后重放未确认输入
延迟补偿	Lag Compensation	服务器回溯到射手看到的时刻判定命中
先手优势	Peeker's Advantage	探头者先看到对方（对方看到的是插值后的过去）

兴趣管理 / 相关性	Interest Management / Relevancy	只把相关对象发给某个客户端
休眠	Net Dormancy	不变的 Actor 停止复制检查
自主代理 / 模拟代理	Autonomous Proxy / Simulated Proxy	本地控制并预测的副本 / 只做插值播放的副本
推送模型	Push Model	属性修改时显式标脏，而不是每次轮询比较
预测键	Prediction Key（GAS）	把一次预测的所有副作用挂在一起，整体确认或撤销
重模拟	Resimulation（Chaos 网络物理）	物理状态偏差超过阈值时回滚重算
CC / CS 模式	Client-Client / Client-Server（HOK）	逻辑在本地 / 逻辑在服务器
AGE	—（HOK）	表现时间轴系统：C++ 决策、C# 执行

APPENDIX · D

对照原文档：勘误与补充

《HOK 帧同步机制详解》整体准确、结构清晰。以下是可以用实测或公开资料补充、或表述需要收紧的几处。

#	原文位置与说法	问题	更准确的说法	依据
1	1.5 节表格、2.4.5 节总对照：回滚“一人卡顿的影响 = 只有他自己画面跳 / 仅自己画面跳”	① 卡顿者网络差时，是**其他人**对他的输入预测失败，其他人看到他的角色跳变；② 卡顿超过预测窗口后，P2P 回滚会让**所有人**停下等待	短抖动：别人看到卡顿者轻微跳变；长断流：全员等待（木桶效应被推迟而非消灭）。原文 2.5.4 节对第 ② 点的描述是正确的，两处表格与之不一致	E3（图 3-3：好玩家在回滚模型下也卡了 1710 ms）
2	20.1 节表格：CC 的优势“手感直接”	定频广播模型下，自己的操作要经服务器往返、等下一次广播才进入逻辑	操作延迟 ≈ 自己到服务器的往返 + 等待服务器 tick（E3 约 112 ms）；“手感直接”来自预表现（指示器、按钮反馈本地响应），这一点原文 6.4.5 节已经说明	E3 · Ch5.2
3	10.3 节：冗余能抵抗“短时间内连续丢 2~3 个包（只要不超过冗余长度 k）”	正确，但没有量化 k 的收益	随机丢包下每多带一帧失败率乘 p；突发丢包下只乘 $1-1/L$（L = 平均连丢长度）。移动网络的突发丢包让“加大 k”收益很低，这正是补发存在的理由	E5 + 推导（Ch6.2）
4	18.1 节：主线程测 ping“表现出来就是帧率越低 ping 越高”	方向正确，但偏差并不单调取决于帧率	平均偏差上限约一个渲染帧，并与“RTT ÷ 帧长”的余数有关（ping 在帧末发出、下一个帧首取包）；真正把偏差放大的是**长帧**（p95 可达 97 ms）	E8（表 8-1）
5	14.3 节：技能全量冗余“每次发送所有未确认的技能”	容易被理解为“比短冗余更费带宽”	因为“确认了就停”，全量冗余平均每个技能只发 1.26~1.39 份，少于短冗余的 3 份；代价是尾延迟（p99 772 ms）	E7
6	1.2 节：状态同步“每帧全量下发不可接受”	结论正确，但状态同步通常不会全量下发	实际是增量 + 量化 + 兴趣管理 + 优先级；100 单位时增量 + AOI 约为帧同步的 2 倍，而不是 30 倍	E1（表 1-1）

7	2.2 节：“种子随帧下发”与 19.1 节“上报附带随机种子”	不是错误，补充一个更隐蔽的坑	种子一致只是必要条件，**随机调用次数**也必须一致：表现层或“只对本地玩家执行”的代码调用了逻辑随机，会让序列错位	Ch4.4

说明

以上“依据”中的 E / D 编号指本书 `demos/` 下的模型实验：参数为假设值，用于说明趋势与量级，不代表 HOK 线上数据。HOK 相关的事实只来自原文档；Unreal 与业界资料以官方文档、源码和公开演讲为准，版本相关内容已标注。